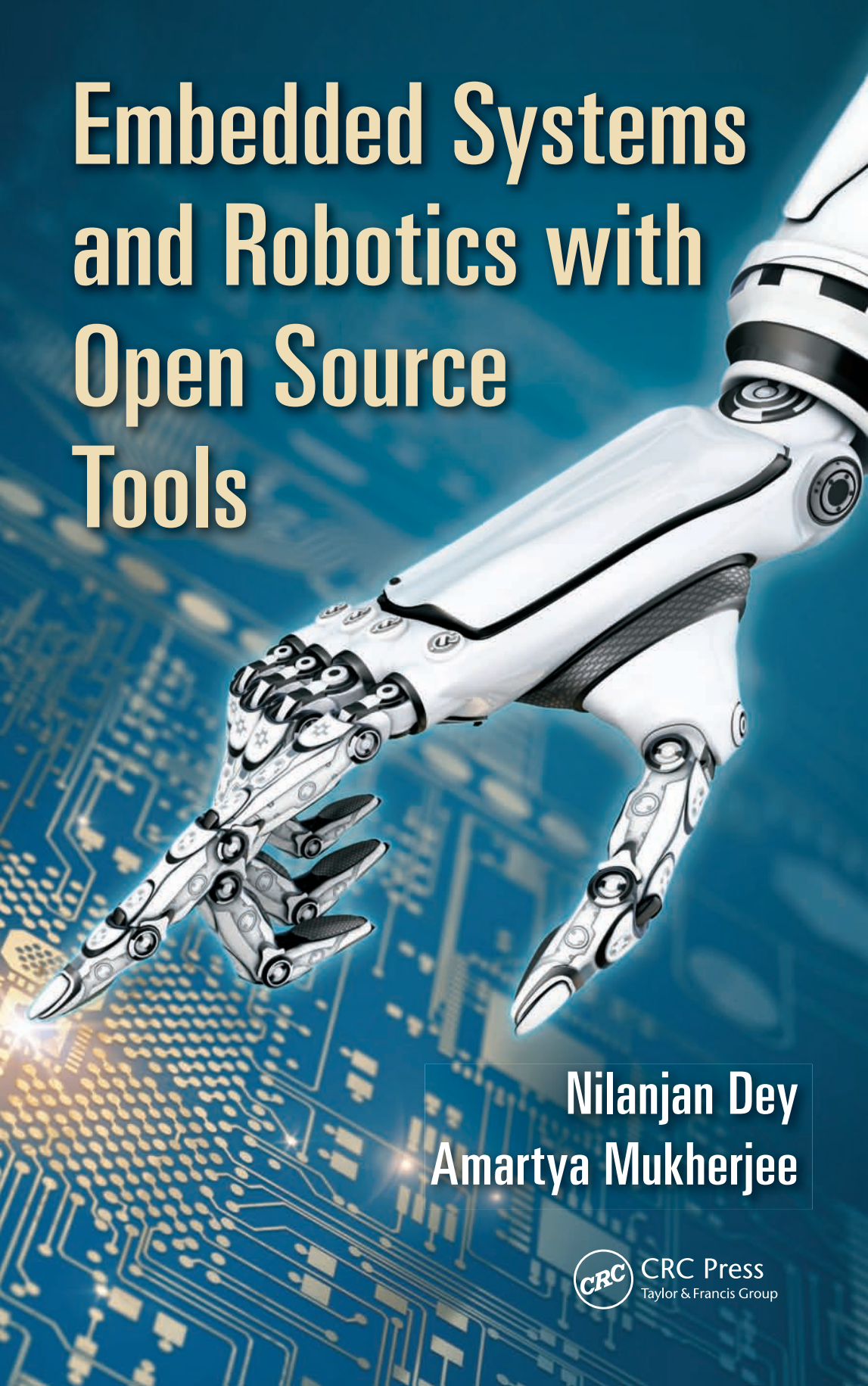


Embedded Systems and Robotics with Open Source Tools

A white robotic arm with black joints and sensors is shown reaching down towards a glowing blue circuit board. The background is a deep blue with intricate white circuit patterns. The arm's hand is open, with fingers slightly curled. The overall image conveys a sense of advanced technology and robotics.

Nilanjan Dey
Amartya Mukherjee



CRC Press
Taylor & Francis Group

Embedded Systems and Robotics with Open Source Tools

Embedded Systems and Robotics with Open Source Tools

Nilanjan Dey
Amartya Mukherjee



CRC Press

Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2016 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works
Version Date: 20160301

International Standard Book Number-13: 978-1-4987-3440-0 (eBook - PDF)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

*In loving memory of the
late Mihir Kumar Mukherjee*

When the tools of production are available to everyone, everyone becomes a producer.

Ð Chris Anderson

Contents

Preface.....	xv
Acknowledgments	xvii
Authors	xix
1. Introduction	1
1.1 Embedded Systems and Robotics	1
1.2 Fundamental Goal of Embedded Systems	1
1.3 Fundamental Goal of Robotics	2
1.4 Main Focus	2
1.5 Motivation	3
1.6 How to Use This Book	3
2. Basics of Embedded Systems	5
2.1 Introduction	5
2.2 Classifications of Embedded Systems	5
2.3 Microprocessors	6
2.4 Microcontrollers	8
2.5 Application-Specific Processors	9
2.6 Sensors and Actuators	11
2.6.1 Sensors	11
2.6.2 Examples of Sensors	11
2.7 Embedded Communication Interface	12
2.7.1 I2C Communication	12
2.7.2 SPI and SCI Communication	13
2.7.3 UART Communication	13
2.7.4 USB Communication	14
2.8 Real-Time Operating Systems	15
2.8.1 Hard Real-Time System	15
2.8.2 Soft Real-Time System	16
2.8.3 Thread-Oriented Design	16
2.9 Typical Examples	16
2.9.1 Smartphone Technology	16
2.9.2 Aircraft Autopilot Unit	17
3. Basics of Robotics	19
3.1 Introduction	19
3.2 Robot Kinematics	19
3.3 Degree of Freedom	20
3.4 Forward Kinematics	22
3.5 Algebraic Solution	22

3.6	Inverse Kinematics.....	23
3.7	Robots and Sensors.....	24
3.7.1	Motion Detection Sensor	24
3.7.2	Gyroscope and Accelerometer	24
3.7.3	Obstacle Detector	25
3.7.4	Location Tracking by GPS	25
3.8	Robots and Motors.....	26
3.8.1	DC Motor	27
3.8.2	Servo Motor	28
3.8.3	Stepper Motor.....	29
3.9	Robot Controller	29
3.10	Frames and Materials	30
3.11	Types of Robots	30
3.11.1	Industrial Robots	31
3.11.2	Medical Robots.....	31
3.11.3	Military Robots	32
3.11.4	Space Robots	33
3.11.5	Entertainment Robots	35
3.12	Summary	35
4.	Aerial Robotics.....	37
4.1	Introduction to Aerial Robotics	37
4.2	History of Aerial Robotics	37
4.3	Classification of Aerial Robots	38
4.3.1	Fixed-Wing Systems	38
4.3.2	Multirotor Systems	40
4.4	Sensors and Computers	41
4.5	Open Research Area	43
4.6	Aerial Sensor Networks	43
5.	Open-Source Hardware Platform	45
5.1	Introduction	45
5.2	Open-Source Hardware Features	45
5.3	Open-Source Hardware Licensing	47
5.4	Advantages and Disadvantages of Open-Source Hardware	47
5.5	Examples of Open-Source Hardware.....	48
5.5.1	Raspberry Pi Computer	48
5.5.2	BeagleBoard	49
5.5.3	PandaBoard.....	50
5.6	Summary	51
6.	Open-Source Software Platform	53
6.1	Introduction	53
6.2	Open-Source Standards	53
6.2.1	Open-Source Software Licensing.....	54
6.2.2	Free and Open-Source Software.....	54

6.3	Examples of Open-Source Software Products	55
6.4	Advantages and Limitations of Open-Source Software.....	56
6.5	Open-Source Future	58
7.	Automated Plant-Watering System.....	59
7.1	Introduction	59
7.2	Architecture of Plant-Watering Systems	59
7.2.1	Soil Moisture Sensor.....	60
7.2.2	Setting Up 433 MHz Radio Tx/Rx Module.....	61
7.2.3	Setting Up the Pumping Device.....	62
7.3	Arduino Programming Code.....	63
7.3.1	Arduino Code for the Radio Transmitter	63
7.3.2	Arduino Code for the Radio Receiver.....	64
7.4	Broadcasting Sensor Data to the Internet via Processing	65
7.5	Summary	69
7.6	Concepts Covered in This Chapter	69
8.	Device to Cloud System	71
8.1	Introduction	71
8.2	Temperature Sensor Data Logging System.....	71
8.2.1	Interacting with Cloud	71
8.3	Components	73
8.4	Temperature Sensor	73
8.5	Circuit Connections	75
8.6	Setting Up Zigbee Communication.....	76
8.6.1	Zigbee Basics	76
8.6.2	Configuring XBee Module.....	78
8.7	Sample Python Code for Serial Read	80
8.8	Sending Data to Cloud	80
8.8.1	More about Raspberry Pi.....	82
8.8.2	Main Components	83
8.9	Installation of Operating System and Python API in Raspberry Pi	83
8.9.1	OS Installation.....	83
8.9.2	pySerial Installation	84
8.9.3	Python Google Spreadsheet API Installation	84
8.10	Configuring Google Account	85
8.11	Python Code to Access Google Spreadsheet.....	86
8.12	Summary	87
8.13	Concepts Covered in This Chapter	88
9.	Home Automation System	89
9.1	Introduction	89
9.2	Home Automation System Architecture	89
9.3	Essential Components	89

9.4	Connection Detail	91
9.5	Setting Up the Web Server.....	92
9.6	Interaction with Server by Processing	95
9.7	Summary	100
9.8	Concepts Covered in This Chapter	100
10.	Three-Servo Ant Robot	101
10.1	Introduction	101
10.2	Tools and Parts Required.....	101
10.2.1	Ultrasonic Sensor	101
10.2.2	Servomotors.....	102
10.2.3	Leg Design	103
10.2.4	Mounting Ultrasonic Sensor	106
10.3	Programming the Leg Movement	106
10.4	Summary	110
10.5	Concepts Covered in This Chapter	110
11.	Three-Servo Hexabot.....	111
11.1	Introduction	111
11.2	System Architecture	111
11.3	Parts and Their Assembly.....	112
11.4	Programming Basic Moves.....	115
11.5	Summary	118
11.6	Concepts Covered in This Chapter	119
12.	Semi-Autonomous Quadcopter	121
12.1	Introduction	121
12.2	Structural Design.....	121
12.3	Component Description.....	122
12.4	Flight Controller Unit.....	124
12.4.1	MultiWii CRIUS SE2.5.....	124
12.4.2	Flight Controller Comparison.....	125
12.5	Assembling Parts	125
12.6	Sensor and Speed Controller Calibration.....	128
12.6.1	MultiWii Setup and Configuration	128
12.6.1.1	Configuring MultiWii Firmware	128
12.6.1.2	Sensor Calibration.....	129
12.6.1.3	ESC Calibration	131
12.6.2	Configure KK 5.5 Multicopter Board	131
12.7	Radio Setup and Calibration.....	132
12.8	Radio TX/RX Binding Technique.....	133
12.9	Connection with GUI Interface.....	134
12.9.1	PID Tuning.....	136
12.9.1.1	Basic PID Tuning.....	136
12.9.1.2	Advanced PID Tuning	136

- 12.9.1.3 Standard Guideline for PID Tuning 138
 - 12.9.1.4 General Guidelines 138
 - 12.10 Position, Navigation, Level, and Magnetometer Performance Tuning 139
 - 12.11 Additional Channel Assignments 140
 - 12.12 Summary 141
 - 12.13 Concepts Covered in This Chapter 142
- 13. Autonomous Hexacopter System..... 143**
 - 13.1 Introduction 143
 - 13.2 Structural Design of the Autonomous Hexacopter..... 143
 - 13.3 Components 143
 - 13.3.1 Frames 144
 - 13.3.2 Motors and ESC..... 144
 - 13.3.3 Radio Units 145
 - 13.3.4 Autopilot Unit..... 147
 - 13.4 Component Assembly 148
 - 13.5 APM Ground Station Software Installation..... 150
 - 13.6 APM Firmware Loading 152
 - 13.7 Sensor and Radio Calibration 152
 - 13.7.1 Accelerometer and Gyroscope Calibration 152
 - 13.7.2 Compass Calibration 153
 - 13.7.3 Radio Calibration 154
 - 13.7.4 ESC Calibration 154
 - 13.7.5 Motor Test 155
 - 13.8 Flight Parameter Settings 155
 - 13.9 Flight Modes 156
 - 13.10 Mission Design 157
 - 13.10.1 Using Ground Station..... 157
 - 13.10.2 Waypoint Navigation Algorithm..... 158
 - 13.10.3 GPS Glitch and Its Protection..... 160
 - 13.11 Adding FPV Unit..... 161
 - 13.12 Final Hexacopter UAV 162
 - 13.12.1 Flight Path Visualization and Log Analysis 162
 - 13.13 Summary 164
 - 13.14 Concepts Covered in This Chapter 164
- 14. Conclusion..... 165**
 - 14.1 Tools Used 165
 - 14.2 Important Safety Notes 166
 - 14.3 Frequently Asked Questions..... 168
 - 14.4 Final Words..... 172
- Bibliography..... 173**

Preface

In the world of computer science, software and hardware are deeply inter-related. A computer system is a combination of the functions of several electronic devices that act collaboratively with the help of software systems. Nowadays, the computer system is not limited to a desktop PC, laptop, palm-top, or a workstation server. The definition of a computer has been changed by the smart phone revolution. Starting from a basic video-gaming device to a more sophisticated unmanned aerial vehicle, everywhere we realize the presence of high-performance embedded computing.

This era is also well known for the open-source revolution. Technological enhancements have been achieved through both open-source software and hardware platforms. One of the very popular tools today is the rapid prototyping environment, which consists of a combination of hardware and software suites. With the help of high-performance microprocessors, microcontroller, and highly optimized algorithms, one can develop smarter embedded applications.

This book aims to present some cutting-edge open-source software and hardware technology and the practical applications of such smarter systems that take the technology to the next level. The chapters are designed in a way to help readers who are not familiar with advanced computing technologies easily understand and learn as they read deeper into the book. The book includes eight high-end, real-time projects for evaluation of the rapid prototype development skill. These projects are properly verified and tested so that one can easily deploy them soon after learning. The book will serve as a guide to undergraduate and postgraduate engineering students, researchers, and hobbyists in the field.

Nilanjan Dey

Amartya Mukherjee

Acknowledgments

This book itself is an acknowledgment to the technical and innovative competence of many individuals who have contributed to this domain. First, we thank our colleagues and coresearcher(s), especially Sayan Chakrabarty, Souvik Chatterjee, and Soumya Kanti Bhattacharaya, for their technical support in all regards. We thank Dr. Amira S. Ashour, vice-chairperson, Department of Computer Science, College of Computers and Information Technology, Taif University, Taif, Kingdom of Saudi Arabia, for extending her expertise in upgrading the literary quality of this book. We also thank Eshita Mazumder Mukherjee for her support in writing the book and our students Anant Kumar, Manish Kumar, and Masoom Haider.

Finally, we thank our parents, wives, and children for their continuous support.

Authors



Nilanjan Dey is an assistant professor in the Department of Information Technology, Techno India College of Technology, Rajarhat, Kolkata, India. He holds an honorary position of visiting scientist at Global Biomedical Technologies Inc., California, and research scientist at the Laboratory of Applied Mathematical Modeling in Human Physiology, Territorial Organization of Scientific and Engineering Unions, Bulgaria. He is the editor in chief of the *International Journal of Rough Sets and Data Analysis*, IGI Global, US; managing editor of the *International Journal of Image Mining* (IJIM), Inderscience; regional editor (Asia) of the *International Journal of Intelligent Engineering Informatics* (IJIEI), Inderscience; and

associate editor of the *International Journal of Service Science, Management, Engineering, and Technology*, IGI Global. His research interests include medical imaging, soft computing, data mining, machine learning, rough set, mathematical modeling and computer simulation, modeling of biomedical systems, robotics and systems, information hiding, security, computer-aided diagnosis, and atherosclerosis. He has published 8 books and 160 international conferences and journal papers. He is a life member of the Institution of Engineers, Universal Association of Computer and Electronics Engineers, Internet Society as a Global Member (ISOC), etc. Detailed information on Nilanjan Dey can be obtained from <https://sites.google.com/site/nilanjandeyprofile/>.



Amartya Mukherjee, MTech, is an assistant professor at the Institute of Engineering and Management, Salt Lake, Kolkata, India. He holds a bachelor's degree in computer science and engineering from West Bengal University of Technology and a master's degree in computer science and engineering from the National Institute of Technology, Durgapur, West Bengal, India. His primary research interest is in embedded application development, including mobile ad hoc networking, aerial robotics, and Internet

of Things. He has written several papers in the field of wireless networking and embedded systems.

1

Introduction

1.1 Embedded Systems and Robotics

Embedded systems and robotics are the most interrelated terms in this cutting-edge technological era. The revolution of smartphone, smart real-time operating system (RTOS), and system-on-chip technology provides a new dimension to the embedded hardware. In the past, the embedded system was a bit complicated to manage and a huge chunk of assembly-level code was to be written to program the whole system. But as things keep changing quite drastically, nowadays embedded systems act as a platform in the development of software/firmware, thus reducing the development time. The architecture of the system also keeps changing day by day so as to increase processing power and to decrease energy consumption. The enhancement of the RTOS-like Android gives another new dimension to embedded systems. On the contrary, robotics has evolved to higher-dimensional applications. In earlier years, robots were only used in industrial and scientific research, but today robotics has reached a new dimension, thanks to open-source hardware; starting from military to medical applications or maybe for entertainment or as a hobby, the concept of robotics has been widely spread. Robotics experts claim that by 2022 they will produce a robotic maid that will cost less than \$100,000.

1.2 Fundamental Goal of Embedded Systems

The growth of embedded systems depends on innovative engineers with exposure to robotic technology. Loosely defined, an embedded system, which is a computer system that is intended to be a general-purpose computer, is a programmable device that drives some specific set to the system. It might be connected with one or more number of sensors and actuators. The main task of the embedded system is to acquire data from the sensor. The system should be smart enough to process and analyze the data using its

own computing device (i.e., essentially a minimum level of artificial intelligence). Finally, it should take some decisions so that a task can be performed in a precise manner. In this context, it can be said that the work might be physical or it may be the control signal that has been imposed on any other device. Basically, this system must provide an output that performs a job in a highly accurate way. Let us take an example of an automated washing machine system. Obviously, it performs the washing based on the clothes that have been fed to it. When there are fewer clothes and an instruction of turbo clean mode is given, the machine checks the water level and the amount of clothes. Then it decides upon the speed of the motor in revolutions per minute and the time required to spin the motor so that it can clean the clothes completely. The main target of the system is fixed in such a way that the system performs in a precise manner to achieve the target.

1.3 Fundamental Goal of Robotics

In recent years, the significance of the robotics domain has increased a lot. Robotics contributes novel benefits in various disciplines and applications. Although robotics and embedded systems are quite interrelated, robots are a concept through which the world can be changed dramatically. The fundamental objective of the development of robotics is to minimize human effort as well as to perform a precise job that can overcome human error. Robots are defined as artificial beasts that can perform huge work within a very short duration of time. A robot is intended to be used for the service of the society; the abuse of a robotic system might create a huge catastrophic situation. Recently, the robotic system of a car manufacturing company in Germany crushed an operator to death due to malfunction. To avoid such a situation, proper education on robot handling is necessary; proper safety measures are to be ensured at all places where robots are widely used. Furthermore, robotics research is a never-ending process. A lot of work using robotics is in progress in the military, space, and medical domains, and more applications are expected for smart robotics system in the near future.

1.4 Main Focus

This book mainly focuses on the approaches and various methods to assist in the implementation of physical devices and gadgets via properly utilizing open-source embedded hardware as well as software tools.

The primary objective of this book is to provide knowledge about the systems and their interaction in a very rapid manner. College or research students can easily build and cultivate their knowledge via open-source tools mentioned in the book as they are highly emphasized on an application level rather than on a theory level. In addition, this book focuses on the main functional area of the open-source system and its interaction with different components. Finally, as one of its most prime objectives, this book aims to provide guidance in implementing device-based embedded systems.

1.5 Motivation

The main motivation of this book is the implementation of embedded systems while learning. The interactive feature is mostly emphasized and is the primary feature of this book. Various projects have been discussed here, which completely provide hands-on experience of learning. The revolution of open-source hardware is another key motivation of this book. All the software and hardware tools used in this book are mostly open source in nature. The promotion of open-source software and hardware technology is one of the key objectives of this project.

1.6 How to Use This Book

This book serves as a guide and reference for open-source projects. No theory-based approach has been made in this book as effective learning can be achieved only via real-world implementation. Most of the components used in this book are available in local stores. The components for making an unmanned aerial vehicle are rarely available in the market, but if the reader wants to develop one, the available resources and components can be found in the web link mentioned in the book.

This book has been organized in the following way. Chapter 2 describes the fundamentals of embedded systems, Chapter 3 provides the knowledge about the “building blocks” of robotics, and Chapters 3 and 4 give a brief description of aerial robotics. Chapter 5 is all about open-source hardware platforms. Chapter 6 provides the knowledge on open-source software and its features. In Chapters 7 through 13, some of the most interesting hands-on projects starting from amateur to professional levels are provided.

2

Basics of Embedded Systems

2.1 Introduction

Today, we are in an era of smart devices as embedded technology is involved in various applications that we use in our daily life by the virtue of microprocessors and microcontrollers. The system might consist of only electronic or electromechanical devices. Since this work is concerned with the application of these technologies, we mainly focus our discussion on several microcontrollers and the embedded system development environments. An embedded system might be a real-time system that performs mission-critical tasks. Most embedded systems are based on sensors and output actuators. A sensor typically examines the behavior of the outside world and sends the information to an embedded microcontroller system. It is typically either digital or analog in nature. An analog sensor sends a voltage level corresponding to the sensed data value, whereas a digital sensor sends a digital pulse-width modulation (PWM) or pulse-position modulation (PPM) pulse corresponding to the sensed value. An actuator can be considered as an output device that responds to the behavior sensed by the sensor device. It may typically be a manipulator, a robotic arm, or a relay-based device that performs a real-time task based on the given sensor data.

2.2 Classifications of Embedded Systems

Typically, embedded devices can be categorized into several classes. Such classifications are based on the processing power, cost, functionality, and architecture. The typical classifications are as follows.

1. *Small-scale embedded system*: A small-scale embedded system is mostly based on either 8- or 16-bit architecture. It generally runs on 5 V battery power, having limited processing power and memory. It commonly uses small-size flash memory or electrically erasable

programmable read-only memory (EEPROM) to store programs and instructions. The system itself is less complicated than other high-end systems. Generally, C language is preferred to program such an embedded system. The device programmer generates the assembly-level instructions and feeds it to the memory of the system. To develop such a system, board-level design is preferred rather than chip-level design.

2. *Medium-scale embedded system*: This type of system is mostly used for the purpose of digital signal processing (DSP). Mostly, 16±32-bit processor architecture is used in such systems. This system supports complex software and hardware design and needs integrated development environments, debuggers, simulators, and code engineering tools to install and analyze the software. Reduced instruction set computing (RISC) is the most preferable architecture in such a case, which supports the transmission control protocol/Internet protocol (TCP/IP) stacking and networking. Rapid prototyping devices, advanced RISC machine (ARM)-based smart data acquisition systems, and automation systems are examples of such embedded systems.
3. *Sophisticated embedded systems*: This type of embedded system has high hardware and software configuration. Both complex instruction set computing (CISC) and RISC architectures are supported by such systems. Most of these systems have higher random-access memory (RAM) configuration. It supports the system-on-chip (SOC) concept. The software that runs on embedded systems is mostly a real-time operating system (RTOS) that supports the TCP/IP network protocol. More high-end applications such as high-definition graphics-based media and gaming have been supported by these systems, for example, smartphones, smart televisions, tablet PCs, and high-end gaming devices such as PlayStation and Xbox.

2.3 Microprocessors

A microprocessor, shown in Figures 2.1 and 2.2, is a digital electronic device having miniaturized transistors, diodes, and integrated circuits (ICs). It generally consists of an arithmetic logic unit (ALU), control unit, registers, and several data and address buses. Microprocessors generally execute a set of instructions in their ALU controlled by the timer clock generated by the control unit of the microprocessor. A microprocessor can be connected with several memory and input/output (IO) devices. Generally, a microprocessor has many register pairs internally connected with it. The instructions executed on the microprocessor are generally fetched from the memory to



FIGURE 2.1
8085 microprocessor package.



FIGURE 2.2
Motorola 68000.

the register pairs. The results are computed by an ALU, and the final value is stored into the registers and then transferred to memory. Typically, the Intel 8085 microprocessor has an accumulator register, BC, DE, and HL register pair. Along with that, a program counter register is available to store the address of the next instruction. A stack pointer register is also available to store the address of the top of the stack. A flag register is dedicated to set up the status of the computation of the microprocessor instruction. The 8085 microprocessor contains an 8-bit data bus and a 16-bit address bus to fetch addresses and data where higher-order buses are common for both address and data transfer.

On the other hand, the Motorola 68000 (often known as m68k) is a 16-/32-bit processor that supports the CISC architecture. It supports a 32-bit instruction set and runs in a 20 MHz clock. The processor has eight 32-bit data registers and eight 16-bit address registers from which the last register is treated as a stack pointer. The 68000 microprocessor was considered the most successful microprocessor in the 1980s era. The first laser printer was developed with the help of this processor, as HP's first laser printer had also used an 8 MHz 68000 microprocessor in 1984.

2.4 Microcontrollers

Microcontrollers, shown in Figures 2.3 and 2.4, are often known as micro-computers and are used in embedded applications in most cases. A microcontroller is an integrated chip that contains a processor, memory, and programmable input and output ports often called general-purpose input/output (GPIO). In general, a microcontroller may have a very small size RAM, a programmable ROM memory, as well as flash memory to store programs and instructions.

Apart from a microprocessor, a microcontroller has power to perform real-time tasks with the help of the embedded software. Microcontroller devices are used in many applications ranging from a very tiny digital clock to a huge industrial automation system. Various classes and specifications of microcontrollers are being used nowadays. One of the most popular among them is Intel 8051. This microcontroller has an 8-bit ALU, 8-bit register, 128-bit RAM, and 4 kB ROM. Microcontrollers of this category consist of one or two universal asynchronous receiver±transmitter (UART) controllers for asynchronous communication between the controller and peripheral devices. The Intel 8051 microcontroller normally runs at about a clock frequency of 12 ± 16 MHz,

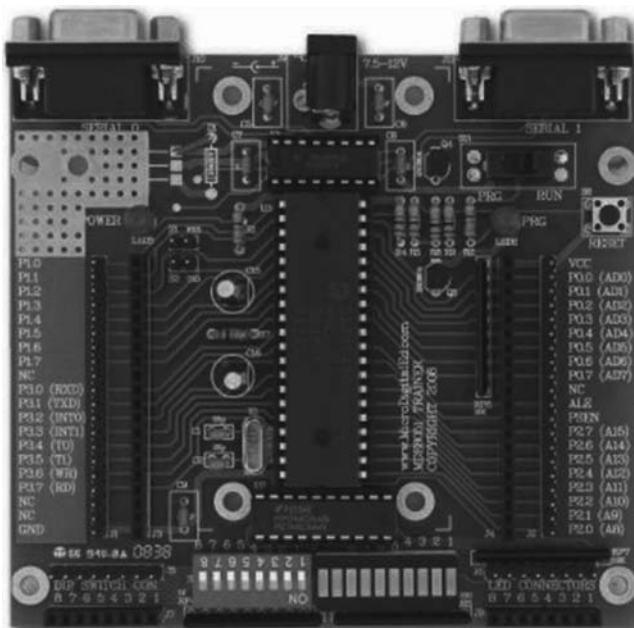


FIGURE 2.3

A microcontroller board. (From diligent.com. <https://www.digilentinc.com/Products/Detail.cfm?NavPath=2,398,1015&Prod=MDE8051>)



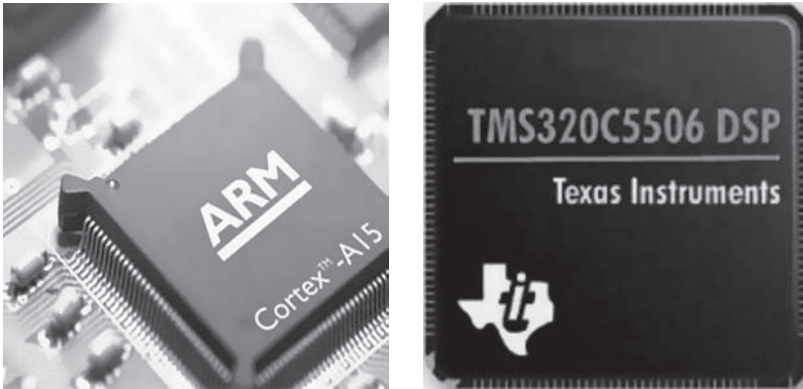
FIGURE 2.4
ATMEGA microcontroller.

but the current advancement of the sophisticated core of the controller runs at a 100 MHz clock rate. Different 8051 variants support a chip oscillator, self-programmable flash memory, additional internal storage, I2C, serial peripheral interface (SPI), and universal serial bus (USB) interface. The controller may also support the ZigBee and Bluetooth module interfacing.

Another modified Harvard 8-bit RISC single-chip architecture is the Atmel advanced virtual RISC (AVR) developed by Alf-Egil Bogen and Vegard Wollan (termed as Alf Vegard RISC). The AVR is one of the first microcontroller families that uses on-chip flash memory that eliminates the write-once phenomena of microcontroller chips. The first AVR chip was a 40-pin dip and the pinouts are almost similar to the 8051 microcontroller. Flash, EEPROM, and static RAM are integrated within the single chip. Some of the microcontrollers have a parallel bus connected so that the external memory can be interfaced.

2.5 Application-Specific Processors

Generally, a processor is used to perform multiple tasks and process multiple instructions at a time. Therefore, a general-purpose processor (GPP) is more costly and may experience serious performance overhead. Consequently, when speed and cost both matter, the most obvious choice is the application-specific processors. Two application-specific processors often used in real life are shown in Figure 2.5. Specific processors can be used in several applications

**FIGURE 2.5**

Two application-specific processors.

such as in digital TV, set-top box, global positioning system (GPS) devices, and musical instrument digital interface instruments. Application-specific processors can be categorized into different subcategories:

1. *Digital signal processors:* These are programmable processors for highly expensive floating-point mathematical computation such as discrete Fourier transform, fast Fourier transform, and discrete cosine transform. Such systems are generally highly expensive in terms of cost and performance. DSP chips are often used in various real-time computations in high-end image processing devices and voice and sound processing devices. With current advancement in SOC technology, a DSP unit may even be available with multicore features. Using the SOC feature, it is possible to considerably reduce the cost and the power consumption of the processor.
2. *Application-specific instruction-set processors:* Such processors are the programmable and the hardware instruction sets that are exclusively designed for a specified application. Sometimes, the entire algorithmic logic is implemented in the hardware itself. The GPP, application-specific integrated processors (ASIPs), and application-specific integrated circuit (ASIC) are the three most important types of processors in this category. In the GPP, function and activity are built on the software level. One of the biggest advantages of such a system is its flexibility, but it is not much ideal in terms of performance. ASIC offers better performance but with lesser extensibility and flexibility compared to GPP, while the ASIP comprises GPP and ASIC. ASIPs are implemented to perform specific jobs with high performance and minimum upgrade of hardware components and offer more flexibility at a low cost compared to GPP and ASIC.

2.6 Sensors and Actuators

2.6.1 Sensors

The most obvious components of an embedded device are the sensors and actuators. Most of the embedded computer-controlled systems constantly monitor the functionality of the system and adjust the system accordingly when an error occurs. A sensor basically senses the real world similar to the human sensing organs. It converts the physical behavior of the system into electrical signals and sends it to the embedded computing device for further processing. Sensors may be broadly categorized into two different types: (1) analog sensors that capture the state of a system directly and convert it into a simple analog signal and (2) digital sensors that convert the analog data into samples and, after quantization, generate 1 or 0 bit corresponding to the information gathered.

An embedded system may contain one or more than one analog, digital, or hybrid sensor modules that perform a collective task of computing and analyzing critical data.

2.6.2 Examples of Sensors

One of the most popular sensors often used to measure temperature from a data acquisition device is the temperature sensor. A wide variety of temperature sensors are available in the market, such as LM35, TEMP36, SHT11, and DHT11, and some are shown in Figure 2.6. Among them, TEMP36 and LM35 are the most popular analog temperature sensor devices that can sense the temperature value and feed raw analog data to the microcontroller device.

SHT11 and DHT11 are digital sensors made of a complementary metal oxide semiconductor (CMOS) chip that measures both temperature and humidity. They have a four-pin package, where pin 1 is used for Vcc, pin 2 is

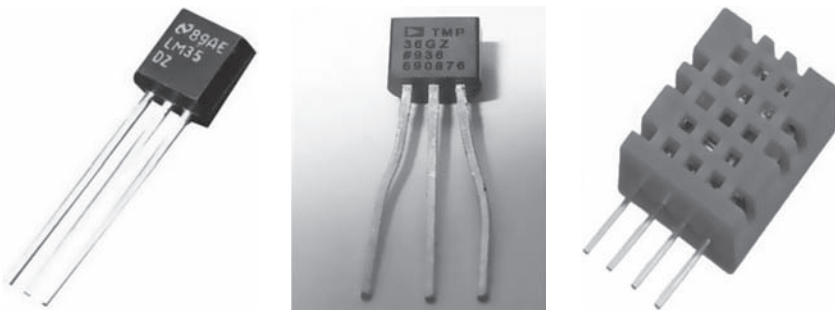
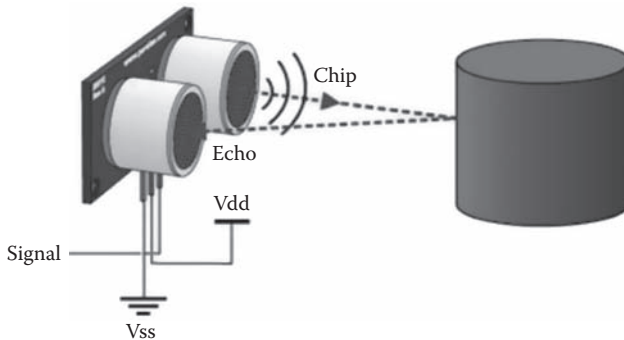


FIGURE 2.6
Three kinds of temperature sensors.

**FIGURE 2.7**

Working principles of the ultrasonic sensor.

data output, pin 3 has no connection, and pin 4 is the ground. These sensors are widely used for weather monitoring.

A photodiode and a photoresistor (often called light dependent resistor) are the most useful sensors to detect light. They can be interfaced directly to the analog pin of a microcontroller using a simple voltage divider circuit. This is a simpler form of sensor that has two terminals. It has no polarity, that is, any one terminal can be used to feed +5 V and the other terminals can be used as a signal input by adding a drop-down resistor in parallel.

Another most widely used sensor unit is an ultrasonic sensor, shown in Figure 2.7, which is often used in various applications such as an ultrasonic rangefinder, automated car parking system, obstacle detector, and avoider system. It generally consists of three to four pins for Vcc, ground (GND), and data output. Most of the ultrasonic modules have different transducers; one is called a transmitter for generating ultrasonic sound. On the other hand, a receiver receives the echo of the same sound generated by the transmitter. As the echo is received by the receiver, it immediately generates a pulse corresponding to the time between the transmission and the receiving event through which the distance of the object is to be computed. The signal is sent to the microcontroller in the form of PWM or PPM techniques.

2.7 Embedded Communication Interface

2.7.1 I2C Communication

The inter-integrated circuit (I2C) is a communication protocol that connects a number of IC devices. This is similar to the synchronous serial communication where two signal lines, one serial data and the other

a serial clock (SCL), go from the master to slave. No chip select line is required. Conceptually, any number of masters and slaves may be connected with these two signal lines for communication among themselves. In the I2C interface, slaves are identified by 7-bit addressing. Data typically consist of 8 bits. Some controls, such as start, stop, and direction bit, are incorporated to manage the communication between the master and the slave. A minimum of 100 kbps and maximum of 3.5 Mbps data rate is chosen for the I2C communication.

2.7.2 SPI and SCI Communication

Basically, in SPI communication, four signal lines are used. A clock signal SCLK is sent to all slave devices from the master. All SPI devices are synchronous with this clock signal. A data line dedicated from the master to the slave is called master out slave in and from the slave to the master is called master in slave out. SPI is also known as a single master-based communication protocol. This is because a central master takes initiative when communicating with the slaves. When a master sends data to a slave, it selects a slave by drawing the SS line low and activates the clock frequency usable by the master and slave at that time.

The serial communication interface (SCI) is a full duplex asynchronous communication interface that uses nonreturn-to-zero signal format where 1 start bit, 8 data bits, and 1 stop bit are available. The SCI devices are generally independent of each other only the data format support same bit rate of all devices. Some of the exclusive features of SCI communication are as follows:

1. It supports an advanced error detection mechanism such as noise detection.
2. Software programmable 32 different baud rates.
3. Software selectable word length.
4. Interrupt-driven operation.
5. Separate transmitter and receiver enable bit.
6. Receiver and transmitter wake-up function.
7. Framing error detection.

2.7.3 UART Communication

For asynchronous communication between two data serial links, UART communication can be made. In most cases, peripherals such as keyboard, mouse, and other serial devices can communicate with this interface.

The transmitter section contains a transmission shift register and transmission hold register. When UART is in the first in, first out (FIFO) mode,

THR is a 16-bit FIFO. When UART transmission occurs, the transmitter sends the following to the sender:

- a. 1 start bit
- b. 5, 6, 7, or 8 data bits
- c. 1 parity bit
- d. 1, 1.5, or 2 stop bits

A UART receiver section consists of the receiver shift register (RSR) and receiver buffer register (RBR); when the UART is in FIFO mode, RBR is a 16-byte FIFO. Based on the chosen settings of the line control register, the UART receiver accepts the following from the transmitter device:

- a. 1 start bit
- b. 5, 6, 7, 8, data bits
- c. 1 parity
- d. 1 stop bit

The UARTn_RXD pin is dedicated to receive the data bits via the UART receiver. Then the data bit is concentrated by RSR and the resulting values are moved into RBR (or the receiver FIFO). Three bits of error status information are stored by the UART: parity error, framing error, or break, to name a few.

2.7.4 USB Communication

The USB is a standard type of connection for different kinds of devices. The basic specification of the USB is USB 1.0 that was introduced in 1996. It has a data transfer rate of 1.5 Mbps. The maximum rate of transfer is 12 Mbps. The second-generation USB specification is USB 2.0, having a rate of up to 480 Mbps, which is quite high. The third-generation USB device (USB 3.0) has been modified to have a speed range of 5 Gbps.

Various USB ports, such as USB a, USB b, and USB micro, are available in the market. The USB device system consists of a standard tree architecture, where the root of the tree is known as a USB root hub. From the USB root hub, there are several USB child hubs generated, forming a treelike structure.

The USB protocol is a standard tree-based architecture with a start frame, handshaking with acknowledgment, and negative acknowledgment (ack.) control signals. All USB ports have four connections: Vcc, GND, data, and no connections.

A USB device may obtain power from external sources or from a USB through the hub, to which they are attached. Externally powered devices can be self-powered. Although self-powered devices may already be powered

before they are attached to the USB and not considered to be in the powered state until they are attached to the USB and VBUS is applied to the device.

2.8 Real-Time Operating Systems

An RTOS is a kind of operating system environment that reacts to the input within a specific period of time. The deadline of such operating system tasks is so small that the reaction seems to be instantaneous. A key feature of an RTOS is the consistent level of the time it takes to accept and finalize the task of the application. An advanced scheduling algorithm is present in every RTOS. The flexibility of the schedule enables a vast, computer-system orchestration to prioritize the process, but a certain narrow set of application exists where the RTOS is more frequently dedicated. Key factors in an RTOS are minimum interrupt latency and minimum latency of the thread switch. An RTOS is greatly valued for how quickly or how precisely it can respond rather than the quantity of work it can perform in a certain period of time.

An RTOS must respond instantaneously to the change in the state of the system, but that does not necessarily mean that it can handle a huge amount of data altogether and its throughput. In fact, in an RTOS, small response times are a much valuable performance metric than higher computational power or data speed. Sometimes, an RTOS will even forcefully drop data elements to ensure that it reaches its strict deadlines. In essence, an RTOS is defined as an operating system designed to meet strict deadlines. Beyond that definition, there are few requirements as to what an RTOS must be or what features it must have. Some RTOS implementations are precisely complete and highly robust, while other implementations are simple enough and suited for only one specific purpose.

An RTOS may be either time sharing or event driven, which is a system that changes state according to its response of an incoming event. A time-sharing RTOS is a system that changes state as a function of time and has two basic categories available.

2.8.1 Hard Real-Time System

A hard real-time system must absolutely hit every deadline very precisely in a mission-critical scenario. Very few systems fall under this category. The functionality of the system, therefore, depends upon the accuracy of the work. If a system fails to fulfill a job properly, it is simply treated as a failure. Some examples are medical applications such as pacemakers; nuclear systems; a variety of defense applications; and avionic equipment like navigation controllers, drone autopilot units, and radar-guided missiles.

2.8.2 Soft Real-Time System

A soft real-time system, often called as firm real-time systems, might miss some of the deadlines, but eventually performance will degrade if too many are missed. A good example is the sound system of a computer. If a few bits are missed, no problem occurs, but when too many bits are missed, an eventual degradation of the system will occur. Similar would be seismic sensors. If a few data points are missed, no problem occurs, but it has to catch most of them to make sense of the data. More importantly, nobody is going to die if they do not work correctly.

Various operating system design standards have to be maintained to design an RTOS.

2.8.3 Thread-Oriented Design

This is a special variation of the RTOS, in which the entire task is done by using a multithreading environment. Threading is the concept of multitasking where a single process creates multiple numbers of processor allocations often called a thread. The main advantages of such a system are that the task becomes very simple and less memory is used. Often Java-based embedded hardware uses a thread-oriented design.

2.9 Typical Examples

2.9.1 Smartphone Technology

Smartphones have advanced computing capability. In earlier days, smartphones had the capability of voice calling and some add-on features like personal digital assistant, digital camera, and media player. Today, smartphones are available with new and advanced features such as GPS assistance, Wi-Fi, touch screen, and lots of other applications.

Smartphone technology is now mostly based on low-power processors that are capable of managing a wide range of applications efficiently without consuming a large amount of power. Various processors like Snapdragon S4, ARM Cortex-A15, A7, Intel Atom, NVIDIA Tegra, and Samsung Exynos are now being used to support a wide range of features in a smartphone.

In addition to this, popular operating systems such as Android, iOS, Blackberry, Windows Mobile, and Bada play a vital role in supporting a wide range of applications in smartphones.

2.9.2 Aircraft Autopilot Unit

An aircraft autopilot unit is said to be a highly mission-critical application of an embedded system. An autopilot mainly consists of sensor sets that are responsible for auto navigation. Among them, components such as gyroscope, magnetometer, accelerometer, and GPS system play a vital part. An altimeter is also attached to get the altitude reading, and the gyroscope gives the pitch, roll, and yaw angle of the aircraft during flight. A magnetometer gives the proper heading of the aircraft during flight, and the GPS provides the actual location of the aircraft during flight. Data supplied by that sensor are finally fed to a computer that takes the control of the entire navigation. During an autopilot control, an alternate manual control is provided in most cases because it is pretty challenging to entirely rely on the autopilot unit, although it is sophisticated enough.

3

Basics of Robotics

3.1 Introduction

A robot is an intelligent machine that can interact with the environment to perform specific tasks to reduce human effort. Various types of robotic systems are available; however, the majority of robots have some common features. Almost all robots have a movable body, some of them have motorized wheels, whereas others have many small movable segments that are typically made of plastic, wood, or metal. In some robots, several joints connect the individual segments of the robot together. The actuator of the robot spins the joint with wheels or a pivot segment.

Robots are classified into several types based on the systems they use: (1) robots that use electric motors as actuators, (2) robots that use a hydraulic system, (3) robots that use a pneumatic system that is driven by compressed gases, and (4) robots that use all of these actuator types. Generally, any robotic system requires a power source to drive its actuators. Most robots have either a battery or other power sources. Hydraulic robots mostly require a pumping system to pressurize the hydraulic fluid, and pneumatic robots mostly need air compressors or compressed air tanks.

In most cases, a microcontroller is used as the brain of a robot, which is sometimes called a microcomputer. All the actuator and circuits are directly connected to microcomputers via some interface systems. Another common feature is that most robots are programmable; thus, a robot's behavior can be changed by writing a new program in its microcomputer.

3.2 Robot Kinematics

From a kinematics perspective, a robot can be defined as a mechanical system that can be designed to perform a number of tasks that involve movement under automatic control. The fundamental characteristic of a robot is its ability to move in a 6D space that includes translational and rotational coordinates.

It is possible to model any robot as a series of rigid links connected by several joints. The joints restrict the relative movement of adjacent links and are generally equipped with motorized systems to control the movement.

Robot mobility (degrees of freedom) is defined as the number of independent parameters needed to specify the positions of all members of the system relative to a base frame. To determine the mobility of mechanisms, the most commonly used criterion is the Kutzbach±Grübler formula. For a robot with x links (counting the base) and t joints, where each joint p allows d_p degrees of freedom, the mobility can be computed using

$$M = 6(x - 1) + \sum_{p=1}^t (6 - d_p) \quad (3.1)$$

But this formula does not provide correct mobility for several types of robots. To overcome this drawback, it is necessary to define direct kinematics for the workspace of the robot. In this method, a set of all possible positions of the end effectors is constructed using every possible combination of the joint variable values in their range that defines the workspace of the robot. Here, the position means location as well as orientation and the workspace of the robot is a six-dimensional subset of the six-dimensional space of rotations.

3.3 Degree of Freedom

Figures 3.1 through 3.3 illustrate different degrees of freedom of the system that can be defined as the number of independent rules by which a dynamic system can perform its movement without violating any constraint imposed on it. In other words, the degree of freedom can be defined as the minimum number of independent coordinates that can specify the position of the dynamic system completely.

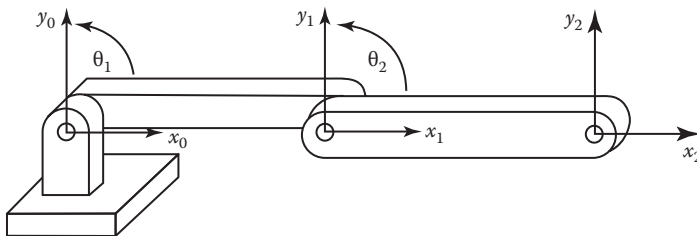


FIGURE 3.1

Revolute joint has 1 degree of freedom (DOF).

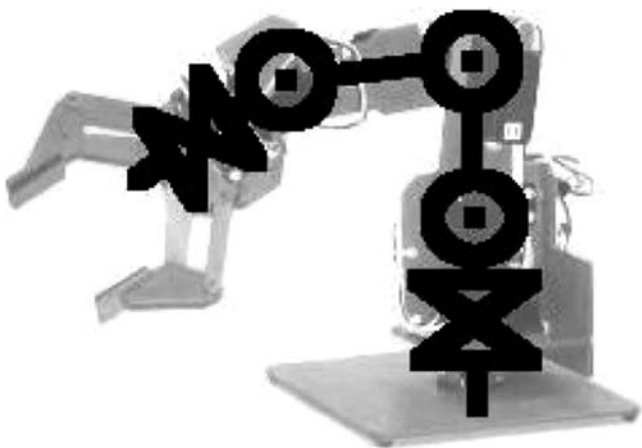


FIGURE 3.2
Claw joint has 2 DOF.

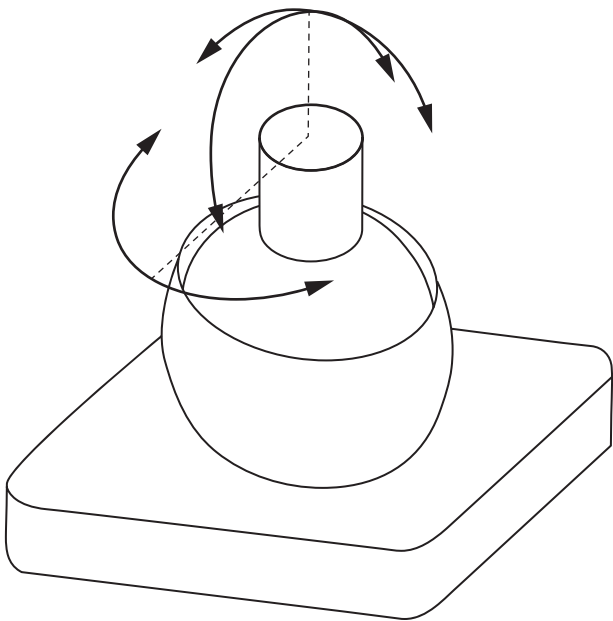


FIGURE 3.3
Ball-and-socket joint has 3 DOF.

3.4 Forward Kinematics

Basically, the forward kinematics is a transformation from angles to position. The length and the angle of each joint are to be given in order to find out the position at any point. For example, consider the situation where a robotic arm that starts out aligned with the x -axis, where the first link to be moved by ξ_1 and the second link to be moved by ξ_2 . Therefore, to determine the final position of the robotic arm end, there are two solutions: (1) the geometric approach and (2) the algebraic approach.

The geometric approach is considered to be the easiest and simplest solution. However, the angles that have been measured are basically relative to the previous links' direction, where the first link is the exception. Therefore, the angle is measured relative to its initial position. For robots with more links and whose arm extends into three dimensions, the geometric approach gets much more tedious.

3.5 Algebraic Solution

Assume that there are three link arms that start out aligned in the x -axis. Each link has lengths l_1 , l_2 , and l_3 , respectively, as shown in Figure 3.4. If the first one moves by ξ_1 and so on as the diagram suggests, find the homogeneous matrix to get the position of the yellow dot in the x_0y_0 frame.

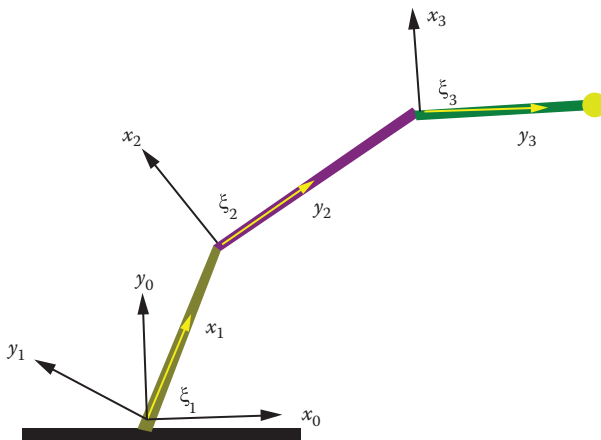


FIGURE 3.4

Visualization of a robotic arm.

Then,

$$H = R_z(\xi_1) * Tx_1(l_1) * R_z(\xi_2) * Tx_2(l_2) * R_z(\xi_3) \quad (3.2)$$

Rotating by ξ_1 will put it in the x_1y_1 frame. Translate it along the x_1 axis by l_1 and rotating by ξ_2 will put it in the x_2y_2 frame. This is continued until it is in the x_3y_3 frame.

The position of the tip of the extreme top of the arm relative to the x_3y_3 frame is $(l_1, 0)$. Multiplying H by that position vector will give the coordinates of the yellow point relative to the x_0y_0 frame.

3.6 Inverse Kinematics

In the case of inverse kinematics, the length of each link and the position of any point on the robot are given and the angles of each joint are calculated to get that particular position.

In the case of combined revolute and prismatic joints as shown in Figure 3.5, consider that

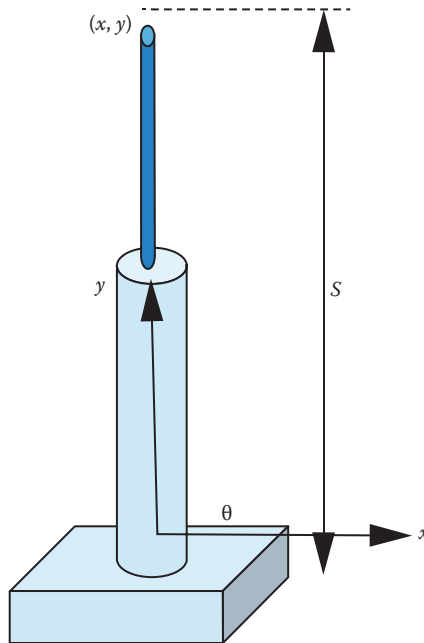


FIGURE 3.5
Revolute joint.

$$\theta = \arctan\left(\frac{y}{x}\right) \quad (3.3)$$

$$S = \sqrt{x^2 + y^2} \quad (3.4)$$

3.7 Robots and Sensors

Typically, the robotic system should have some basic sensing capabilities. A very basic wheel robot must sense the path where it is moving or the obstacles that might be in front of its path. A robot consists of several sensors at a time to detect orientation, location, direction, etc. The sensor systems that are often used in a basic robot are explained in the following sections.

3.7.1 Motion Detection Sensor

The infrared motion sensor is the most popular type of motion sensor. Infrared radiation basically lies in the range of the electromagnetic spectrum at a wavelength that is much longer than visible light. It cannot be seen, but it can be detected. It is a property that an object generates heat and infrared radiation at the same time and those objects, including the human body, animals and birds, whose radiation is strongest at a wavelength of 9.4 μm .

3.7.2 Gyroscope and Accelerometer

The main job of a gyroscope sensor is to sense the orientation of a system with respect to the earth. Its basic principle is the angular momentum of the system. A mechanical gyroscope system comprises a spinning wheel or disk whose axis is free from any orientation. Generally, a gimbal system has been introduced to protect the gyroscope from the external torque.

Nowadays, most of the electronic robotic systems use micro electro-mechanical system (MEMS) gyro system. The gyroscope typically acts on the principle of the Coriolis acceleration. Coriolis acceleration is basically proportional to the velocity and the angular rate of the body given by

$$A_c = 2\Omega \times v \quad (3.5)$$

where

Ω is the angular velocity of the body

v is the linear velocity of the point in the reference frame of that body

In the MEMS gyroscope, a proof mass is attached that is actuated with a resonant frequency causing an oscillation in the vertical plane. Actuation of proof mass is an oscillation that creates a sinusoid. These sinusoidal vibrations are captured using piezoelectric transducers and converted into an electrical signal.

An accelerometer helps robots to sense body acceleration. A MEMS accelerometer consists of a proof mass suspended by two suspension units. The acceleration caused by the body deflects the suspension. The acceleration of the proof mass is proportional to the deflection of the suspension. The deflected state of the suspension is converted into an electric signal using a capacitive pickup.

3.7.3 Obstacle Detector

A very well-known obstacle detector is the ultrasonic sensor shown in Figure 3.6. It can sense the distance of the object by sending an ultrasound toward the object. Then the sending and receiving time of the sound are recorded. Finally, the distance of the object can be determined from the velocity of the sound using the following equation:

$$\text{Standard parallax ultrasonic sensor distance} = \frac{\text{Time}}{74/2} \quad (3.6)$$

3.7.4 Location Tracking by GPS

Figure 3.7 demonstrates the global positioning system (GPS) that provides the users with 3D information of any region on earth. This system was

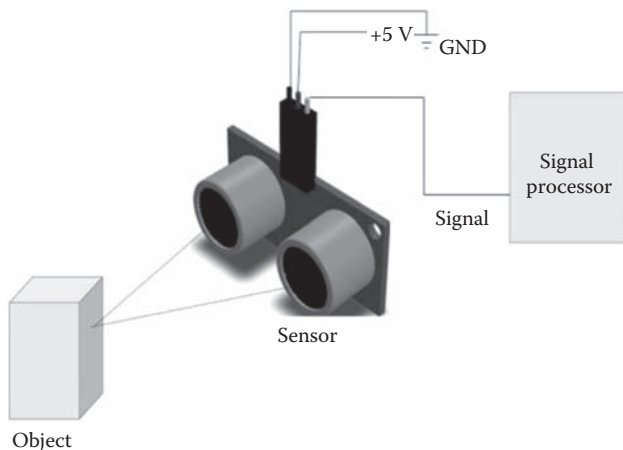


FIGURE 3.6
Obstacle detection.

**FIGURE 3.7**

A global positioning system module mounted on an aerial vehicle.

mainly developed for navigation. It was developed as NAVSTAR GPS by the U.S. Department of Defense in 1993. Initially, it was restricted to defense uses, but currently it is available for common use. In most cases, the robotic system highly depends on GPS when performing search and navigation tasks.

The main functional component of the GPS is the configuration of 24 satellites' orbit having an altitude of 20,180 km from the earth. The orientation of the satellites is such that at any time at least four satellites are visible from the earth. By measuring the time of signal travel from each satellite to the receiver, the location of the receiver can be determined. The time taken by a signal to travel is used to determine the location of the receiver, and because clock synchronization error exists between the satellite and the receiver clock, the location might not be exact. Such range is called *pseudorange* and can be distinguished from the true range.

3.8 Robots and Motors

From conventional robots to space robots, all kinds of robotic systems use several kinds of motors. Most robotic systems use hybrid motor units.

3.8.1 DC Motor

A direct current (DC) motor, shown in Figure 3.8, has one set of coils, known as an armature winding, inside another set of coils or a set of permanent magnets (stator). Applying a voltage to the coils produces a torque in the armature, resulting in motion. The stationary outside part of a DC motor is called the stator of the motor. The stator of the permanent magnet DC motor comprises two or more permanent magnet poles. A magnetic field can be created by an electromagnet alternatively; in such case, a separate stator winding is applied to the stator. In this case, a DC coil is wound around a magnetic material, like iron, that forms the stator. The rotor is the inner part of the motor that rotates. A rotor consists of windings called armature windings, which are generally connected to the external circuit via a mechanical commutator. Both stator and rotor are made up of ferromagnetic materials and are separated by an air gap. Winding of stator and rotor coils is made up of series or parallel connections. Field winding is a process through which current is passed to produce flux (for the electromagnet), whereas armature winding is winding through which voltage is applied or induced. The windings are usually made of high-quality copper wire.

Two conditions are necessary to produce force on the conductor: (1) The conductor must be carrying current and (2) it must be within a magnetic field. When these two conditions exist, force will be applied to the conductor that will attempt to move the conductor in a direction perpendicular to the magnetic field. This is the basic theory by which all DC motors operate.



FIGURE 3.8
Direct current motor.

3.8.2 Servo Motor

A servo is a mechanical, motorized device that can be instructed to move the output shaft attached to a servo wheel or arm to a specified position, as shown in Figure 3.9. Inside the servo box is a DC motor mechanically linked to a position feedback potentiometer, gearbox, electronic feedback-control-loop circuitry, and motor drive electronic circuit. A typical R/C servo looks like a plastic rectangular box with a rotary shaft coming up and out the top of the box and three electrical wires out of the servo side to a plastic three-pin connector. Attached to the output shaft at the top of the box is a servo wheel or arm. These wheels or arms are usually plastic with holes in it for attaching push/pull rods, ball joints, or other mechanical linkage devices to the servo. The three electrical connection wires on the side are the $V-$ (ground), $V+$ (plus voltage), and S control (signal). The control S (signal) wire receives pulse width modulation (PWM) signals sent from an external controller and is converted by the servo onboard circuitry to operate the servo.

The R/C servos are controlled via sending PWM from an external electronic device that generates the PWM signal values, such as a servo controller, servo driver module, or R/C transmitter and receiver. The PWM signals sent to the servo are translated into position values by electronics inside the servo. When the servo is instructed to move (receiving a PWM signal), the onboard electronics convert the PWM signal to an electrical resistance value and the DC motor is powered on. As the motor moves and rotates, the linked potentiometer is also rotated. The electrical resistance values of the moving potentiometer are sent back to the servo electronics until the potentiometer value matches the position value sent by the onboard servo electronics



FIGURE 3.9
Servo motor.

that was converted from the PWM signal. Once the potentiometer value and servo electronic signals match, the motor stops and waits for the next PWM signal input signal for conversion.

3.8.3 Stepper Motor

For precise positioning and speed control without the use of feedback sensors, the stepper motor is the obvious choice. Each time a pulse of electricity is sent to the stepper motor, the shaft of the motor moves a number of degrees that we want, which is the basic principle of the stepper motor. Since the shaft of the motor only moves the number of degrees that it desired when each pulse is delivered to the motor that can be controlled or to control the positioning and speed by using a controller. Like a simple DC motor, the rotor of the motor produces torque from the interaction between the stator and rotor's magnetic field. In general, the strength of the magnetic fields is directly proportional to the number of turns in the windings and the amount of current that has been sent to the stator. The stepper motor uses the theory of operation for magnets to make the motor shaft turn a precise distance when a pulse of electricity is provided—like poles of a magnet repel and unlike poles attract. In general, for a stepper motor, the rotor has six poles (three complete magnets), and the stator (or stationary winding) has eight poles. An electricity requirement of 24 pulses is mandatory for the rotor to move 24 steps to make one complete revolution. This means that the rotor will move precisely 15° for each pulse of electricity that the motor receives. We can calculate the number of degrees the rotor will turn when a pulse of electricity is delivered to the motor by dividing the number of degrees in one revolution of the shaft (360°) by the number of poles (north and south) in the rotor. In this stepper motor, 360° is divided by 24° to get 15° .

3.9 Robot Controller

Controlling a robotic system is not an easy task, and thus, there are various components related to it. First is the microcontroller that controls the robotic systems and then the software and the program that controls the functionality of the robotic device. A robotic controller basically consists of a set of feedback control devices performing some dedicated tasks. A robot controller is used to decrease the errors of control signals to zero or somewhere close to zero. It can be classified into six different types:

1. On±off control
2. Proportional control
3. Integral control

4. Proportional and integral control
5. Proportional and derivative control
6. Proportional, integral, and derivative control

In the first case, an on±off controller provides two separate states: (1) on (state 1) and (2) off (state 2). The purpose of an on±off control is to protect the controller from swinging with very high frequency. This is made possible by moving the error through several ranges before the operation starts. Here, the range is considered as the differential gap.

In the second case, a control signal is produced by this controller that is proportional to the error. It is basically used as an amplifier by means of a gain. The proportional controller will be best suited for providing a *smooth control action*.

A control signal produced by the integrated controller is *altered* at a rate proportional to the error, that is, the control signal maximizes quickly if the error is big and maximizes slowly if the error is small.

The Pololu 3pi robot is an example of an autonomous robotic device. It is mostly used for maze solving and line-following applications. The Pololu 3pi system is based on AVR ATmega 168 or 328 as it has an infrared motion sensor. Today, several robotic controllers are available at a cheaper rate.

3.10 Frames and Materials

Selecting a proper frame design and material is a critical task when designing robots. Conventionally, a lighter and durable material is preferred. Most of the professional robots are made of carbon fiber as it is a highly durable material. Other materials like aluminum and hard plastic like high density polyethylene are also preferred. To design hobby robots, balsa wood is also often preferred due to its lightweight. Fiberglass is another material that is often used to build the chassis and the arm of the robot. Styrofoam is an often overlooked material that is preferred when designing the component of robots, mostly in the case of aerial robots. The advantage of Styrofoam is that it is easy to shape, lightweight, and extremely cheap.

3.11 Types of Robots

Several types and domains of robotics are involved in today's world. They are classified based on the services they provide and are detailed in the following sections.

3.11.1 Industrial Robots

Industrial robots have specific use in various industries as shown in Figure 3.10. In general, these robots are the robotic arms or manipulators that perform the task of welding, material handling, painting, and tightening of several parts. Robots in the production line are good examples of such a robotics system. An industrial conveyor belt can also be treated as an industrial robot that moves several products and parts from one area of an industry to another.

3.11.2 Medical Robots

Medical robots are used in medical industries to perform numerous tasks such as equipment lifting, washing of equipment, and mostly surgery, as shown in Figure 3.11. In robotic surgery, the surgeon controls the robotic arm using a very small tool that is attached to the robotic arm via a control computer. In such case, the surgeon makes small cuts to insert the instruments into the human body. A thin tube attached to the camera in front is used to capture the enlarged real-time image of the part of the body where the surgery will take place. The robot matches the movement of the hand of the surgeon, and the software of the computer tunes the precision



FIGURE 3.10

ABB industrial robot. (From dhgate.com.)

**FIGURE 3.11**

A robot performing surgery. (From abcnews.go.com.)

of the movement of the hand such that the surgery becomes highly accurate. Robotic arms are highly useful for the following types of surgery: (1) coronary artery bypass, (2) hip replacement, (3) gallbladder removal, and (4) kidney removal and transplant.

3.11.3 Military Robots

Military robots are developed to be used by the armed forces such as for searching, search and destroy, rescue, and surveillance operations; they are often called artificial soldiers. Such robots are used not only as ground vehicles but also as aerial, water, or underwater vehicles. The concept of military robots started immediately after the Second World War. Germany's Goliath and the Soviet TT-26 Teletank are some of the examples of military robots of that era.

Foster-Miller TALON, a remotely operated ground vehicle, is shown in Figure 3.12. Today, it is one of the popular military robots that can move through sand and water and can climb stairs. It can be operated from 1 km away; it has infrared and night vision.

Another example of a military robot is MQ-1 Predator, as shown in Figure 3.13. It is an unmanned aerial vehicle used by the U.S. Air Force and CIA. This vehicle is considered to be a medium-altitude long-endurance vehicle. Along with military applications, civilian applications such as border enforcement, scientific studies, and wind direction monitoring are additionally performed by this drone.

Aeryon Scout, illustrated in Figure 3.14, is a multirotor drone that is widely popular for its primary task of performing surveillance. It can be operated from a 3 km distance and is functional within the temperature range of -30°C to $+50^{\circ}\text{C}$.



FIGURE 3.12
Foster-Miller TALON.



FIGURE 3.13
MQ-1 Predator.

3.11.4 Space Robots

Research on space robotics primarily focuses on two areas of interests: (1) orbital robotics and (2) planetary rovers. Orbital robotics has an interest in the research domain of manipulation and mobility for scenarios such as international space stations and satellite servicing. Planetary rovers address scenarios such as Mars and lunar exploration using mobile robots on the

**FIGURE 3.14**

Aeryon Scout. (Courtesy of Aeryon Labs Inc., Waterloo, Ontario, Canada.)

surface and other situations such as asteroid and comet exploration. Robotics research in a low-gravity scenario poses unique challenges to space robots and algorithm design and to areas such as electromechanical design and control, micro gravity locomotion, command and control interface, including teleoperated mode, power source and consumable recharging techniques, thermal effects in space robot design. For planetary rovers, the surface environment poses unique challenges. The main area to be emphasized is sensing and perception for planetary exploration, including terrain-relative precision position estimation.

**FIGURE 3.15**

AIBO, the entertainment robot.

3.11.5 Entertainment Robots

As the name suggests, such robots are not used for utility purposes and are mostly made for fun, pleasure, entertainment, and sometimes domestic service. Entertainment robots are widely seen in the context of media and arts, where artists employ advanced technologies to create the environment and artistic expressions to allow the sensors and actuators to react according to the changes related to the viewer. Being relatively cheap and mass produced, entertainment robots are used as mechanical and sometimes interactive toys that can perform several tricks and take several commands. AIBO shown in Figure 3.15 is an iconic series of robotic pet designed by Sony Corporation. Sony announced a prototype of this robot in the late 1990s. It had been used in many popular movies and music videos.

3.12 Summary

In this chapter, we have discussed the basics of robotic systems starting from a simple robotic arm to heavily sophisticated space robots. Robots have become a part of our everyday life, and by 2020, the revolution of the robotics industry will be paramount.

4

Aerial Robotics

4.1 Introduction to Aerial Robotics

Aerial robotics serves as a platform for the next generation that consists of a robotic concept emphasized from a small micro aerial vehicle to a large fixed-wing multirotor drone. The application of such a platform widely depends upon the utilization. Starting from aerial surveillance to traffic monitoring and geological and weather survey in agriculture, there is a wide range of applications for aerial robotics; in addition, services such as inspection and maintenance are also being performed using aerial robots. Various organizations currently offer different professional aerial robotic platforms. Besides, the advancement of open-source hardware, microelectromechanical system (MEMS), and smartphone technology accelerates the growth of aerial robotics in various aspects. Three-dimensional (3D) robotics plays a leading role in glorifying the field of aerial robotics. Current research focuses on specific aerial robotics domain such as biologically inspired aerial robots, robots with hybrid locomotion that can travel in both air and water, as well as on ground and in deep forest environment. Aerial robots are also being used to study the evolution of flight.

4.2 History of Aerial Robotics

Aerial robotics history starts from the American Civil War in 1861±1865. In 1944, Japan released high-altitude balloons that could carry bombs. Afterward, in 1950, the United States carried out research on high-altitude aerial platforms named Project Gopher and Genetrix. The balloons were outfitted with an automatically triggered camera. Through the years 1960±1970, several developments have been made. In the late 1950s, the purpose was to design target drones, with the first appearance of Ryan Firebee Series jet propelled UAVs. During the Gulf War in 1991, there was an utmost necessity

of aerial robots. At that time, UAVs were used as strategic tools. The Global Hawk was one of the famous UAV platforms at that time. Over the past 10 years, both fixed-wing and rotary-wing UAVs were in use. The RQ-8 Fire Scout recently achieved a good result in firing the target through missile. Dragonfly and Aeryon Scout are multicopters dedicated for aerial surveillance. Work is still in progress to produce more tactical and efficient UAV aerial robots.

4.3 Classification of Aerial Robots

Broadly, aerial robots (UAVs) can be classified into two basic categories: (1) fixed-wing aerial vehicle and (2) rotor craft systems. Each robotic system has unique features that are discussed in the following sections.

4.3.1 Fixed-Wing Systems

The fixed-wing system has natural gliding phenomena that are basically driven by the airfoil created during the passing of air over and beneath the aircraft wing, as shown in Figure 4.1. Based on various architectural criteria, the fixed-wing system can be categorized by a V-tail (Figure 4.2), T-tail, inverted V-tail, and H-tail. The lighter V-tail aircraft is lightweight and has a better and faster turning capability because the rudder and elevator is common. It also offers a less amount of air drag.



FIGURE 4.1
A normal fixed-wing model.

**FIGURE 4.2**

A V-tail fixed-wing model.

Inverted V-tail shares many pros and cons of V-tail, but it is not widely used in the aircraft industry. The MQ-1 Predator drone is the most common example of this class. The inverted V-tail architecture is a kind of collapsed Y-tail configuration. The advantage of such a configuration is that it has the tendency to roll efficiently, and the disadvantage is that it has a reduced flare potential.

A Y-tail aircraft is supposed to be a variation of a V-tail aircraft that has an additional vertical surface. Like the V-tail, it needs a control mixer. The architecture of an inverted Y-tail is more popular because of its great improvement in stall recovery. A McDonnell Douglas F-4 Phantom fighter is an example of this architecture.

The advantage of T-tail is that the chance of the aircraft stalling is minimum. Along with that, at a very high angle of attack, the rudder is not blanked by the horizontal effects, which makes it more effective to get out of a spin behavior.

The H-tail configuration is one of the optimal solutions when the overall height of the airplane becomes an issue. This configuration is very much useful for airplanes with two or more engines. The H-tail configuration basically reduces the usage of the huge rudder and offers an additional rudder area that provides more flight stability.

The Delta Wing (Flying Wing) aircraft shown in Figure 4.3, on the other hand, is basically a tailless aircraft that has no specific fuselage. This model is basically an experimental design. This structure significantly reduced air drag due to the elimination of tail and specific fuselage, making it an advantage. A non-lift-producing surface has also been eliminated in this structure. Therefore, it may achieve a tremendously high speed. But a high angle of attack is required for takeoff and landing, which is a drawback. Also, due to lack of control surface and stabilization, it is very hard to control the attitude of aircraft.

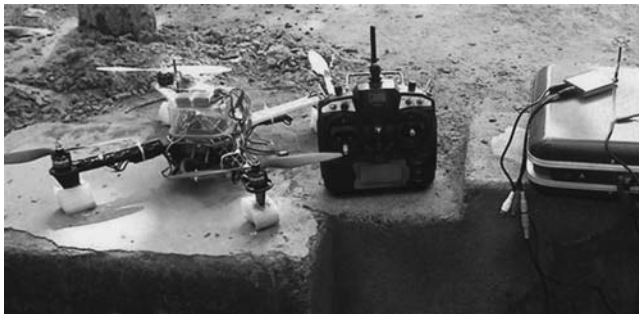
**FIGURE 4.3**

A fixed-wing (Delta Wing) aerial drone.

4.3.2 Multirotor Systems

A multirotor craft shown in Figure 4.4 is mostly used for hovering and holding position within a certain location in space. A multirotor may consist of two or more rotating arms. The two-rotor aircrafts are often called bicopters, and as the number of rotating arms increases, the name changes to tricopter, quadcopter, hexacopter, octacopter, and so on. Tricopter, quadcopter, and hexacopter are the most popular structures in this category.

Multirotor systems are highly usable in indoor and outdoor operations. In such cases, the architecture and configuration can be changed according to the requirement. The significant factor is the direction in which the propellers of the rotor craft rotate. In general, for a bicopter, the direction of the two rotors is reverse in nature, that is, mostly, the left rotor rotates clockwise and the right rotor rotates counterclockwise. In the case of a tricopter, the rotation of the left rotor is clockwise, whereas the rotation of

**FIGURE 4.4**

A multirotor aerial test platform.

the right rotor and tail rotor is counterclockwise. A special servo mechanism has also been added at the tail rotor to control the direction of the movement of the copter. In this case, as the direction of the servo changes, the pitch of the propeller changes, resulting in the change of the yaw of the copter.

In the case of a quadcopter, the movement entirely depends upon the applied thrust and the direction of the motor movement. Here, the two-diagonal motor will move in a similar direction (either clockwise or counterclockwise). The change in the roll, pitch, and yaw, therefore, completely depends upon the thrust applied. In this case, if the thrust of the rear motors becomes greater than the thrust of the front motor, the copter will feel the pitch effect in the front direction and the opposite condition makes a reverse pitch effect. Whereas if the thrust of the left motors becomes greater than the thrust of the right motors, then the copter will feel the roll effect toward the right, and the similar effect has been produced in reverse to roll on the right side. For the yaw to be in a particular direction, one diagonal motor should spin with a higher speed than the other diagonal. Depending upon the shape and size of a quadrotor system, various applications, such as aerial surveillance and 3D terrain mapping, are possible. A small-sized quad is also used for acrobatic flight.

The hexacopter system operates in a similar way and the only difference is that the number of rotating arms is increased by two. As the rotating arm increases, the payload capacity also increases. A hexrotor configuration is often used for high-quality aerial photography purposes where the size and the weight of the photography equipment are huge. As the size of payload increases, a more powerful octacopter or greater can be used.

4.4 Sensors and Computers

Most autonomous aerial robots are sensor dependent. Sensors are the main components that navigate the aerial robots in the proper direction. Various kinds of sensor modules are incorporated within an aerial robot. A basic aerial fixed-wing drone does not have any sensor. Typically, a simple remote controlled airplane can be treated as a very elementary version of aerial robot. A basic kind of autonomous feature within it can be added by applying a gyroscope that gives aerial stability to the drone. A triple-axis gyroscope can measure the number of rotations around three axes, namely, x , y , and z . Some gyroscopes come in a single- and dual-axis variety, but triple-axis gyroscopes are more popular. Most of these gyros are MEMS, which are basically $1\pm 100\text{ }\mu\text{m}$ in size. When the gyro is rotated, a small proof mass gets shifted as the angular velocity changes. This

movement gets converted to a very low electrical signal that can further be amplified and used by the microcomputer of the aerial robot. Since this is a very basic sensor, it is not so efficient in controlling the entire navigation in an autonomous fashion.

A barometric pressure sensor within the system provides altitude data of aerial robots on the fly. A high-precision barometric pressure sensor provides a very good altitude reading that is sometimes quite necessary for multicopter and fixed-wing drones, mostly while performing the altitude lock of the aerial robot. Most barometric pressure sensors give the pressure reading in pascals (Pa), as 1 Pa is a very nominal pressure reading. The microcontroller converts a floating point value corresponding to the pressure reading. In general, 1 hPa (hectopascal) = 100 Pa, which is to be measured as 0.00098693 atm (standard atmospheres). As temperature affects the mass of the air and hence affects the density of the air and pressure depends upon the density, therefore, temperature has a direct effect on the pressure of the air.

To design an autonomous aerial robot, two additional sensors, magnetometer and global positioning system (GPS) sensor, are highly required.

A magnetometer is also a MEMS device that basically measures the magnetic field or magnetic flux density in the form of tesla. These sensors completely depend upon the mechanical motion of the structure as Lorentz force acts over the current-carrying conductor on the magnetic field. The motion of the MEMS can be sensed using an electrical signal. An electrostatic and piezoresistive transduction method can be used in electronic detection. A magnetometer is highly important when dealing with a robot of auto navigation capability. The compass-bearing value has a great significance while performing autonavigation.

Finally, let us discuss GPS, which is important when performing autonomous waypoint navigation. It was developed by the U.S. Department of Defense for navigation. A GPS device works based on the available GPS satellite deployed to orbit the earth 20,180 km above (called MEO). Generally, if at least four GPS satellites are visible from the ground by the receiver, then the location of the receiver can be easily traced. This technique is called triangulation. The accuracy of the location most receivers give is 10 ± 100 m. There are two types of starting techniques:

1. Hot start: Here, the GPS device remembers the satellite in view and its last computed position and the information about all the satellites in the constellation (called almanac). The coordinated universal time (UTC) of the system initiates an attempt to lock onto the same satellites and compute a new location based upon the previous buffered information. This is the quickest form of GPS lock, but it is only applicable if we are in the same location as we were when the GPS was last turned off.

2. Warm start: Here, the GPS device remembers its last calculated position, almanac used, and the UTC but not the satellites that were in view. It actually performs a reset and attempts to obtain the satellite signals and calculates a new position. Finally, the cold start is where the GPS device dumps all the information and attempts to locate satellites and then calculates a GPS lock. This takes the longest time because no previously known information is available.

4.5 Open Research Area

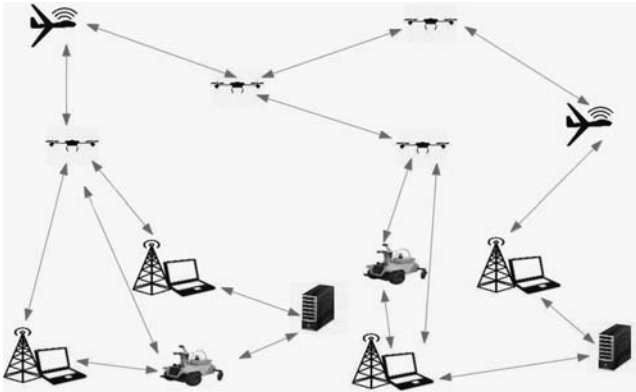
Recently, several domains of aerial robotics have been under research. One of the most common domains is the aerial swarm robotics where a group of aerial vehicles (may be multirotor or fixed wing or hybrid) will be organized in such a manner that they can perform a collaborative task. The path planning and the formation of such swarm robots are quite challenging. Along with that, the intercommunication between several ground vehicles and the aerial vehicle is another challenging task that needs to be solved in a more optimal way.

Another research issue is interdrone communication and drone-to-base station communication in a line out of sight situation, where a drone moves within a vast geographical region. In most cases, such problem can be solved using a high-altitude aerial platform that can be used as a router and stores and forwards messages coming from the group of aerial robots.

Flying ad hoc network is another concept that has good research potential under this domain. A FANET can be treated as a subset of mobile ad hoc networks and the vehicular ad hoc networks where the nodes of the network fly with even more high speed than a ground vehicle. Existing routing and the mobility model fail under such circumstances. Therefore, the development of a new kind of routing protocol is quite a challenge.

4.6 Aerial Sensor Networks

An airborne wireless sensor network (WSN) shown in Figure 4.5 comprises a bird-sized micro aerial vehicle that enables a low-cost, high-granularity atmospheric sensing system. The system will be applicable in atmospheric sensing, storm dynamics, and wildlife monitoring. An airborne WSN provides the capacity to enhance so many applications of interest to various scientific and research communities by offering finer granularity 3D sampling of several phenomena of interest that would be feasible. One such

**FIGURE 4.5**

A conceptual hybrid multiagent aerial sensor network.

popular application in this area is chemical dispersion sampling. In this case, the flock of the micro aerial vehicles (MAVs) has been deployed for sensing and communicating their data back to a network of ground stations enabling researchers to study the rate of dispersion of pollutant, chemical, and natural or man-made toxic material. For example, the study of the distribution of CO_2 concentrations in the atmosphere and its relation to global warming has also been done by the flock of aerial vehicles. In all these cases, a flock of micro aerial vehicles enable accurate sampling of the parameter of interest simultaneously over large regions as well as large volume.

In addition, since MAVs can be controlled independently, they can be targeted to track the dispersion rate of toxic plume and their study, fly toward the source of the plume if unknown, and redistribute to map the boundaries of the plume. Another class of applications that would benefit from an airborne WSN is that involving atmospheric weather sensing. Here, a flock of aerial vehicles are equipped with temperature, pressure, humidity, wind speed/direction, and/or other sensors that can provide detailed mapping of weather phenomena such as hurricanes, thunderstorms, tornadoes, and return data that would be useful in improving storm track predictions and in the understanding of storm genesis and evolution. Modeling the local weather produced by wildfires to better predict their evolution and improve the deployment of firefighting resources is another wider domain in these aerial sensor network fields.

5

Open-Source Hardware Platform

5.1 Introduction

Fundamentally, open-source hardware (open hardware) is a concept based on the open-source design principle. Physical designs, circuits, or any other physical objects that can be redistributed, modified, studied, or created by anyone are treated as open-source hardware. As already known, the *source code* for open hardware blueprints, computer-aided design (CAD) drawings, schematics, logic designs, and the source file is completely available for enhancement and further modification under permissible licenses. Users with access to the tools can read and manipulate all these source files and can update and improve the code that is further deployed on the physical device. They can add features/fix bugs in the software or even modify the physical design of the object itself aside from their ability to share such modifications.

Open hardware's resource files should be accessible to anyone, and its components are preferably easy to obtain. Essentially, the common roadblocks to the design and manufacture of physical goods are completely eliminated by open hardware. It provides as many users as possible with the ability to construct, remix, and share their knowledge of hardware design and function.

5.2 Open-Source Hardware Features

Open hardware has a great feature of scalability and versatility, and hence, it has a wide range of applications. Its unique feature is its ability to rapidly deploy as one can customize it at a very small duration of time; therefore, the growth rate of the open hardware platform and related application is extremely high. Recently, open hardware is being applied in sectors such as research and development as well as in robotics, consumer sector, electronics, entertainment, hobby drone projects, networking, and music industry.

Advanced microprocessor and microcontroller technology has made the open-source hardware philosophy possible and popular. One of the greatest pioneers of these open-source hardware concepts is Arduino (www.arduino.cc). Basically, Arduino is a community-driven project that is a software and hardware suite. An Arduino board can be purchased or assembled using “do-it-yourself” kits. This project is fundamentally based on a microcontroller family board manufactured primarily by Smart Projects, Italy, using various versions of 8-bit Alf and Vegard RISC or Advanced Virtual RISC (AVR) Atmel Microcontrollers (ATMEGA 328P) or 32-bit Atmel processors. These systems provide a series of digital and analog input/output (I/O) pins that can be interfaced to various extension devices, boards, sensors, and other analog or digital circuits. The boards have serial communication interfaces, including various versions of universal serial bus (USB) such as USB mini or USB B interface, depending upon the models, for loading programs in personal computers. In programming the Arduino microcontroller boards, the Arduino platform provides an integrated development environment based on Java that includes support for C and C++ programming languages. The Arduino software platform is based on a wiring language that provides a standard environment for interacting with the basic Arduino hardware. The Arduino UNO hardware is shown in Figure 5.1.

The Raspberry Pi Foundation is a charitable organization that developed a single-board computer named the Raspberry Pi, in 2011, as illustrated in Figure 5.2. This is a credit card-sized microcomputer that can perform a lot of sophisticated jobs like image processing, gaming, and web server hosting and can even be used as a smart television.

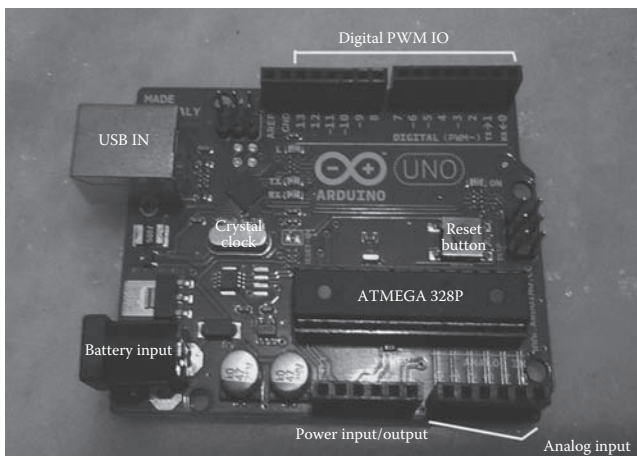


FIGURE 5.1
Arduino UNO.

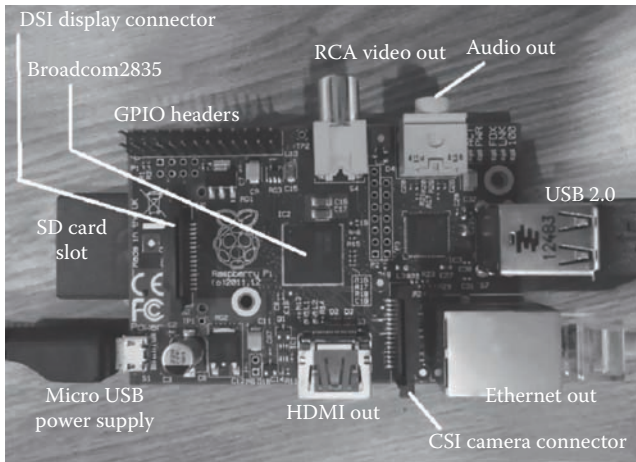


FIGURE 5.2
Raspberry Pi Model B.

5.3 Open-Source Hardware Licensing

Open-source hardware licenses generally support the recipients and the rebuilders of the design and documentation to study, update, redistribute, and finally distribute any further modifications. Additionally, open hardware licenses do not prevent anyone from divulging or even selling the project and its documentation. Very common open hardware licensing groups are the GNU Public License, Creative Commons Licensing, MIT, and Berkeley Software Distribution (BSD). These licenses are often pretty good at things like firmware, CAD drawings, and layout designing. They do not take into account the differences in hardware, particularly patents and derivative works.

5.4 Advantages and Disadvantages of Open-Source Hardware

Open-source hardware has several advantages. First, the design of circuit schematic diagrams is freely available. Therefore, one can easily update and modify the base design as per the requirements. In addition, as the licensing permits for redistribution, a new product can be manufactured from the open-source designs that might be salable in the market. For an example, an open-source prototyping board developed by the Arduino community has been modified by several other vendors; they made a customized board

that is completely based on the basic design of the Arduino. A good example of such kind of by-product is the Multiwii 2.5 CRIUS series autopilot board, which is a modified version of Arduino NANO. On the other hand, the ArduPilot mega series autopilot is a modified version of the Arduino Mega. The cloned version of the Arduino is mostly derived from the basic schematic of the Arduino.

5.5 Examples of Open-Source Hardware

5.5.1 Raspberry Pi Computer

Raspberry Pi is a very tiny and flexible single-board computer that consumes 4 W of power and costs between \$25 (model A) and \$35 (model B). Two basic flavors of this computer were designed initially in the United Kingdom. Various models with latest updates are available in the market; they are of higher configuration and expensive as well. This computer was first developed to assist school children in computer education. Raspberry Pi supports several open-source operating system platforms such as Pidora Linux (Pi version of Fedora), BSD Linux, and RISC OS. It supports several programming environments such as Python, C, C++, JAVA, Arduino, and Processing. Raspberry Pi was built by Eben Upton, whose main target was to create a hardware that is more lightweight and consumes less power compared to other computers but can run the latest programming platform like Python. The name Raspberry Pi is basically the combination of the name of a fruit (raspberry) with Python. The concept was to build a computer in the year 2005, but the vision turned into building a high-performance, single-board computer between 2006 and 2011. A few model versions are as follows: (1) Model A has no Ethernet interface networking that can only be possible through USB add-ons where 256 MB of RAM is presented and (2) Model B ver. 1 has 256 MB of RAM with an ARM7 processor and costs around \$35. It supports most peripherals. In Version 2, the size of RAM increased up to 512 MB. The processor of the computer is a Broadcom 2835 system on chip (SOC) that is fundamentally based on a 32-bit ARM RISC CPU core (it is not compatible with X86 architecture). The graphics processing unit (GPU) is VideoCore IV GPU. The default clock speed is 700 MHz. Any secured digital (SD) card is compatible and the kernel of Linux will boot from it. Two video output options are present in Raspberry Pi: One is the high-definition multimedia interface (HDMI), and the other is the digital visual interface via a cheap adapter. The two most common video standards, NTSC/PAL, are also supported. A wide range of resolutions display high-quality images. The audio connectors are basically available via either HDMI out or stereo output jack, but no audio input is available. Networking can be done via the RJ45 cable on model B with 10/100 Mbps. Wireless Internet may be configured

through a USB interface by including extra add-ons. The power source of the Raspberry Pi is primarily micro USB; a 1 A battery can provide the required power to drive the system. To drive a hard disk, 2 A of current is needed for this system. Most of the existing Raspberry Pi models have a current-limiting fuse in the USB socket path. Therefore, a high-powered peripheral device must add an external USB adapter to power up the device. The general purpose I/O is also present on the board such as parallel I/O ports UART (Linux console support). The I2C, SPI for peripheral support is present and the 3.3v logic via 26 pin header. Along with that, the DSI LCD panel support, CSI camera support, and additional general purpose input output (GPIO) are also available via the header.

5.5.2 BeagleBoard

Another very popular open-source hardware board is BeagleBoard (beagleboard.org) (Figure 5.3). Various types of this board exist in the market. Some popular types are BeagleBoard-XM, BeagleBone Black, and so on. BeagleBoard XM is an ARM Cortex-A8-based device and is cost efficient. Currently, it is available as a DM3730 processor manufactured by Texas Instruments. The early version of XM is BeagleBoard. There are several distinctions between BeagleBoard and BeagleBoard XM. BeagleBoard XM



FIGURE 5.3
BeagleBoard. (From beagleboard.org.)

has a 1 GHz ARM processor, whereas BeagleBoard has a 720 MHz processor. The double data rate RAM size of XM is 512 MB, whereas that of BeagleBoard is 256 MB. BeagleBoard XM has a camera header, overvoltage protection, power LED turnoff feature, and serial port power turnoff feature, whereas BeagleBoard has no such features. BeagleBoard is designed specifically to address the open-source community and has been designed with a minimum set of features that experiences the power of the processor in the context of an open-source development board. By utilizing standard interfaces, the BeagleBoard is highly capable of adding many functionalities and interfaces. It is not designed for use in end products. All the design information and schematics are freely available and can be used as the basis for a particular end product. BeagleBoards will not be sold for use in any product as this hampers the ability to get the boards to as many community members as possible and to grow the community. The BeagleBoard XM processor is available in DM3730CBP 1GHz version and it comes with a 0.4 mm pitch package on package (POP). The POP is a methodology where the memory is mounted on top of the processor. Because of this, when looking at the BeagleBoard, the user will not find the actual part labeled DM3730CBP but instead will find the memory part number. Additionally, some of the core features of BeagleBone are as follows. On the board, there are four USB A connectors. Each port can provide power on/off control and provide a power supply in the range of 500 mA to 5 V. The port cannot be powered by USB-OTG jacks. A standard 3.5 mm stereo audio output jack is provided to access the stereo output of onboard audio codec. A four-pin DIN connector has been provided to access the S-video output of the BeagleBoard. This is basically a separate output from the processor and contains different video output formats. The BeagleBoard can drive the LCD panel equipped with a DVI-D digital input. This is the standard LCD panel interface of the processor and will support 24-bit color output. A single micro SD connector is provided as an interface for the main nonvolatile memory storage on the board. An external SSD is also supported through USB. This replaces the 6 in. 2 SD/MMC connector found on the BeagleBoard.

5.5.3 PandaBoard

PandaBoard (pandaboard.org) (Figure 5.4) is a very-low-power development board/minicomputer that is based on OMAP4430 SOC manufactured by Texas Instruments. It runs on a 1 GHz dual-core ARM Cortex-A9 processor with 304 MHz power virtual reality graphics processing unit (VR GPU). It has a 2 GB POP LPDDR2 internal RAM as well as connectors for camera, LCD expansion, generic expansion, and composite video header. The PandaBoard has a 38.4 MHz 1.8 V CMOS square-wave oscillator. The FREF_SLICER_IN input (ball AG8) of the processor and the MCLK input of the TWL6040 Audio Companion IC have been driven through it. This clock is used as an input to the Phase Lock Loop within the OMAP4430 processor so that it can generate

**FIGURE 5.4**

PandaBoard. (From pandaboard.org.)

all the internal clock frequencies required for system operation. The device basically runs on a Linux kernel with Ubuntu, Android, or Firefox OS distribution, although a Ubuntu 12 or higher version may slow down the performance of the PandaBoard. Therefore, Xubuntu can be installed, which is a lightweight derivative of Ubuntu. In addition, Ubuntu can be tuned to perform by disabling the swap space. The swap space is basically a virtual memory that can be disabled from the `/etc/fstab` (just put a # mark before that). The board is also compatible with Windows CE, Palm OS, Windows Mobile, and Symbian OS.

5.6 Summary

In summary, various open-source platforms are available worldwide, and research is still ongoing to develop more eco-friendly and user-friendly hardware. Moreover, the variety of hardware product required explicitly depends upon the need of the technology and the user. As the technology changes, the system specifications also consequently change.

6

Open-Source Software Platform

6.1 Introduction

Open-source software is similar to proprietary software; however, it can be distinguished from the others by its license/terms of use that ensures certain freedom that proprietary software does not offer. Open-source software guarantees the right to access and modify the source code as it can use redistribution and reuse properties. In general, no royalty/service charges are applicable for open-source software, although at times there may be an obligation to share, update, and enhance open-source software products widely. As a result, the entire community benefits and enjoys the newly introduced features of that software.

Any open-source software guarantees the following criteria:

1. Reduction of software price to zero
2. No service charges
3. No vendor-specific locking and therefore no security vulnerabilities
4. Diversity of support as the service is community based
5. Encourage reusability
6. Improves the functionality of the key software

6.2 Open-Source Standards

The following are considered to be open-source standards:

1. Results are to be summarized through an open/autonomous process.
2. The standards are approved by specification and some standardization organizations such as World Wide Web Consortium (W3C), International Organization for Standardization (ISO), and Creative Commons.

3. Software should be systematically documented and publicly available with very low cost.
4. Intellectual property should be irrevocable on a royalty-free basis.
5. Overall, software can be deployed or shared within different development approaches.

6.2.1 Open-Source Software Licensing

Typically, open-source software licensing takes place under certain terms that provide the user with four different freedoms, which are as follows:

1. To view the source code
2. To use the source code uninterrupted without any access restriction
3. To redistribute the source code
4. To improve the source code and derogatorily publish the modified version of the software

The Open Source Initiative and Open Source Definition are the two standard communities that are recognized globally as certifying authorities, but there are many that authorize open-source licensing. Creative Commons and GNU Public License are the most widely used free software licenses. However, the legal and commercial overhead for managing open-source licensing is significantly reduced due to this flexibility. The term *free* has a dual meaning for the open-source community: primarily it means “zero price” and secondarily the liberty of use.

6.2.2 Free and Open-Source Software

The free and open-source software (FOSS) programs have a license that allows the users to freely run the software components for modification, development and research, education, and commercial use, with allowable free software distribution.

One vital cause for growth of the FOSS community is that the user has complete access to the software code that allows the repair of flaws/fault of the software.

FOSS does not have to be free of charge, besides being able to construct business models around the software based on commercial aspects. A company can receive direct payment by using a large number of licensing schemes and models. These models can be included in the overall definition of what we mean by FOSS. Thus, the source code is available to the customer.

6.3 Examples of Open-Source Software Products

A large variety of open-source software products are available online. Sites such as SourceForge provide users with the ability to get and deploy open-source software products. Nearly 70,000 categories of software are available; a few popular software products are as follows:

- *Linux*: This is probably the most well-known open-source software in the current era. Several versions of the free and open-source Linux such as Fedora, Ubuntu, and openSUSE Linux are available. A subset of these Linux versions is available in the form of open-source software, such as Pidora, that is specifically built for advanced RISC machine-based Raspberry Pi architecture. Lubuntu is a lightweight version of Ubuntu that has some specific features that are able to run in a very-low-configuration computer.
- *LibreOffice suite*: This is a quite famous bundle of software that has an open-source word processor called LibreOffice Writer. All the word processing tasks of this book have been done using this software, as we are a great fan of it. LibreOffice Draw is a drawing tool that has great versatility in making and drawing diagrams. LibreOffice Calc is a spreadsheet application that is highly used as a substitute of Microsoft Excel. LibreOffice Impress on the other hand is the open-source substitution of Microsoft Power Point.
- *Scilab*: This is an open-source version of MATLAB® that makes the MATLAB program portable to open-source environment. It is considered as quite a revolutionary software; therefore, Scilab can be used for applications where development cost is considered.
- *Gummi*: This is an open-source LaTeX editor and is very versatile and free to use. It gives a real-time document update in PDF format.
- *DigiKam*: This is a software like GNU Image Manipulation Program (GIMP) that is very popular nowadays for image editing and processing.
- Moreover, the open-source programming languages are taken as a revolution in the open-source movement. Languages like Python, Pearl, Ruby, and PHP are some of the leading language platforms. Although Java is one of the leading open-source giants, some features of Java are not available currently. MySQL is a good example of an open-source database in this context. Various software examples are given in Figures 6.1 through 6.4.



FIGURE 6.1
DigiKam software.

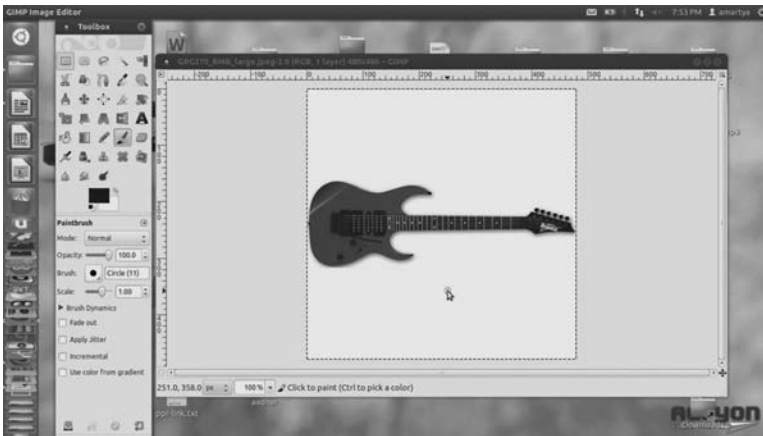


FIGURE 6.2
GIMP image editor.

6.4 Advantages and Limitations of Open-Source Software

Basically, the open-source concept is driven by community-based projects. Apart from research work, there are some convincing reasons causing developers to release their software as open source. One of the reasons is that the code provides more market share and that a development platform is built to offer long-term sustainability. Developers who do not prefer to commercialize their code consider open source as their best choice. As the coder is always in need of a particular project, many software products are being

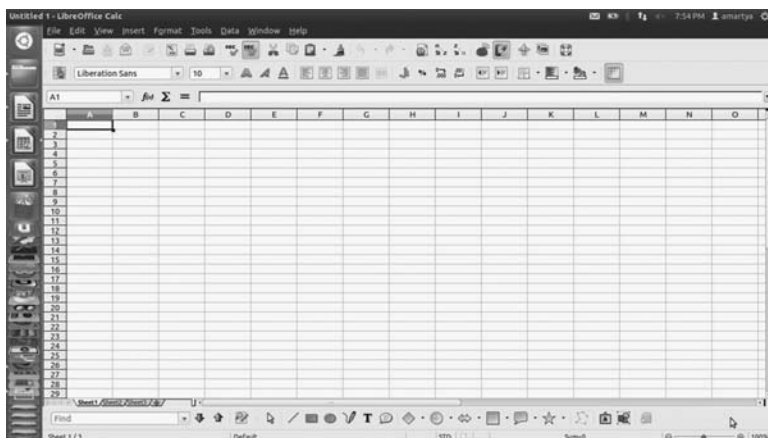


FIGURE 6.3
LibreOffice Calc software.

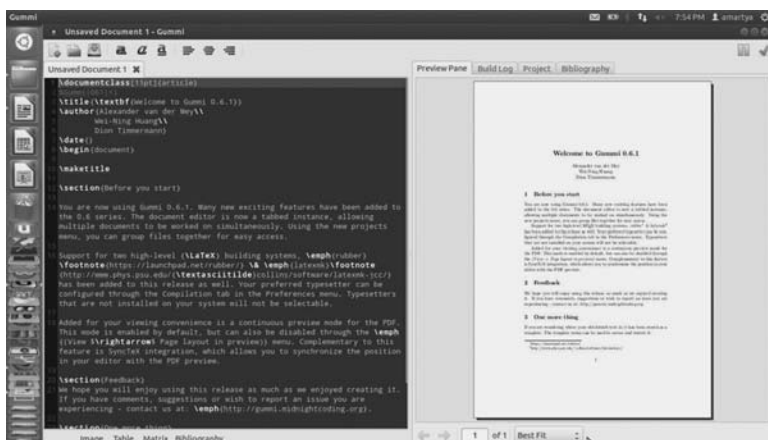


FIGURE 6.4
Gummi software.

released. These projects are then released to a massive development community and then to the end user. A superior example is the autopilot software such as Multiwii (<http://code.google.com/p/multiwii/>). This attracts interested coders, and therefore, the software is successively improved. Each group works on the same project out of its own interest, but on the whole, an exponential growth of the project may occur, becoming a tangible benefit for all users. Although the open-source community-driven project has a lot of advantages, the development community still faces more challenges. One such challenge is debugging faulty software components and providing proper service.

6.5 Open-Source Future

The Annual Future of Open Source Survey has emphasized that ^aOpen Source has now become a default choice.^o This survey reveals that 78% respondents are now running their business with open-source software. Among them, two-thirds of the software that has been built for their customer is based on open-source technology. The more significant news is that the percentage of the open-source respondents who actually participated in open-source development has increased from 50% to 64%; in addition, 88% respondents declared that they expect to contribute to open source projects within the next 3 years.

7

Automated Plant-Watering System

7.1 Introduction

A plant-watering system may be useful when water scarcity exists. It is highly useful in rural areas in the desert or in regions where there is less rainfall. It is an automated sensor-based system that measures soil moisture to calculate the volume of water to be taken for watering using a portable pumping unit. For cultivation to be successful, several parameters that affect the composition of the soil are to be considered.

This system is an automated miniaturized system for intelligent irrigation, which can be divided into two parts: (1) a sensor node that deploys into the field and (2) a receiver device that receives the data sent by the sensor node. The receiver is placed in the control room near the irrigation field. Then, the data from the receiver are broadcasted via cloud-hosting sites. In addition, based on these data, the sensor controls the pump unit to provide an optimum measurement of water to be given to the soil. Finally, when the water level exceeds the threshold level, the microcontroller unit automatically stops the pump.

7.2 Architecture of Plant-Watering Systems

The system architecture of the sensor node and the receiving system is shown in Figures 7.1 and 7.2. The plant-watering system consists of two different layers. The first layer consists of a sensor hub that collects data from the sensor in real time and drives the pumping unit based on the predefined logic given to the Arduino. In addition, the sensor hub is used to broadcast information on soil moisture to the remote base station via a one-channel radio unit of 433 MHz. The second layer consists of a radio-receiving unit that is connected to a personal computer where data get broadcast to cloud service via HTTP request protocol. Special open-source software called ^aProcessing^o is used to interact with the receiver hardware and the cloud-based sensor data-hosting site.

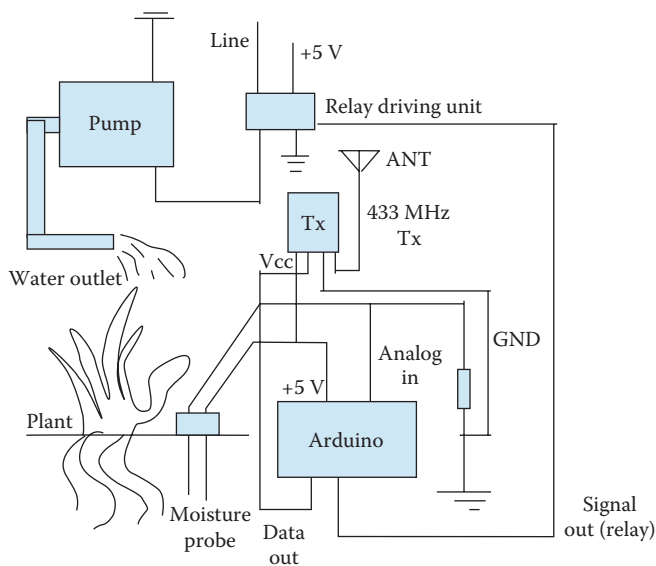


FIGURE 7.1
The sensor node with pump controller (Tx).

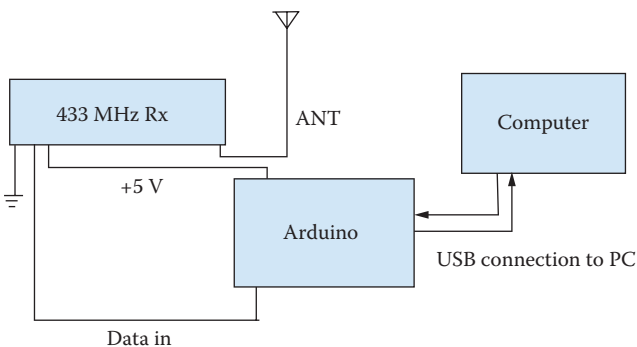
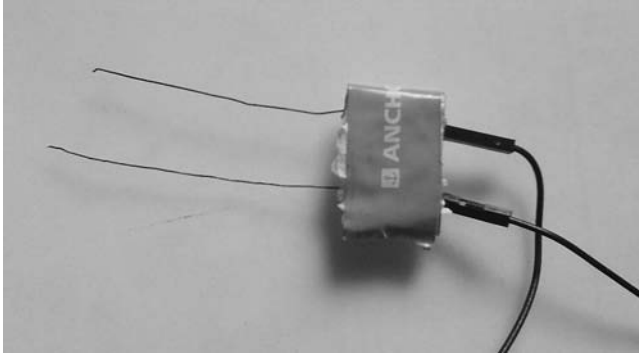


FIGURE 7.2
The receiver side.

7.2.1 Soil Moisture Sensor

The soil moisture sensor is the primary part of this project. Here, we have made a simple homemade soil moisture probe as shown in Figure 7.3. It is simply an analog device that contains two probes, one connected to the Vcc and the other connected to the analog pin of Arduino via a voltage divider circuit. Basically, the sensor that we have made has no polarity and works as a variable resistance. The sensor operates based on the water flow

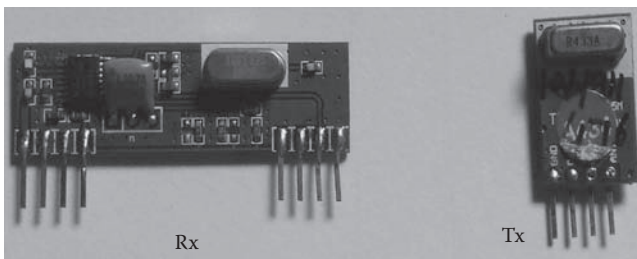
**FIGURE 7.3**

The soil moisture sensor.

through the soil, as it is placed deeper into the soil. If the volume of water is high, then high electric current is passed from one pole of the sensor to another, thus decreasing the resistance value. As the volume of water decreases, the resistance increases, and the value of soil moisture changes. The logic circuit has been designed in such a way that if the threshold value of soil moisture decreases below 10 units, it immediately starts the pumping unit, whereas if soil moisture increases above the threshold value, it immediately stops the pump.

7.2.2 Setting Up 433 MHz Radio Tx/Rx Module

In this stage, it is required that the soil moisture data received by the Arduino be sent to the base station via a 433 MHz radio frequency (RF) module. Essentially, this is a one-channel module that has a single transmitter and a single receiver as demonstrated in Figure 74. The transmitter (Tx) has four pinouts named from left to right as GND, DATA, Vcc, and ANT, whereas the receiver (Rx) has eight pinouts named from left to right as ANT, GND, GND, GND, GND, GND, GND, and GND.

**FIGURE 7.4**

The 433 MHz transmitter/receiver unit.

Vcc, Vcc, DATA, DATA, and GND. Both the Tx and Rx can run +5 V with a transmission range of up to 200 m with a proper antenna.

After setting up the radio transmitter, we have to write the programming code on the Arduino, including the VirtualWire.h header file that helps to set up an RF link through the RF module. Both the Tx and Rx modules need this file to use necessary functions. Here, the function `vw_set_ptt_inverted(true)` is used to establish the RF link, while `vw_setup(2000)` is used for mentioning the bits per second for transmission. The `vw_set_tx_pin()` function refers the Tx pin number of Arduino to communicate and the `vw_send((uint8_t *)msg, strlen(msg))` function actually sends the string message, while the function `vw_wait_tx()` method has been used to wait for the transmission ends.

At the receiver end, the data are to be received using the `vw_set_rx_pin()` function. The function `vw_rx_start()` is used to start the data receive operation from the RF receiver to the Arduino. In the loop function, we have to declare a buffer that carries the message in a `uint8_t` data type array, which is an unsigned character whose range is from 0 to 255 and takes 8 bits of memory.

7.2.3 Setting Up the Pumping Device

The pumping device is connected to the Arduino via a relay driver module shown in Figure 7.5 that uses a relay driver unit called DIY, which consists of an electromechanical relay (its working principle is discussed in Chapter 9),

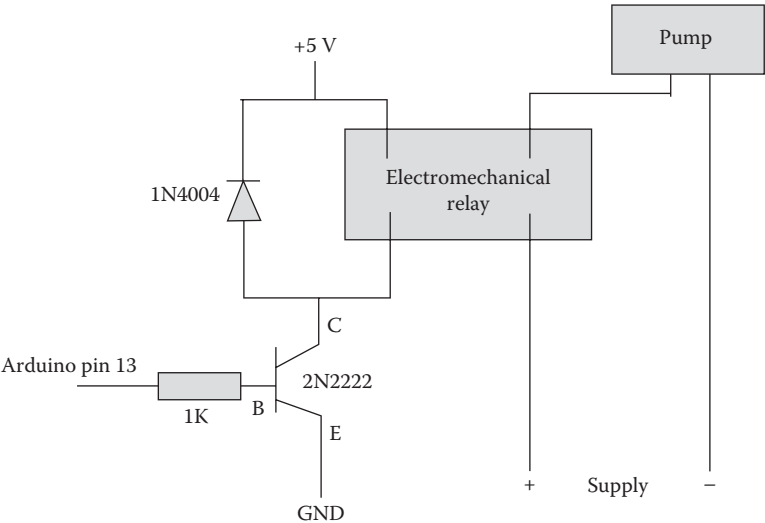


FIGURE 7.5
The relay driver module.

a 2N2222 transistor, a 1N4004 diode, and 1K resistance. The Arduino is connected to the transistor base and acts as a switch. As the transistor base receives a signal from the Arduino, it immediately excites the primary coil that activated the relay and instantly the main coil gets activated and starts the pumping unit. If the Arduino stops sending signals, the relay is immediately cut, as the primary coil becomes an open circuit.

7.3 Arduino Programming Code

After setting up the radio transmitter and receiver, the programming code on the Arduino should be written as follows.

7.3.1 Arduino Code for the Radio Transmitter

```
#include <VirtualWire.h>

int Pump=13;

void setup()
{
    vw_set_ptt_inverted(true); // Radio Link
    vw_setup(2000);           // Bits per sec
    vw_set_tx_pin(3);         // Tx link module is
                              // assigned in this pin

    Serial.begin(9600);

    pinMode(led,OUTPUT);
}

void loop()
{
    int sig = analogRead(A0);
    if(sig<10)

        digitalWrite(Pump,HIGH);// start pump

    else

        digitalWrite(Pump,LOW);//stop pump
    char *str  ="hii"; ;
```


7.4 Broadcasting Sensor Data to the Internet via Processing

The broadcasting of sensor data at the receiving end is quite a challenging task. The implementation of this data broadcast methodology can be done with the help of a processing tool. The processing tool (www.processing.org) supports the eeml (<http://www.eeml.org/library/>) library that is specifically designed for interaction with the free cloud-based sensor data-hosting site like Xively (xively.com).

The processing code to interact with the Xively Service is as follows:

```
import eeml.*;
import processing.serial.*;
DataOut dOut;

String data=null;

float lastUpdate;
String b="hey";
int d;
Serial port;
void setup(){

    Serial.list();

    port = new Serial(this, "/dev/ttyACM0" , 9600);

    //port.bufferUntil('\n');

    dOut = new
        DataOut(this,"https://api.xively.com/v2/feeds/775407089.xml",
            "s0L5YbLf7LPZ2QSSu92qyfW0z0FokANT7Jv9txXHyf0l7F4");

    dOut.addData(0,"LDR Sensor...");
}
void draw(){
    b = port.readString();
    //d=float(port.readStringUntil('\n'));
    try{

        d=Integer.parseInt(b.trim());

    }catch(NullPointerException ne){

        d=12;
    }
    //padding null values
}
```

```

        catch(NumberFormatException ne){
// pass on
            d=11; // padding unwanted values}

        println(d);

        if ((millis() - lastUpdate) > 5000){

            println("ready to PUT:");

            dOut.update(0, d);

            int response = dOut.updatePachube();

            println(response);

            lastUpdate = millis();

        }
        delay(200);

    }
    void onReceiveRequest(DataOut d1){

        d1.update(0, d);

    }
}

```

To deploy the sensor data to the Xively service, a free account has to be created. Then the device that sends the sensor data to the designed profile has to be added, as illustrated in Figure 7.6. In the channel setting option, the channel name (usually, the name of the sensor from where the feed is coming from), initial value, units, and so on have to be added. In addition, the location from where the feed is coming has to be added in the location field. After the device has been added successfully, Xively will give us a feed ID as in <https://api.xively.com/v2/feeds/775407089>, as shown in Figure 7.7. This feed ID should be fed to the processing application that connects the device to the Xively. The Xively also provides an application programming interface (API) key, for example, (s0L5YbLf r7LPZ2QsSu92qyfw0z0Fo-kANT7Jv9txXH yf0l7F4), as illustrated in Figure 7.8. This API key is used to validate and authenticate the device that is currently connected to the cloud service. After the device has been successfully connected to the Xively service, the data will be sent via the HTTP GET or POST method in Xively console; the value is shown in Figure 7.9. This figure shows the status of the feed. If the code is 404, it means that the feed is not connected to the device, whereas if the code is 400, then the device is not authenticated. If the

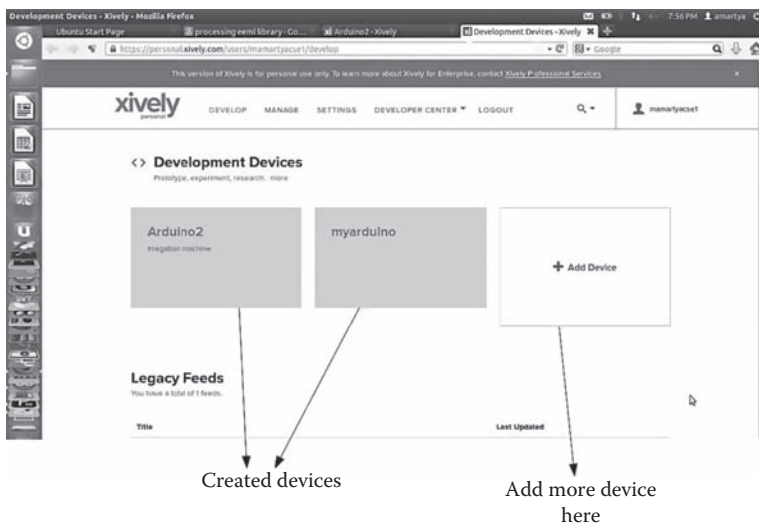


FIGURE 7.6
The Xively home page.



FIGURE 7.7
The Xively feed URL.

API Keys

Auto-generated Arduino2 device key for feed
775407089

sOL5YbLfr7LPZ2QSsU92qyfWOzOFokANT7Jv9txXHfOI7F4
permissions READ,UPDATE,CREATE,DELETE
private access

API key that is needed to access the Xively service

FIGURE 7.8
The API key.

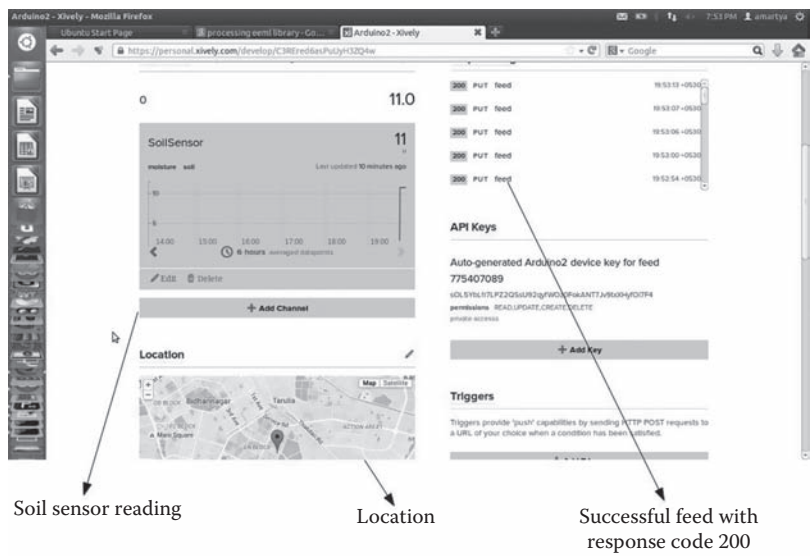


FIGURE 7.9
The sensor data broadcast via Xively.

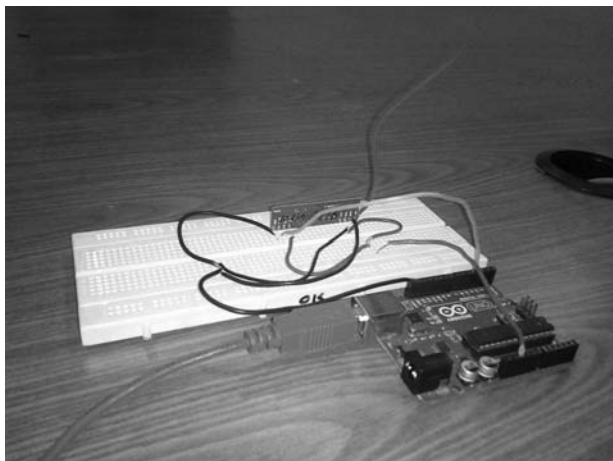


FIGURE 7.10
Receiver unit.

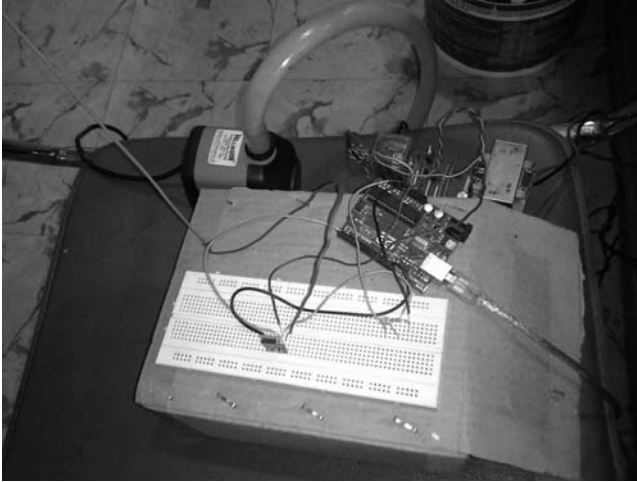


FIGURE 7.11
Irrigation controller.

response code is 200, then the service establishes an authenticated device and fetches the data from the device connected. The final device setup of the Xively output is shown in Figures 7.9 through 7.11.

7.5 Summary

In this chapter, we have described how to connect a hardware device to a third-party cloud-service provider via processing language. Processing eeml library supports helps us make a direct connection with the Xively service; therefore, it is quite simple to deploy our application via cloud. The feed sent to the service will be updated on a real-time basis, and the status of the soil can be viewed from anywhere in the world. This concept is formally known as device to cloud mechanism and is a very powerful system nowadays.

7.6 Concepts Covered in This Chapter

- Data transmission through Arduino and 433 MHz RF module
- Sending data feed to a cloud-based data-hosting service (xively.com)
- Controlling a pumping device by processing soil moisture value
- Implementation of a prototype of an automated irrigation machine

8

Device to Cloud System

8.1 Introduction

Cloud computing is defined as the sharing of computing resources via the Internet. It is a borderline idea of service sharing and infrastructure convergence. The main advantage of this *cloud* concept is its capability of dynamically sharing and reallocating the resources among an n number of users concurrently. For example, the Internet giant Google provides a large number of cloud services for storing and managing data. Some wonderful applications in this context are Google Docs and Spreadsheet offered by Google Drive cloud service. Although Google Drive is basically a storage service, its service becomes interactive as the docs and spreadsheet applications are incorporated. The user can edit documents, perform mathematical calculations, generate graphs and charts, etc. The significance of this service is that it allows users to create and edit documents online, collaborating with other users from computers, mobile phones, tablets, and many more.

The term *device to cloud* is used when a mobile, sensor-based embedded device or any other handheld device interacts with a cloud service. This concept can be visualized as a means in which either information is transferred from a specific device to a cloud service or vice versa.

In this chapter, we will discuss a device to cloud concept through the Arduino platform along with the ZigBee communication module (IEEE802.15.4 Tx/Rx) that is used to send data to a Raspberry Pi computer. The computer then interacts with the Google cloud platform via a Python script.

8.2 Temperature Sensor Data Logging System

8.2.1 Interacting with Cloud

In this project, a temperature sensor data logging system is implemented. The implementation of the project is done in two steps. First, the Arduino and

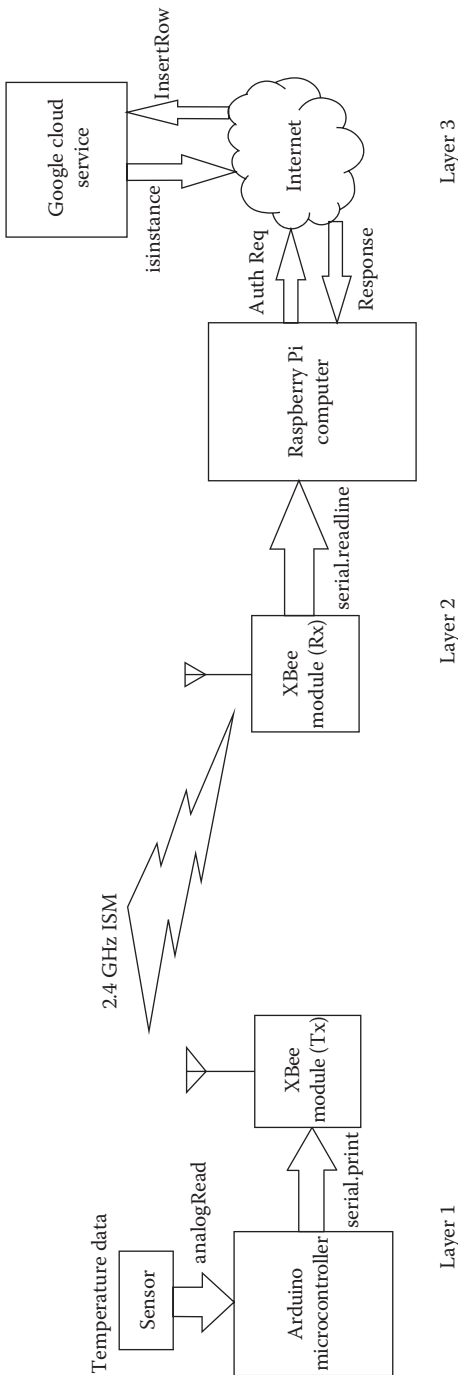


FIGURE 8.1
The temperature sensor data-logging system architecture.

Raspberry Pi computers are connected via ZigBee protocol with a communication bridge among them. Second, a Python script to push the data given from the XBee module to the cloud is written/executed. Mainly, the entire system has an architecture that consists of three layers as shown in Figure 8.1. At layer 1, the physical level sensor node is implemented with a sensor module (LM35), a microcontroller unit (Arduino UNO), and a communication module (XBee) to send data to the Internet gateway. At layer 2, the bridge node is implemented using a computer (typically Raspberry Pi). Here, a 2.4 GHz XBee receiver module will receive the temperature data. After receiving the data, a software program (developed using Python) establishes a connection with Google Spreadsheet cloud service. Finally, layer 3 comprises the cloud service itself and thus consists of the data storage and the authentication procedure. When data are pushed from layer 2 to layer 3, authentication becomes mandatory via Google account authentication procedure.

8.3 Components

The components required to execute this project are as follows:

- One Arduino UNO R3 board
- One LM35 temperature sensor
- Two XBee radios
- One XBee shield
- One XBee adapter
- One Raspberry Pi computer module
- One universal serial bus (USB) cable and one Ethernet cable
- One 12 V, 900 mA power adapter

8.4 Temperature Sensor

The temperature sensor used in this project is an LM35 precision integrated circuit temperature sensor and is shown in Figures 8.2 and 8.3. A temperature of 1°C gives an output voltage of 10 mV. It has three pins from left to right: +Vs, Vout, and GND, respectively. The sensor can be operated from a 4±20 V DC supply. The sensor does not require any kind of external calibration, and it provides ±0.25°C accuracy in room temperature. It is presumed that the ambient temperature is equal to the surface temperature of the sensor module in the TO-92 plastic package sensor.

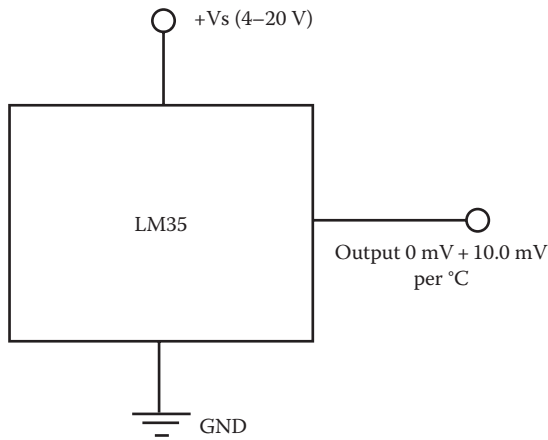


FIGURE 8.2
Sensor circuit.

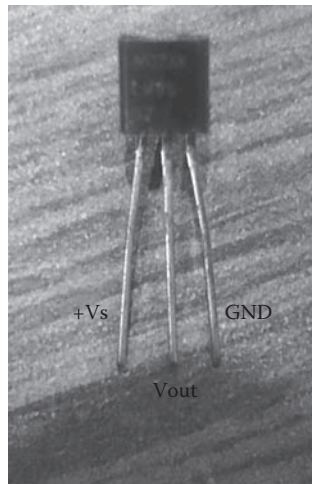


FIGURE 8.3
LM35 TO-92 package.

By changing the reference voltage level, the accuracy of the analog-to-digital conversion can be improved significantly. A reading of 10 bits by the Arduino basically means 1024 steps. The input range maximum will be given by the reference voltage that is set to the default value of +5 V. It is available to change the reference as an internal reference that gives 1.1 V. Therefore, 1.1 V is now the maximum input voltage for sensor. As the aRef has changed to 1.1 V, the highest possible resolution for the LM35 sensor can be achieved.

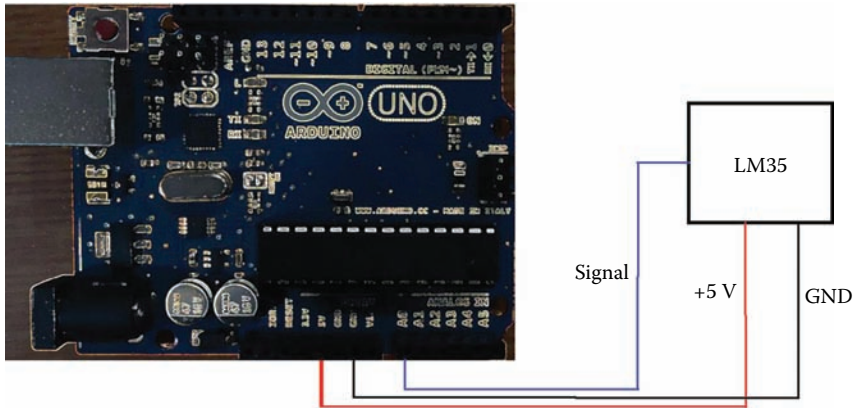


FIGURE 8.4
Connection diagram for the temperature sensor.

8.5 Circuit Connections

Arduino has six analog input pins, from A0 to A5, and thus, it is possible to use any pin as a sensor input (as LM35 is an analog sensor). In LM35, the middle lead is typically used as a signal or Vout pin, and the input voltage is applied at +Vs pin. The allowable voltage range for the LM35 sensor is between +4 and +20 V. In addition, the voltage required can be taken either from Arduino (+5 V supply) itself or from any regulated DC power supply, whereas the GND pin must be connected to the Arduino ground. Figure 8.4 shows the connections for the temperature sensor.

Assume that the float variable `tempinC` is used to store the finally calibrated temperature data in decimal. The `int sensval` is used to store the raw sensor data from analog input. In addition, the `int temp` is the input pin of temperature sensor (typically A0 here). Therefore, the source code to obtain the temperature value from the temperature sensor is given as follows:

```
float tempinC;
int sensval;
int temp = 0;

void setup()
{
    analogReference(INTERNAL);
    Serial.begin(9600);
}
```



```
void loop()
{
  sensval = analogRead(temp);
  tempinC = sensval / 9.31;
  Serial.println(tempinC, DEC);
}
```

Here, the function `analogReference(INTERNAL)` has been used to assign the reference voltage as 1.1 V in `aRef`. If you divide 1.1 V over 1024 (Arduino read 10 bits = ~ 1024 steps), each step of reading the analog signal is equal to approximately 0.001074 V that is equal to 1.0742 mV. If 1°C is equal to 10 mV, then $10/1.0742 = \sim 9.31$. This means that the temperature changes by 1°C for each 9.31 change in the analog temperature reading. The function `Serial.begin(9600)` will initialize the serial port with 9600 baud rate. `Serial.println(tempinC, DEC)` will print the value of the temperature in decimal format. Figure 8.5 shows the Arduino with LM35 and XBee shield.

8.6 Setting Up Zigbee Communication

8.6.1 Zigbee Basics

Zigbee is an IEEE802.15.4 standard that is widely used to construct the personal area network like Bluetooth technology as illustrated in Figures 8.6 and 8.7.

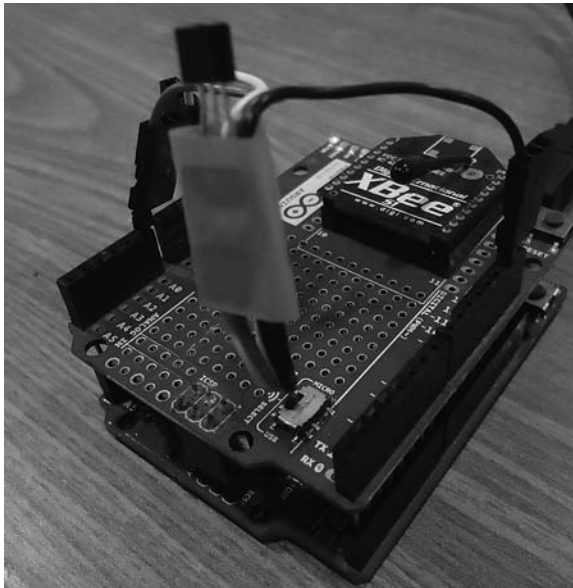


FIGURE 8.5
Arduino with LM35 and XBee shield.

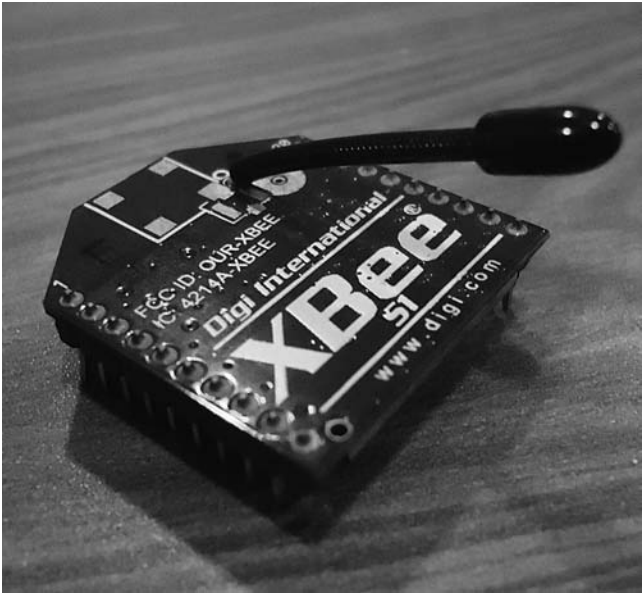


FIGURE 8.6
Xbee radio.

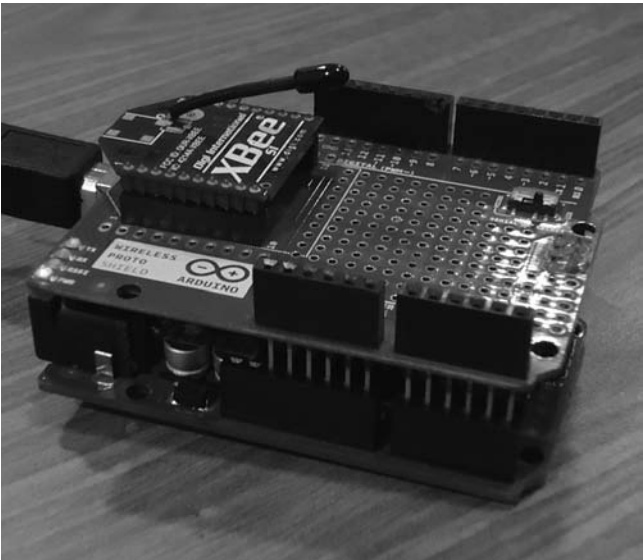


FIGURE 8.7
Arduino Xbee shield.

It has a transmission range of about 10 ± 100 m. It runs at 2.4 GHz on the industrial scientific and medical band. The transmission speed of the Zigbee radio is about 250 kbps. As the transmission range is low, it is mostly used for very-short-range data communication. It is widely used in sensor network deployment and data transfer. However, the transmission range can be augmented by establishing a Zigbee mesh network system.

ZigBee technology is greatly supported by Arduino, as Arduino provides a direct ZigBee support in the form of XBee shield, where XBee is a brand of ZigBee standard provided by digi (<http://www.digi.com/products/>).

8.6.2 Configuring XBee Module

An XBee adapter is necessary to configure the XBee module through a USB cable as shown in Figure 8.8. First, XBee radio is put on the XBee adapter by maintaining proper direction. Then it is connected to a computer via USB. Run a hyperterminal program such as PuTTY (as in <http://www.chiark.greenend.org.uk/~sgtatham/putty/>), as shown in Figure 8.9. Then XBee radio is configured, and while opening the PuTTY window, a configuration window will appear. Here, the serial port option for the XBee adapter and baud rate of the serial data transfer (9600 by default) are selected. A terminal menu will appear at the left-hand side of the PuTTY configuration window.

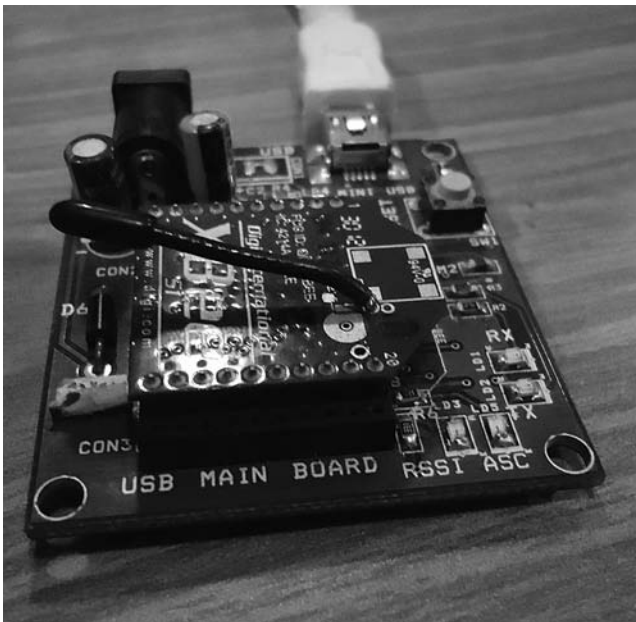


FIGURE 8.8

XBee module with adapter.

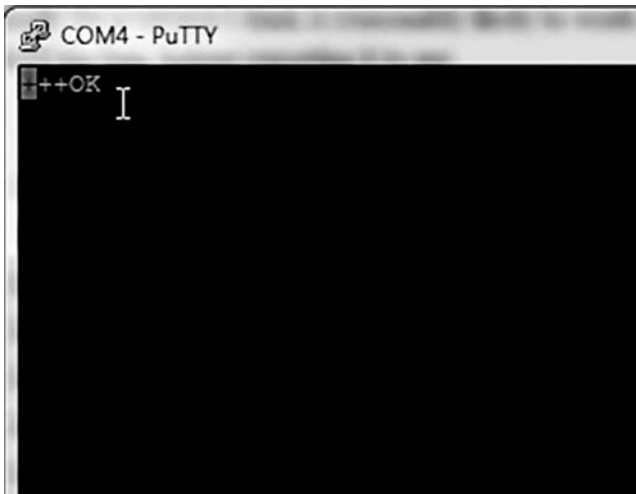


FIGURE 8.9
PuTTY terminal.

A new window will appear by clicking that. In the line discipline option, you must select local echo to force on, then go back to session, and press OK.

Now, the XBee wireless module is connected and the task is to program the XBee module using the following instruction:

- Enter programming mode type `+++`. An OK message will appear.
- Then type `ATMY1000`. This sets the module ID to 1000; press enter to get OK.
- Then type `ATDL1001`. This sets the destination module ID 1001; press enter to get OK.
- Now type `ATID1111`, (use comma) and press enter to get OK. Set the personal area network ID to 1111.
- Then type `WR` and press enter to get OK. This means that settings have been written.

The same procedure is followed for the second Xbee module to configure it for the same personal area network; only the host and the destination ID must be swapped. Now, the XBee modules are ready to communicate with each other.

Note that to configure the XBee radio, an Arduino board is to be connected to an XBee shield, but to do so we must physically remove the microcontroller to program the XBee radio.

To check whether data are available, a Python script has been introduced that can print serial data from the USB is written. As this project focuses to connect the XBee receiver with the Raspberry Pi minicomputer, the name

of the USB port at its operating system (OS) should be known. In Raspberry Pi, a lightweight version of Fedora is used (Pidora is shown in Figure 8.13), which will be discussed later. To fetch data from the USB port typically in the Linux environment, the name of the USB port connected to the computer should be known. In most Linux systems, it is situated under `/dev` directory. So to obtain the entire terminal in Linux, the command `ls /dev/tty*` is to be written. This gives all the terminal present in the system. To get a list of the USB port, the command `ls /dev/ttyUSB*` is to be given. The PySerial API for the Python serial port support is available at <http://pypi.python.org/pypi/pyserial>. To unpack the archive, enter the *pyserial-x.y* directory and run **python setup.py install**, or for python 3.x, run **python3 setup.py install**; the installation process will automatically start.

8.7 Sample Python Code for Serial Read

```
import serial
mser = serial.Serial('/dev/ttyUSB0', 9600)
p = mser.readline()
print p
```

After the attachment of ZigBee and Raspberry Pi as shown in Figure 8.10, the serial data coming from the USB are tested in the Linux environment by writing a simple Python script as shown earlier. Here, the serial API is imported, and an object of the serial port is created by calling `ser = serial.Serial('/dev/ttyUSB0', 9600)`, where `/dev/ttyUSB0` is the USB port name and 9600 is the baud rate. The `ser.readline()` function will return a string data by reading the serial port. The print statement will print the value on the console (Figure 8.11).

In a Windows environment, we can use the X-CTU terminal software (<http://www.digi.com/products/wireless-wired-embedded-solutions/zigbee-rf-modules/xctu>) to test the message obtained by the XBee transceiver. The USB XBee adapter (or Arduino shield) in the USB port and the X-CTU software automatically detect the communication port corresponding to the XBee adapter with a baud rate of 9600.

8.8 Sending Data to Cloud

The next step of the project is *talking to clouds*. To do this, we need a Raspberry Pi minicomputer module. It is preferable to use the Raspberry Pi because of its tiny size, high processing power, and low power consumption. A Raspberry Pi module can be used as a small-size sensor network node that can act as a

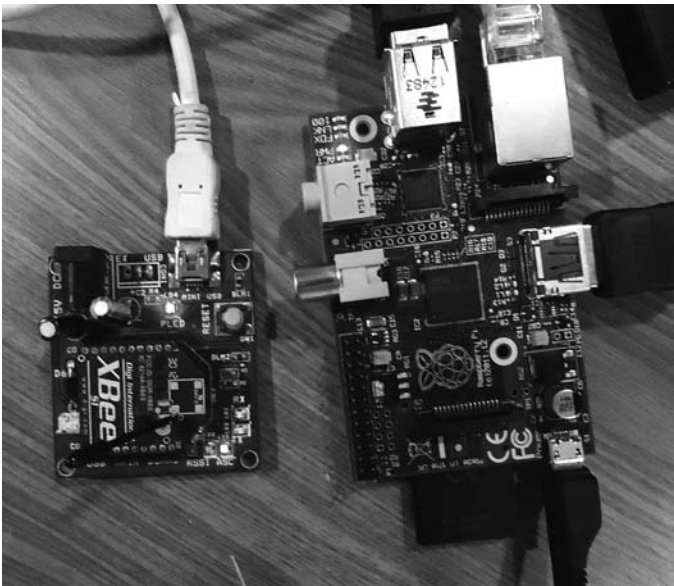


FIGURE 8.10
XBee radio connected with Raspberry Pi via USB.

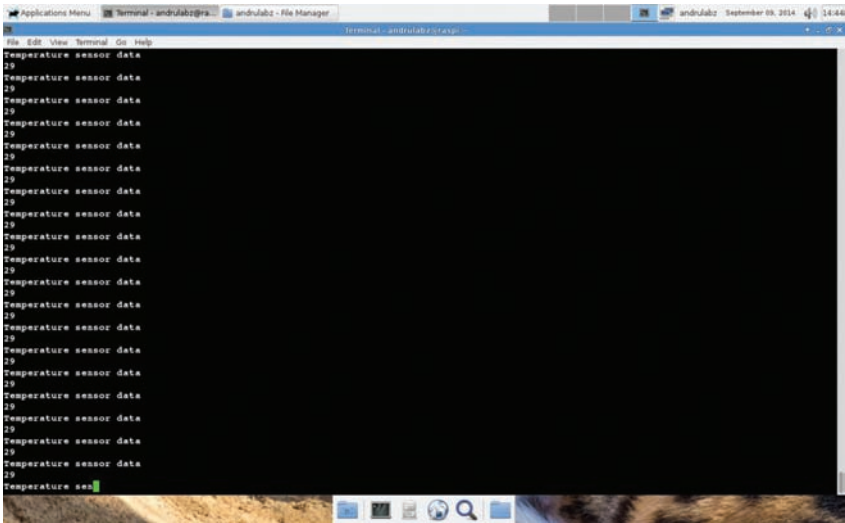


FIGURE 8.11
Serial data at Raspberry Pi terminal.

gateway to the cloud service. It is easily deployable anywhere with battery power (+9 V battery) and Internet support and acts as a bridge node between cloud and sensor networks.

8.8.1 More about Raspberry Pi

The Raspberry Pi is a single-board computer system, as shown in Figure 8.12. It is as tiny as a match box. The main power of this computer system is its processor BroadcomBCM2835 system-on-chip multimedia processing unit. This clearly implies that a wide range of graphics, audio, and communication capability are incorporated within the system. The system has 256 MB (for model B v2; currently, a B+ model is available with 512 MB of RAM) of RAM and up to 40 GB expandable static nonvolatile memory slot (SD card). Its BCM2835 processor is basically an advanced RISC machine (ARMv7) processor having 700 MHz clock frequency. The instruction set is completely different from X86, as this is a reduced instruction set computer, and hence, this machine can be operated with low power. The most popular OS that can be used in the Raspberry Pi are (1) Raspbian Wheezy, a lightweight version of Debian Linux; (2) Pidora, a lightweight version of Fedora; (3) RaspBMC, a media center±OS similar to Xbox media center (but this is the open-source version of the media center software); (4) RISC OS, a non-Linux version OS exclusively designed for RISC processors; and (5) ARCH Linux, a very lightweight Linux platform for handheld device highly suited for ARM Architecture (link: <http://www.raspberrypi.org/downloads/>).

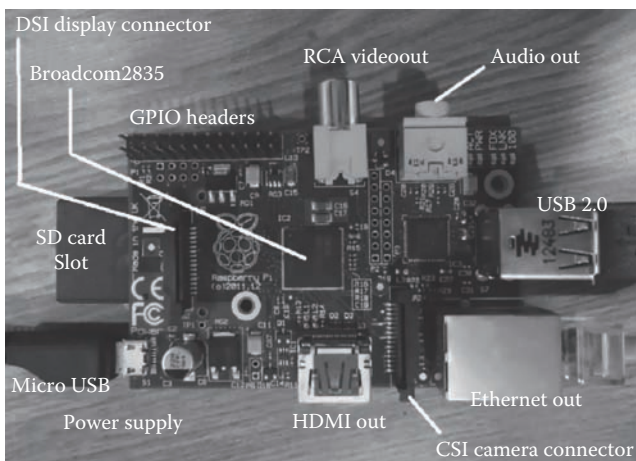


FIGURE 8.12
Raspberry Pi computer.

8.8.2 Main Components

The Broadcom BCM2835 700 MHz ARM1176JZFS processor has a floating point processing unit and a VideoCore 4 graphics processing unit (GPU). The GPU provides Open GL ES 2.0 support (Graphics Library), hardware-accelerated OpenVG (video graphics), and 1080p30 H.264 high-profile decoding technique. The GPU is capable of 1 Gpixel/s, 1.5 Gtexel/s, or 24 GFLOPs with texture filtering and DMA infrastructure. Besides the use of the 10/100 BaseT Ethernet, HDMI, 2 USB 2.0 port (in the latest version B+ model two additional USB ports are added) and RCA video output is available. The SD card socket is 40 GB expandable, is powered from a micro-USB socket (minimum power requirement is +5 V 1 A), and has 3.5 mm audio out jack.

8.9 Installation of Operating System and Python API in Raspberry Pi

8.9.1 OS Installation

The installation procedure of Pidora OS is quite easy. Download the Pidora image from the link <http://pidora.ca/>.

In the Linux platform, insert the SD card (at least 8 GB) to the computer card slot using a card reader and type *df-h*. A message appears as `/dev/sdd1`, where `sdd1` is the partition number. This may vary according to the number of partitions available in the SD card. Now, unmount the SD card using command `umount /dev/sdd1` so that the image is written on the SD card. To write the .img image file into SD card, type `dd bs= 4M if =pidora-18-r2c.img of = /dev/sdd1`. Then write the image to the SD card.

Use the Win32DiskImager program (<http://sourceforge.net/projects/win32diskimager/>) to install the Pidora OS using a Windows platform. Just extract the zip file, insert the SD card in card reader, and execute the program by putting the path of the pidora.img file as shown in Figure 8.13. Pidora is a remix of Fedora project exclusively designed for Raspberry Pi.

The latest version of the OS is Pidora 20, kernel version 3.12.23. The package is basically a combination of Fedora with other third-party software. Pidora project was developed by Seneca Centre for Development of Open Technology (<http://cdot.senecacollege.ca/>). Exclusive features of the OS are as follows:

1. Exclusive looks and feel.
2. Graphical first boot configuration.

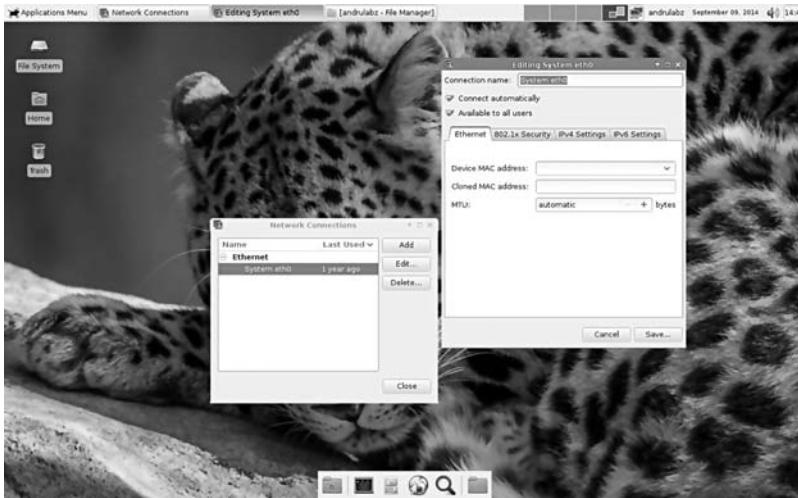


FIGURE 8.13
Pidora operating system.

3. Specifically compiled to utilize the maximum resources of Raspberry Pi.
4. Automatic creation of swap memory on demand.
5. Support for C by default, Python, and Pearl programming languages.
6. Network information is readable from audio output as well as LED blink.

8.9.2 pySerial Installation

The pySerial API can be obtained from the link <https://pypi.python.org/pypi/pyserial>. In Pidora platform, open the terminal window, and then type the following command to install pySerial in Raspberry Pi.

```
mkdir pyserial-2.6
tar -zxvf pyserial-2.6.tar.gz pyserial-2.6
cd pyserial-2.6
python setup.py install
```

8.9.3 Python Google Spreadsheet API Installation

The main interest is to upload the temperature data to the Google cloud (typically Google Spreadsheet in our case); we need the spreadsheet API for Python distribution (<http://code.google.com/p/gdata-python-client/downloads/list>) to get gdata-2.0.18.tar.gz. Now, type the following commands to install gdata in Raspberry Pi.

```
tar -xf gdata-2.0.18.tar.gz
cd gdata-2.0.18
python setup.py install
```

After successful installation of the gdata API, we can access the Google Spreadsheet from a Python script and can send the sensor data. Make sure that the Raspberry Pi is connected to the Internet cable or a 3G dongle.

8.10 Configuring Google Account

Due to security reasons, the Google cloud server does not make use of the direct authentication procedure from any third-party application. Make sure that you have already done the two-step authentication in Google account. Afterward you must seek an application (App) password that can be used to access the spreadsheet from your native application. As you get the App password from your registered Google account, you can now access the Google Spreadsheet from the Python application running at your local Raspberry Pi computer. After the completion of authentication, just put the App password offered by Google in place of the actual password of your Google account written in Python code (in the code the user ID and password should be delivered). At this instant, just create a new spreadsheet from the authenticated Google account and name it as *'temp_1'* and take the spreadsheet key from the address bar of the web browser as shown in Figure 8.14. The spreadsheet key is a unique ID through which the services identify each and every file separately.

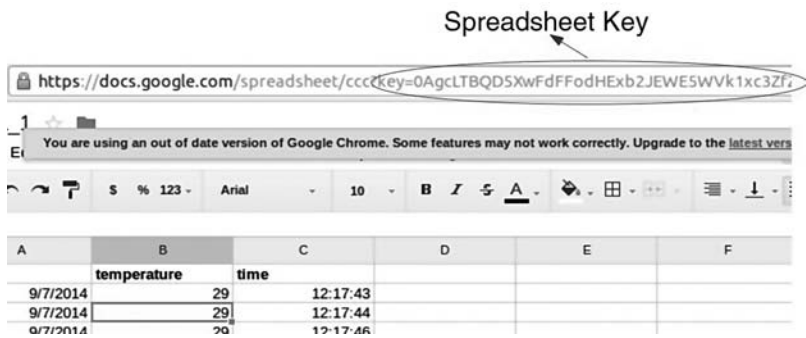


FIGURE 8.14
Google Spreadsheet URL.

8.11 Python Code to Access Google Spreadsheet

A python code to access the Google Spreadsheet is as follows:

```
#!/usr/bin/python
import serial
import time
import gdata.spreadsheet.service
newser= serial.Serial('/dev/ttyUSB0',9600)
mymail = 'my.gml.id1432@gmail.com'
paswd = 'rchqtsgzfyboakbr'
spsht_ky = '0AgcLTBQD5XwFdFFodHExb2JEWE5WVk1xc3ZfZDBa0Wo'
ws_id = 'od6'
sp_cnt = gdata.spreadsheet.service.SpreadsheetsService()
sp_cnt.email = mymail
sp_cnt.password = paswd
sp_cnt.source = 'temp_1'
sp_cnt.ProgrammaticLogin()
i=0
while(i<200):
    i=i+1
    dic= {}
    dic['date'] = time.strftime('%m/%d/%Y')
    dic['time'] = time.strftime('%H:%M:%S')
    dic['Temperature'] = newser.readline()
    print dic
    e = sp_cnt.InsertRow(dic, spsht_ky, ws_id)
    if isinstance(e, gdata.spreadsheet.SpreadsheetsList):
        print "row inserted successfully"
    else:
        print "unable to insert row"
```

In this code, the serial, time, and spread service of gdata API are imported. Here, we store the temperature data with time stamp (time and date) at Google Spreadsheet. The 'newser' variable is a serial port object that is used to connect and fetch the serial data. The 'paswd' variable stores the app password given by the Google authentication services. The 'spsht _ ky' variable stores the spreadsheet's unique key. In all spreadsheets, there must be a worksheet. The default ID of the worksheet for Google Spreadsheet application is 'od6'. A spreadsheet client object 'sp _ cnt' by invoking `gdata.spreadsheet.service.SpreadsheetsService()` method `sp_cnt.email`, `sp_cnt.password`, and the `sp_cnt.source` will store the e-mail ID, app password, and spreadsheet name, respectively. The `sp _ cnt.ProgrammaticLogin()` method is the login method from the client side. The method authenticates the user and sets the GData authentication token. A temporary authentication token is retrieved while logging in. These tokens should be used with further requests to GData service. The GData client object is responsible for

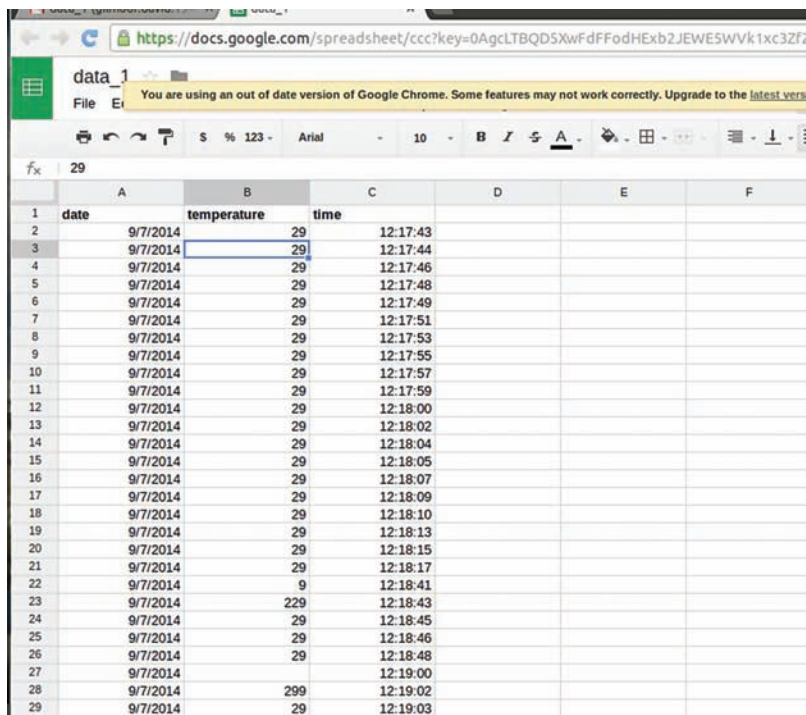


FIGURE 8.15 Google Spreadsheet real-time data update.

storing the authentication token. Then a dictionary is created that consists of three attributes—time, date, and temperature—obtained from the sensor. An instance is inserted to the spreadsheet by calling *sp_cnt.InsertRow(dic, spsht_key, ws_id)* method of spreadsheet service. Figure 8.15 shows the uploaded data to the Google Spreadsheet service.

8.12 Summary

In this chapter, first, we have explained the temperature sensor interfacing and calibration methodology. Second, we have learned how to interact with Zigbee technology and have discussed how to configure an XBee wireless module. Third, we have explained how to install Pidora OS in Raspberry Pi. Fourth, a few test runs of the Python serial interface in Pidora OS to receive the data from the Zigbee protocol have been discussed. Finally, we have discussed the interaction with Google cloud service using the Python application to store the sensor data in real time.

8.13 Concepts Covered in This Chapter

- How to interface a temperature sensor with Arduino
- How to configure an XBee module
- How to install Pidora OS in Raspberry Pi
- How to install pySerial in Raspberry Pi
- How to install gdata API in Raspberry Pi
- How to run a Python script in Raspberry Pi
- How to interact with a Google Spreadsheet using Python

9

Home Automation System

9.1 Introduction

Home automation provides the power to control several devices at home from a mobile device anytime, anywhere. It not only refers to outlying programmable units, such as thermostats and sprinkler systems, but also more accurately describes homes in which almost every appliance such as the electrical outlets and heating, lighting, and cooling systems is attached to a network that is remotely controllable. From a home security perspective, this includes smoke detectors, windows, alarm system, automatic doors, locks, surveillance cameras, and any other sensors that are connected to it.

9.2 Home Automation System Architecture

Several kinds of home automation system such as those manufactured by Leviton and Savant are available in the market nowadays. Most systems provide the user with security as well as the ability to control home appliances from remote locations. This will be specifically useful when the users are away from home. Technically, this system consists of a few stages. In our project, we have tried to develop an elementary level of home automation system by implementing some of the major stages of the system shown in Figure 9.1.

9.3 Essential Components

The essential components of the automated system can be categorized as follows:

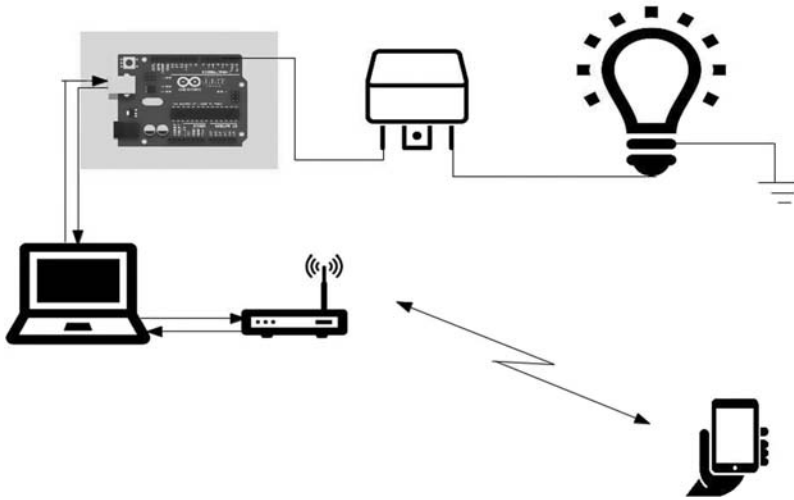


FIGURE 9.1

The home automation system architecture.

1. *Server*: Server is the central part of the system that receives the status of any appliances as well as sends the command to the device to be controlled. A server can be designed using a minicomputer (such as Raspberry Pi) or may be with a portable small-sized laptop. In this example, we typically choose a laptop and execute the Apache Tomcat v. 7.0.19 to set up the Tomcat server. The free server software is available for downloading from <https://tomcat.apache.org/download-70.cgi>. Now, install the software into the server and write the Java Server Page (JSP) file to interact with the device to the remote users. The JSP is a server-side program that runs in Apache Tomcat server based on Java virtual machine.
2. *Processing interface script*: Here, the server interacts with the remote user and according to the response it will generate an XML script file that stores the status of the user sent by the JSP program. The Processing script (<https://processing.org/>) is a special type of programming environment through which we can interface the Arduino hardware with any other device or platform. Primarily, it is used for graphical visualization but additionally it can be utilized as an interface between the hardware and other software platforms.
3. *Arduino controller*: The Arduino is an obvious component of this project. Here, the Arduino UNO (one in Italian) has been interfaced with Processing. The Processing interface has sent 1 or 0 digital data toward the Arduino by parsing the XML file.

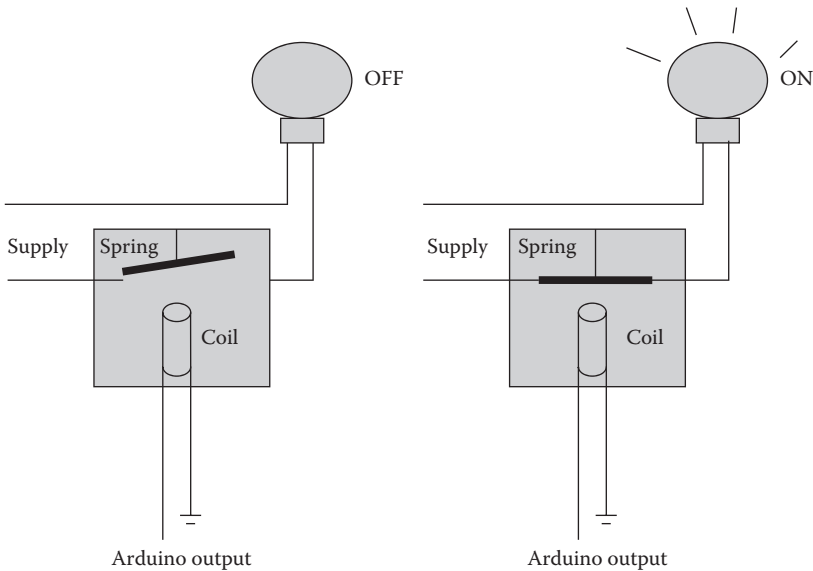


FIGURE 9.2
ON/OFF state of the device.

4. *Relay unit:* The signal fed to the Arduino controller is then processed by ATmega; an ON or OFF signal is then generated and is fed to a relay unit as shown in Figure 9.2. The main task of the relay unit is to control the automatic switch. As Arduino sends an ON signal, the relay will connect the circuit, and therefore, the home appliances connected to the relay will be ON. The OFF signal from Arduino enables it to shut down.

The basic principle of the relay unit is as follows. A relay is a device that senses the electrical signal (current or voltage) and transmits it to the ON/OFF device or switch to either open or close a circuit as per requirement. Large electrical circuits are protected by electromagnetic-type relays; numerical programmable relays are used in modern protection technology. Small and sensitive electronic circuits are protected by electronic relays. Figure 9.3 shows the magnetic relay principle used in this project.

9.4 Connection Detail

For this project, instead of using a readymade relay driver, we actually made a relay driver circuit and then drove the relay by using Arduino. The circuit diagram of the relay driver used in this project is shown in Figure 9.3.

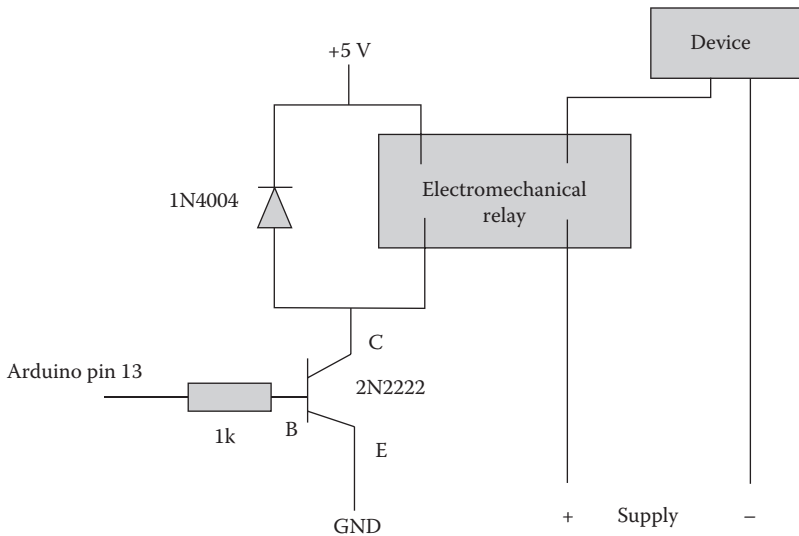


FIGURE 9.3
Relay driver module.

Here, the Arduino output pin is connected to the base of the 2N2222 transistor with a 1 kΩ resistor in series. The collector is connected to the control coil of the relay and in parallel with the 1N4004 diode as a protector circuit. Other terminals of the relay control coil and diode are parallel connected to a +5 V power supply. Emitter of the bipolar junction transistor is connected with ground.

9.5 Setting Up the Web Server

To set up the web server, install the Tomcat server in the laptop, and then go to `apache tomcat\ webapp\ ROOT` folder and create the `.jsp` file. In this case, we have created an `index.jsp` and `process.jsp` file, where the `index.jsp` generates the home page for device control. It has an ON/OFF radio button that takes the response of the user in a mutually exclusive form. The page sends a request object to the `process.jsp` where the ON/OFF request has been processed. The form tag in `index.jsp` is useful to generate the html form through that we are sending the parameter `<form method=get and action = http://192.168.0.7:8080/process.jsp>` that signifies HTTP request method type and the URL of the target webpage. Within the body of the form, we have added the radio

button for taking the input status (ON/OFF). The process.jsp takes the response and accordingly generates an A.xml. In process.jsp, the path object is generated using request.getPath() method. GetPath() is a method of request object of HttpRequest class. Next, we have to generate the object of the PrintWriter class. In the PrintWriter class, the FileOutputStream object is passed with the name A.xml. In that XML file, the status is written using pw.println() method. As the writing is completed, the pw.close() method has been called to close the PrintWriter object. After successfully writing the XML file, the status of the device is displayed on the web page. The server-side script is as follows.

index.jsp

```
<html>

<head><center><h1>Device control system</h1><center></head>

<hr/>

<body bgcolor=cyan>

<!--<form method = "get" action ="http://117.227.119.0:8080/
process.jsp">-->

    <form method = "get" action ="http://192.168.0.7:8080/process.jsp" >

<center>

<input type="radio" name="status" value="on"/> On<br/>

<input type="radio" name="status" value="off"/> Off<br/>

<input type="submit" value="Submit">

<center>
</form>

</body>
</html>
```

Processing.jsp

```
<%@ page contentType="text/html; charset=iso-8859-1"
    language="java" import="java.io.*" %>

<html>

<center><head><h1>
```

Device Control System

```
</h1></head></center>
```

```
<hr/>
```

```
<body bgcolor ="cyan">
```

```
<%
```

```
try {
```

```
    String path = request.getRealPath("/");
```

```
    //out.println(path);
```

```
    PrintWriter pw = new PrintWriter(new
        FileOutputStream(path+"A.xml"));
```

```
String s= request.getParameter("status");
```

```
pw.println(s);
```

```
//out.println(s);
```

```
%>
```

```
<center> <h3> Current Device Status : <% out.println(s); %>
    </h3> </center>
```

```
<hr/>
```

```
<a href = "http://192.168.0.7:8080/index3.jsp">back to home</a>
```

```
</body>
```

```
</html>
```

```
<%
```

```
pw.close();
```

```
}
```

```
catch (Exception e) {
```

```
// Handle exceptions
```

```
out.println("Exception is "+e.getMessage());
```

```
}
```

```
%>
```

9.6 Interaction with Server by Processing

Although Processing (Figure 9.4) is a powerful tool for visualization, it can also be used for a different purposes. Here, our goal is to develop an interface between the server-side scripting and the hardware. The Processing script has to run on a computer local to the server. In Processing, we have to declare the XML file along with a feed URL. The Processing application by default gives an applet frame through which it displays the graphical visualization. In our project, we did not directly use the visualization frame, but we created a logo while running the application. This was done by the `img = loadImage("andru.png")` method. The `font = loadFont("Times.vlw")` loads the specific font to be displayed. The `port = new Serial("COM31", 9600)` creates a new serial port object to access Arduino (COM31 for windows, /tty/USB1 for Linux). The baud rate is 9600. The public void `draw()` method declares the background, text, and image by `background(0,0,0)`, `text("powered by..",10,40)`, `image(img,0,0)`, respectively. Afterward, the function `fetchandwrite()` is implemented to parse the XML file and send the flag data to the Arduino.

An object to represent the URL to get the XML file is done by a try block `URL url = new URL(feed1)`. The `URLConnection conn = url.openConnection()` method is used to create a connection via the URL object. After creating the connection, the connection is made available by calling `conn.connect()`. Then, a buffer reader object that reads and sends data from the XML file is created using the statement `BufferedReader in = new BufferedReader(new InputStreamReader(conn.`

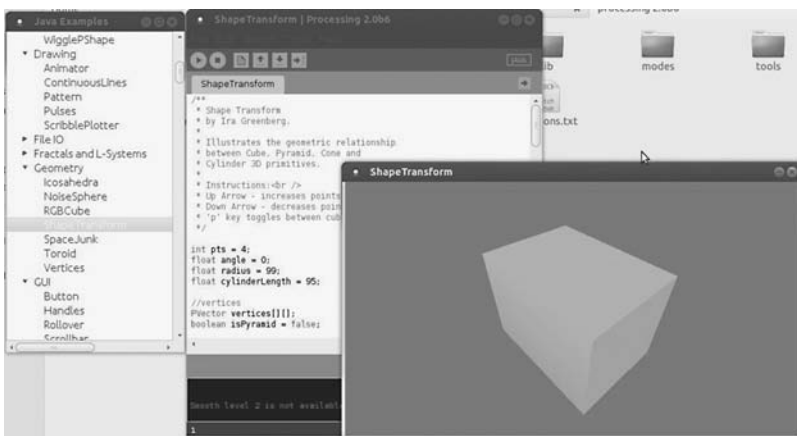


FIGURE 9.4

Processing visualization tool.

`getInputStream())`). In the buffer reader object, we have to pass the input stream corresponding to the connection object. Subsequently, within a `while` loop, we have to take the token of the XML file to parse the syntax of the XML file. It can be performed by creating `StringTokenizer` object `st = new StringTokenizer(data, "\"<>, .()[] \")`, where the data are the token character set. `st` object is treated as a character chunk. Now, the tokens are to be checked indexwise, but before that they have to be created in lowercase. This can be done using `chunk = st.nextToken().toLowerCase()`. The condition `if (chunk.indexOf("on") >= 0)` will check the index of `on` token. If it is zero, it means that it is the current token and a `T` message is sent to the Arduino via serial; otherwise, an `F` message is sent.

Processing code for server interfacing

```
import processing.serial.*;

String feed1 = "http://192.168.0.7:8080/A.xml";

char stat;

Serial port;

PFont font;

PImage img;

void setup(){
    size(700,300);
    frameRate(10);
    fill(255);
    img = loadImage("andru.png");
    font = loadFont("Times.vlw");

    port = new Serial(this,"COM31" , 9600); // connect to Arduino
}

void draw(){
    background(0,0,0);

    text("powered by..",10,40);

    image(img,0,0);
```

```

fetchandwrite();
//delay(2000);
}

void fetchandwrite(){

    // we use these strings to parse the feed
    String data;
    String chunk;

    try{

        URL url = new URL(feed1); // An object to represent the
                                   URL

        // prepare a connection

        URLConnection conn = url.openConnection();

        conn.connect(); // now connect to the Website

        BufferedReader in = new BufferedReader(new
            InputStreamReader(conn.getInputStream()));

        while ((data = in.readLine()) != null) {

            StringTokenizer st = new StringTokenizer
                (data,"\"<>,.()[] "); // break it down

            while (st.hasMoreTokens()) {

                //each chunk of data is made lowercase

                chunk= st.nextToken().toLowerCase() ;
                if (chunk.indexOf("on") >= 0 ){
                    stat = 'T';
                    println(stat);
                    port.write(stat);
                    text("Device is : ON",100,220);
                }
                if (chunk.indexOf("off") >= 0 ){
                    stat = 'F';
                    println(stat);
                    port.write(stat);
                    text("Device is : OFF ",100,220);
                }
            }
        }
    }
}

```

```

    }
  }
  }catch(Exception e){
    println(e.getMessage());
  }
}

```

Arduino code to control device

```

int relaypin=13;
byte data_frm_serial;

void setup(){
  Serial.begin(9600);
  pinMode(relaypin,OUTPUT);
}

void loop(){

  data_frm_serial = Serial.read();
  // Serial.println(data_frm_serial);

  if(data_frm_serial == 'T')

    digitalWrite(relaypin,HIGH);

  else if(data_frm_serial == 'F')

    digitalWrite(relaypin,LOW);

}

```

This code is used to fetch the status of the device produced by server and does the necessary task accordingly. Here, a byte variable `data _ frm _ serial` is used to store the status of the device given by the user from remote side. The variable `relaypin` is used to send an ON/OFF signal to the relay via pin 13. In the `void loop()` function, the condition has been checked whether `data _ frm _ serial` is `'T'` or `'F'` as supplied by the Processing script by fetching the A.xml file. `DigitalWrite()` function has been called accordingly with HIGH or LOW value based on the `'T'` and `'F'` value provided by the XML file. Figure 9.5 shows the selection window of two states, and Figure 9.6 shows the device status. Figures 9.7 and 9.8 show the OFF and ON state of the lamp, respectively.



FIGURE 9.5
Device state selection.



FIGURE 9.6
Device status.



FIGURE 9.7
Device in OFF state.

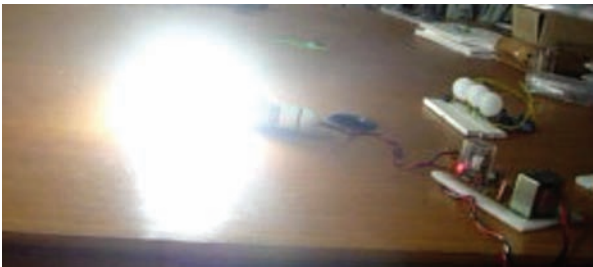


FIGURE 9.8
Device in ON state.

9.7 Summary

In this chapter, we have discussed how hardware gets interfaced with the Processing interface to interact with the server. The basic principle of a home automation system has been explained with the entire open-source components. Initially, the implementation of the hardware component and then the interaction with open-source Processing tool have been discussed. Finally, the control of the lamp by using a remote computer/smart phone has also been explained.

9.8 Concepts Covered in This Chapter

- Electromechanical relay interfacing with Arduino
- Interaction with Processing and Arduino
- Creation of web server and control of Arduino from web page hosted in web server
- Writing a file using Processing

10

Three-Servo Ant Robot

10.1 Introduction

In this chapter, we will discuss how the basic four-legged structure of an ant robot is built. The ant robot is a basic robot that can be designed with minimal electronic, mechanical, and electrical resources. Our project basically uses three servomotors: two standard-sized servomotors for leg movement and one submicro-sized servomotor for neck movement. The entire control of the ant robot is programmed using an Arduino UNO development board. The primary objective of developing this robot unit is to learn the basics of robotics as well as to become familiar with the basics of the Arduino development board and to figure out the functionality of several sensors and actuators that connect, besides performing collaborative tasks.

10.2 Tools and Parts Required

A very basic ant robot can be made using a minimum of components. The system architecture is shown in Figure 10.1. Out of the three servos, two are used to create the leg movements of the robots and one is mounted on the top of the robot that carries the sensor module. The function of the third motor is to perform ant-like neck movement and sense the distance of the nearby object that falls within a visual range of 90° by ultrasonic range finder.

10.2.1 Ultrasonic Sensor

The ultrasonic sensor is an important component of these ant robots and is shown in Figure 10.2. It is programmed as an obstacle avoider robot; therefore, the distance to the obstacle has to be measured. A very efficient way to do that is to interface the ultrasonic sensor. The sensor acts as the eye of the ant robot. The basic principle of an ultrasonic sensor depends upon the speed of sound. Generally, it consists of a transmitter and a receiver end. The transmitter transmits the ultrasonic sound that reflects on the object and

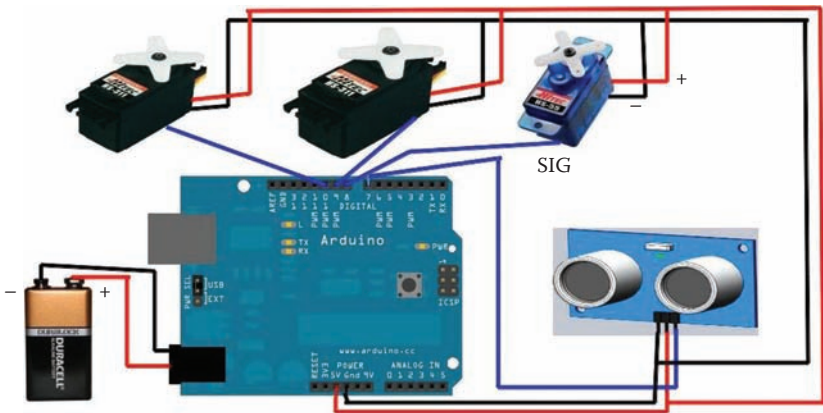


FIGURE 10.1
System architecture.

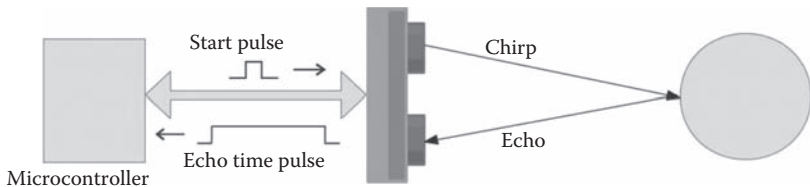


FIGURE 10.2
The ultrasonic sensor.

produces an echo. The receiver collects the echo and converts it into a digital pulse width modulation (PWM) pulse and sends it to the microcontroller. The received pulse in the microcontroller can be used to compute the distance between the object and the robot by the following formula:

$$\text{Distance} = \frac{\text{Duration (ms)}/74}{2}$$

(10.1)

where the duration in microseconds has been supplied by the sensor and the transmitter actually sends the high and low pulse with an interval of 2 μs. The same pin is used to receive the echo pulse captured by the receiver. The Arduino function `pulseIn(pinno, value)` gives the millisecond duration.

10.2.2 Servomotors

Figure 10.3 demonstrates the servomotor, which is a DC, AC, or brushless DC motor. A position-sensing device such as a digital decoder is combined



FIGURE 10.3
Servomotor.

with it. A three-wire DC servomotor is incorporated with a DC motor, a gear-box, a potentiometer for position feedback controller, limited stops beyond which the shaft cannot turn, and an integrated circuit for position control. The three wires are +5 V power supply, ground, and control signal. As long as the coded signal is applied on the input line, the servo maintains and holds the angular position of the shaft as illustrated in Figure 10.4. A change in the coded signal changes the angular position of the shaft.

Control circuits and a potentiometer associated with the servomotor are connected to the output shaft of the motor. The control circuitry is responsible for monitoring the current angle of the servomotor which is directed by the potentiometer. If the shaft is at a proper angle, then the servo shuts off. If the circuit finds that the servo is not at a correct angle, then it changes its direction and adjusts the angle accordingly.

The output shaft of the servo can move around $0^\circ \pm 180^\circ$. Usually, it is in a range of $0^\circ \pm 210^\circ$, which varies according to the manufacturer. A half-rotation normal servo is restricted to rotate within the range of a certain limit. This could be done by applying a mechanical stop on the main output gear. A proportional control has been developed to turn off the servo. At a large angle, it will move in maximum speed, while for a low angle, it will move in a slower speed.

10.2.3 Leg Design

The legs are designed with one degree of freedom, as the ant robot in this project has only two legs. These can be achieved by attaching the center of

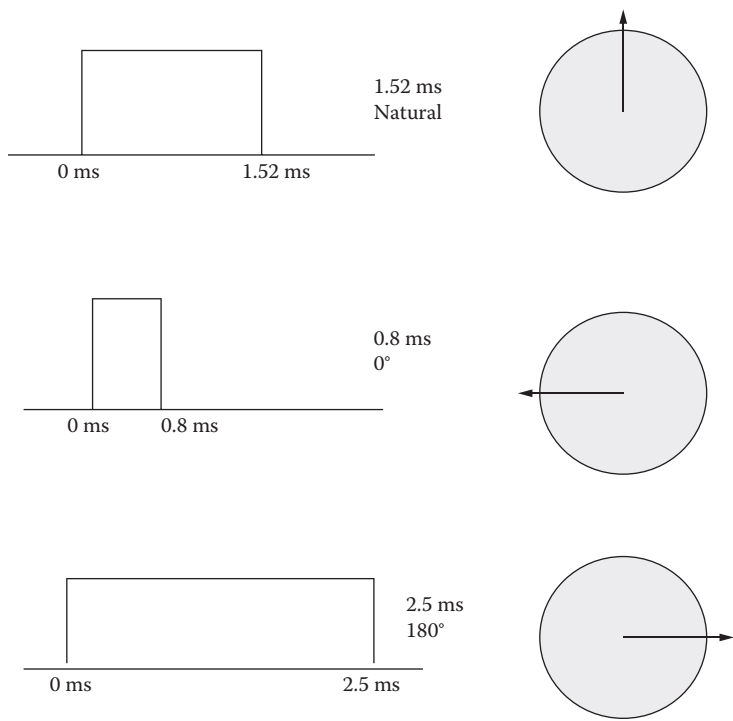


FIGURE 10.4
Servomotor working principle.

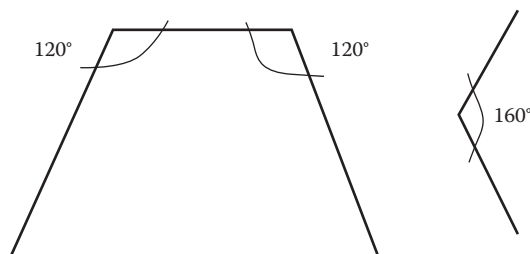


FIGURE 10.5
Leg design.

the leg to the pinion of the servo. Hot glue or tapes are highly suitable to attach the leg to the servo pinion. Basically, the legs are designed by bending copper or aluminum wire. In this project, aluminum wire is used. We have used 28 cm wire for the front and 25 cm wire for the rear leg. While designing the front leg, it has to be ensured that the leg can bend almost 180° backward to get a better grip as shown in Figure 10.5. On the bottom of the leg,

heat shrink tube or rubber padding is applied so that it gives a better grip to the robot and can move over any rough surface.

Next is the attachment of the leg to the servo pinion, which is an important task. In general, a servo comes with several different plastic attachments. The legs can be directly attached by pulling the wire through the servo holes. These can be secured by tightening with some pieces of wires. After that, some hot glue is placed on the joint to permanently tighten the leg to the servo pinion.

Details on the assembly of the leg with the servo pinion are shown in Figures 10.6 and 10.7.

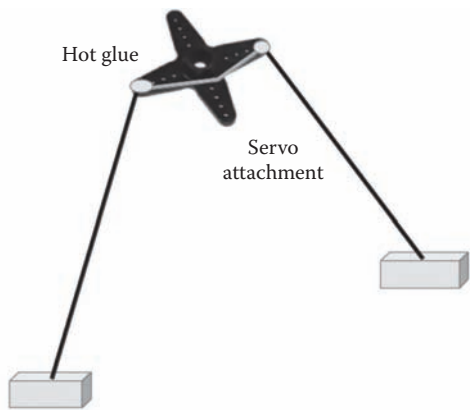


FIGURE 10.6
Front leg assembly.

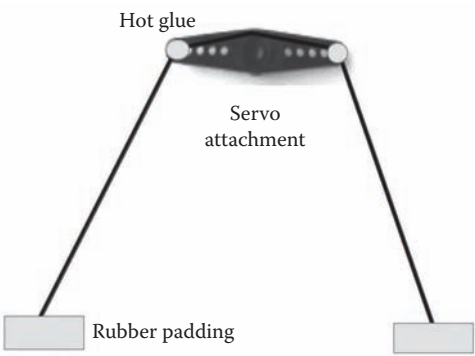


FIGURE 10.7
Rear leg assembly.

10.2.4 Mounting Ultrasonic Sensor

After assembling the leg, it is necessary to mount the ultrasonic sensor on the ant robot so that it can detect the obstacle. Here, the sensor is mounted on the top of the 9 g servomotor so that it can rotate its head from -60° to $+60^\circ$. In order to perform this, zip ties or black tape that should be fitted tightly to the servo attachment are used. It is necessary to use a proper servo extension wire to attach to the ultrasonic sensor.

10.3 Programming the Leg Movement

To get proper leg movement, the microcontroller that sends the proper PWM signal to the servomotors has to be programmed. The movement strategy of this is designed as shown in Figure 10.8.

The servo has to be programmed initially to move the ant robot in the forward direction; therefore, the front and rear servomotor should be moved from -40° to $+40^\circ$. This produces a zigzag motion of the ant robot toward the front. Parallely, the reading of the ultrasonic sensor is taken to find out the distance of any object placed in front of the ant robot. As any object has been encountered in front of the robot within a specific threshold distance (here we have taken it as 20 in.), the robot stops its motion and moves

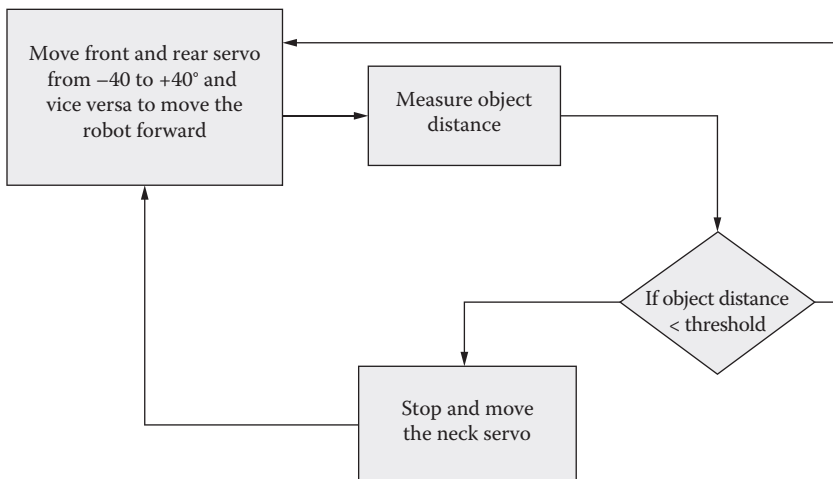


FIGURE 10.8

Instruction sequence for the forward movement.

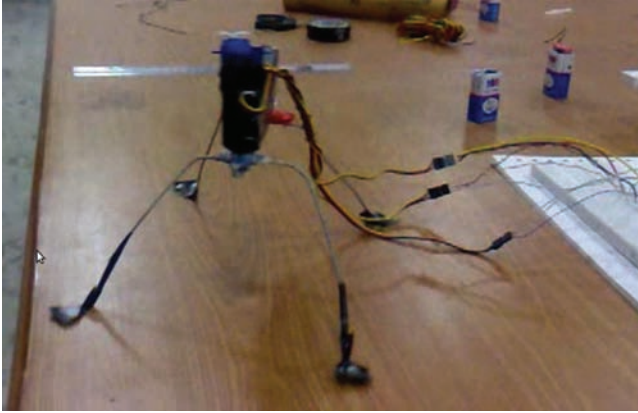


FIGURE 10.9
Partial assembly.

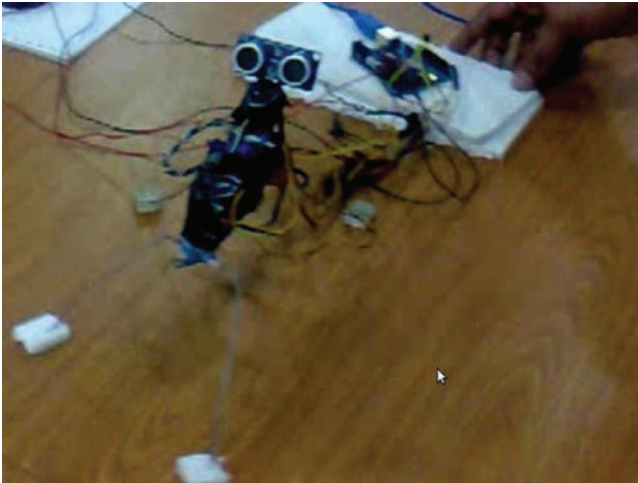


FIGURE 10.10
Final robot.

the neck left right to search the entire obstacle. As the obstacle is removed from the front of the robot, the robot again performs its normal behavior. Figures 10.9 and 10.10 show the partial assembly and the implemented final robot, respectively. For programming obstacle avoidance with neck movement, the following code is used.


```

#include <servo.h>

Servo myservo,myservo2,myservo3;          // create servo object
                                           // to control a servo
int flag=0;                               // flag value for neck movement
int pos = 0;                               // variable to store 1st servo position
int po=30;                                 // variable to store 2nd servo position
int pingPin=8;                             // ultrasonic ping pin
int po1=0;                                 // variable to store 3rd servo position
//int tempPin = 0;
//float temp;
long duration,inches;                      // time duration and corresponding
                                           // distances

void setup()
{
myservo.attach(5);                         // 1st servo attached on pin 5 to the
                                           // servo object
myservo2.attach(7);                       // 2nd servo attached on pin 7 to the
                                           // servo object
myservo3.attach(9);                       // 3rd servo attached 3rd servo on pin
                                           // 9 to the servo object
Serial.begin(9600);                       // read serial port at 9600 baud rate
}

void loop()
{
  //temp = analogRead(tempPin);
  // Serial.print(temp,DEC);
  flag=0;
  for(pos = -10,po=40,inches=fn(); pos <=40,po>=-10; pos += 1, po--)
  // goes from -40 degrees to +40 degrees & parallel check the
  // obstacle distance
  {
    // in steps of 1 degree
    myservo.write(pos);                   // tell servo to go to position in
                                           // variable 'pos'

    myservo2.write(po);
    //Serial.println(pos,DEC);
    //Serial.println(po,DEC);
    delay(10);
    if(inches<=20){
      flag=1;
      // check the obstacle distance is <= 20 inches or not if so
      // make flag =1 and terminate the loop
      break ;
    }
  }
  delay(500);
  for(pos = 40,po=-10,inches=fn(); pos>=-10,po<=40; pos-=1, po++)

```

```

// goes from +40 degrees to -40 degrees & parallel check the
// obstacle distance
{
    myservo.write(pos);          // tell servo to go to position
                                // in variable 'pos'

    myservo2.write(po);
    delay(10);
    if(inches<=20){
        flag=1;                // check the obstacle distance is
                                // <= 20 inches or not if so make flag =1 and terminate
                                // the loop
        break ;
    }
}
if(flag==1) {
    fn1(); // make neck movement
}
delay(500); // waits 500ms for the servo to reach the
            // position
}
void fn1(){ // code to move the neck
    myservo3.write(50);
    delay(1000);
    myservo3.write(120);
    delay(1000);
    myservo3.write(90);
}
int fn(){
    // ultrasonic sensor data acquisition
    pinMode(pingPin, OUTPUT);
    digitalWrite(pingPin, LOW);
    delayMicroseconds(2);
    digitalWrite(pingPin, HIGH);
    delayMicroseconds(5);
    digitalWrite(pingPin, LOW);
    pinMode(pingPin, INPUT);
    duration = pulseIn(pingPin, HIGH);
    inches = microsecondsToInches(duration);
    /inches=inches;
    Serial.println(inches);
    return inches;
}
long microsecondsToInches(long microseconds) //ms to inch
                                              //conversion
{
    return microseconds / 74 / 2;
}

```

10.4 Summary

In this chapter, we have discussed how a basic legged robot can be made. Although an elementary form of robot is discussed, it is still a great experience for amateur students and robot hobbyists. The elementary leg movement, distance sensing, and processing have been discussed here, which gives further direction to make even more sophisticated types of legged robot.

10.5 Concepts Covered in This Chapter

- Basic concepts of servomotor driving via Arduino
- Working principle of ultrasonic motion sensor
- An elementary robot design
- Implementation of basic artificial intelligence

11

Three-Servo Hexabot

11.1 Introduction

A hexabot robot is the most standard form of robotic architecture in legged robot class. This chapter explains how to build a hexabot robot by using three servomotors. The most optimized architecture can run six legs by using only three servomotors. Most hexabots use six servomotors to maintain static balance, but in this case, the hexabot is designed in such a way that for each move at least three legs touch the ground so that it maintains its balance like a tripod. Here, the robots are made to move by alternatively moving their legs side by side in a cyclic fashion (Figure 11.1). The middle leg, on the other hand, moves up and down, which balances the robot. When the front and the rear legs that are in the ground sweep back, the robot moves forward, and the same procedure is repeated by the other pair of legs alternately. The middle leg, on the other hand, provides balance during the movement.

11.2 System Architecture

The structure of the system is basically unique in nature. It is a system where six legs are controlled by three servomotors, which is a minimum requirement for a six-legged robot. As shown in Figure 11.2, it consists of three standard-sized servos (preferably HS-311 type); the servos should be connected to the Arduino pins 9, 8, and 7. The hexabot here is a 2-DOF robot. In most of the cases, the legged robot has three degrees of freedom (DOF). Regarding an amateur robot, two DOF is the best choice. The more the DOF, the more agile the robot becomes. In addition, more DOF require a higher number of joints and a higher number of motors to move each joint.

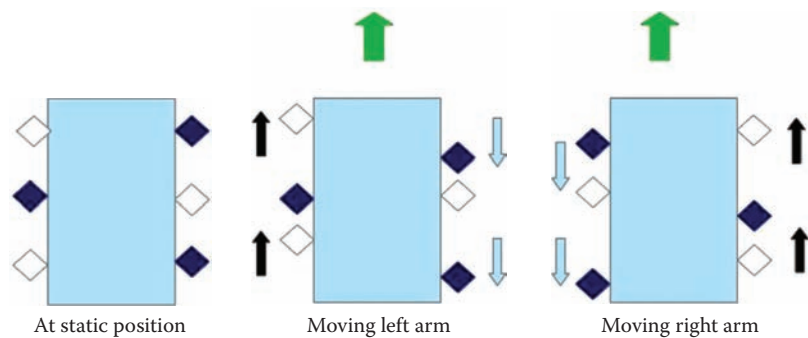


FIGURE 11.1
Movement logic of hexabot.

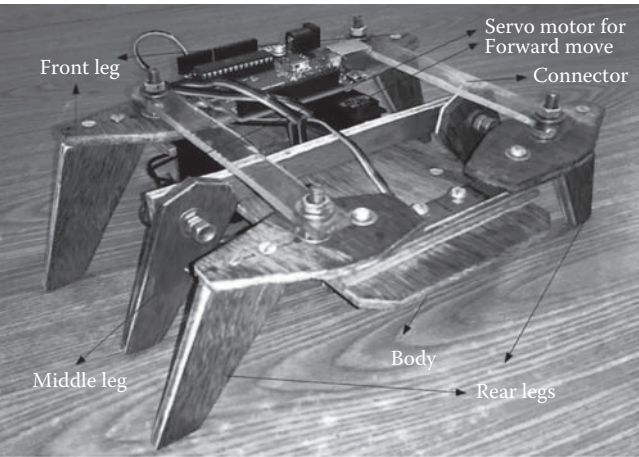


FIGURE 11.2
System components.

11.3 Parts and Their Assembly

Several parts have been assembled properly. The frame structure of these robots is pretty challenging. We prefer to design such a frame by using PVC material or wood. In this case, wood is chosen. The leg design of the robot is shown in Figure 11.3. The front and the rear legs are basically made of two panels. The middle leg is just a single piece. The bottoms of all legs have been cut on a 30° angle. The angle is made by a protractor. Care should be taken to ensure that the angles are accurate enough so that the legs work perfectly. Metal screws with hex nuts are the most useful components to attach all

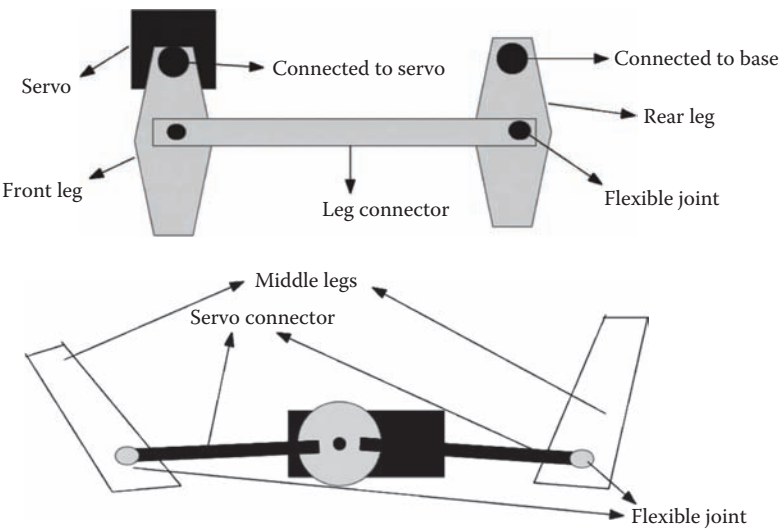


FIGURE 11.3
Leg assembly.

the joints. M3 steel hex nuts and M3 socket head screw are preferred. One can also use M3 nylon nuts and screws as well.

The next step is to cut the base pieces (Figure 11.4); the standard size of the base is 3.5 × 7 in. Care must be taken while drilling the base piece. A portable electric drill can be used to do that. With the base piece, we should add a servo holder (Figure 11.5) to hold the servomotor that controls the center leg. The groove in the servo holder should be cut in such a way

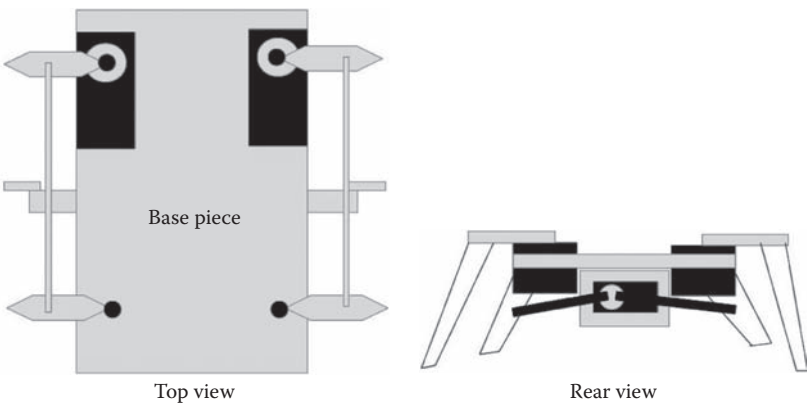


FIGURE 11.4
Detailed structure of hexabot.

**FIGURE 11.5**

Front/rear leg design.

that it easily fits the servo that is being used. In this case, the groove is created for HS-311-type servomotor. The detailed structure of the hexabot is shown in Figure 11.4.

Front legs are attached to the base by attaching them with the servomotor that has been mounted on the base (Figure 11.3). The rear legs should be directly attached to the base by using a nut-bolt joint. The joints must be flexible enough so that they can move freely. Rubber washers can be used to make the joints more flexible. To drive the front and rear legs simultaneously, we must attach a connector between the front and rear legs. A connector may be designed by wood or glass fiber material. The middle leg, on the other hand, must be connected to the servo that has been attached under the base piece; here, two servo connectors must be used to attach to the left and right legs (Figure 11.6) so that they can move in the opposite direction. The joints of the servo connector should also be flexible enough so that they can move freely. Servo holder installation is a tricky task (Figure 11.7). It must be installed in a way such that the shaft of the servo lies exactly at the center so that the distance between the servo shaft and two middle legs becomes equal.

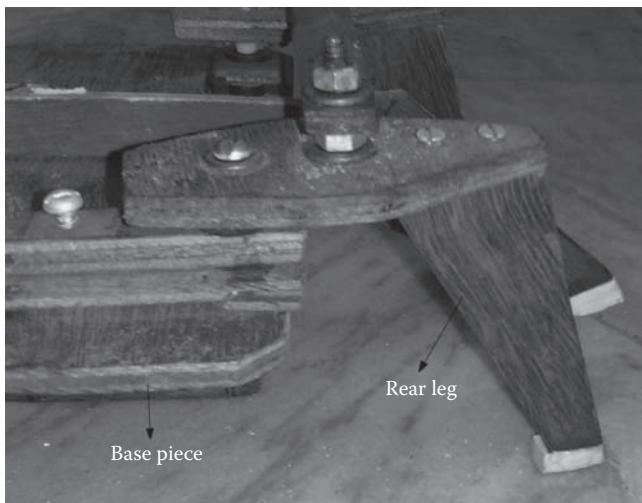


FIGURE 11.6
Base piece with rear leg.



FIGURE 11.7
Servo holder for middle leg movement.

11.4 Programming Basic Moves

Arduino code to move the hexabot front is as follows.

```
#include <Servo.h>
```

```
Servo legC;
```



```
Servo legR;
Servo legL;
int delayL = 255;
int delayS = 353;
boolean dbug = false;
void setup()
{
    legC.attach(9); // attaching servos
    legR.attach(8);
    legL.attach(7);
}
void loop()
{
    if(dbug) {
        leg_net(); // static position
    }
    else {

        upLL();
        delay(delayS);
        legLL_rev();

        legRR_fwd();
        delay(delayL);
        upRR();
        delay(delayS);
        legLL_fwd();
        legRR_rev();
    }
}
```

```
        delay(delayL);
    }
}

void upRR() {
    legC.write(160); // Right Center Leg Up
}

void upLL() {      // Left Center Leg up
    legC.write(20);
}

void legRR_fwd() {
    legR.write(110); // forward move right leg
}
void legRR_rev() {
    legR.write(75);  // reverse move right leg
}

void legLL_fwd() {
    legL.write(75);  // forward move left leg
}

void legLL_rev() {
    legL.write(110); // reverse move left leg
}

void leg_net() {
    legC.write(90);
    legR.write(90);
    legL.write(90);
}
```

Similar to the three-servo ant robot, in our hexabot, three servo objects are made and attached in Arduino PWM pins 9, 8, and 7, respectively. The `void loop()` function consists of an if condition, which checks whether `debug = true`. If it is true, then the program stops the movement of the robot by calling a function, where the position of the servo becomes 90° . To perform a forward move the robot should do its left leg up, after some amount of delay the robot must do a reverse movement by performing reverse for left leg and forward for right leg. Now after some delay for second move, the right leg should be up and with some delay perform left leg forward action and right leg reverse action.

11.5 Summary

The code given earlier shows the basic forward and reverse moves of the front and rear legs of the hexabot. The hexabot can even be modified as a sensor-controlled system by attaching an obstacle detector. In that case, we must check the obstacle distance and move the legs accordingly to do forward and reverse motion. The final hexabot in running condition is shown in Figure 11.8.



FIGURE 11.8
Hexabot on the ground.

11.6 Concepts Covered in This Chapter

- Building hexabot
- Dealing with mechanical design of legged robot
- Programming basic forward movement of a hexabot
- Understanding the movement of a basic six-legged robot.

12

Semi-Autonomous Quadcopter

12.1 Introduction

The quadcopter is the most popular vertical takeoff and landing (VTOL) multicopter structure in the world. The word quad specifies that this structure has four arms, with each arm connected to a single motor. One specific advantage of such a system is its aerial stability. It offers more stability compared to a normal heli structure. In a helicopter system, the direction and orientation are controlled by adjusting the pitch of the spinning rotor. The tail rotor of the helicopter provides stability against the yaw effect created by its spinning rotor. As discussed previously, the multirotor aircraft never uses pitch changing of the rotor at the time of spinning. As it completely depends upon the speed of a different rotor, its design complexity has been reduced.

12.2 Structural Design

The most popularly used quadcopter structures are the ^aX^o quad, ^a+^o quad, or ^aH^o quad. Apart from these, numerous designs of quadcopter are available. While it is highly recommended that light and strong materials be used in the design of the quadcopter frame, carbon fiber or strong glass fiber materials are used primarily. Alternatively, wood, plastic, or light stainless steel can also be used. Materials that cause more vibration should not be used as they will make the copter unstable. To design a beginner's-level copter, a standard frame kit is recommended. The frame-designing steps are shown in Figure 12.1 and are as follows.

Step 1: Prepare four equally cut pieces for making four arms.

Step 2: Make the center platform by using a square-type material, where the flight controller and other accessories can easily be accommodated.

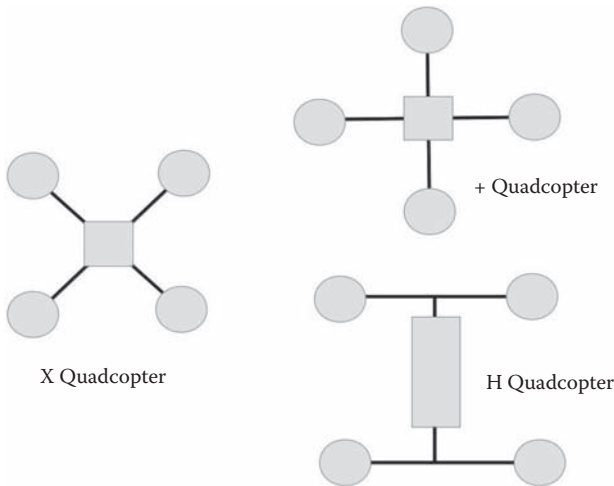


FIGURE 12.1
Various types of quadrotor design.

Step 3: Note that while assembling the arm with square platform, the distance between one end of the arm and the platform should be constant for all four arms.

Step 4: Check the center of gravity (CG) after assembling all the arms by applying a rope method. In this case, a rope is attached to the middle of the platform and the frame is hanged; the orientation of the frame is now observed. If it is horizontal to the ground, then the CG is almost in the middle of the frame. Otherwise, a few modifications of the frame should be done so that the CG acts within the middle of the frame (Figure 12.2).

Step 5: If the CG is good, then you can apply further modification in the frame. Make sure that the modification does not change the required CG too much.

In our example quadcopter, we have used a standard HJ 450 frame shown in Figure 12.3 to obtain high precision.

12.3 Component Description

After designing the frame, the components necessary to fly and control the aerial vehicle are to be added. The following are the required components:

- Flight controller: MultiWii CRIUS 2.5/K.K 5.5
- Motors: 4, 850 kV 2830 brushless motor unit

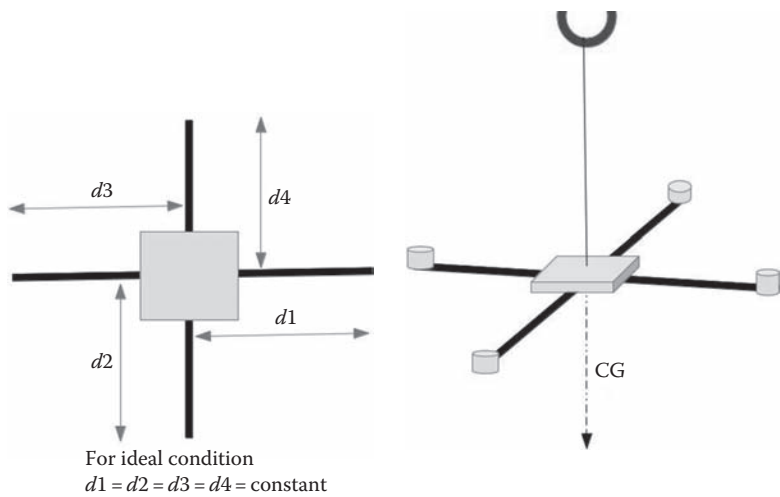


FIGURE 12.2
Quadrotor frame balancing.

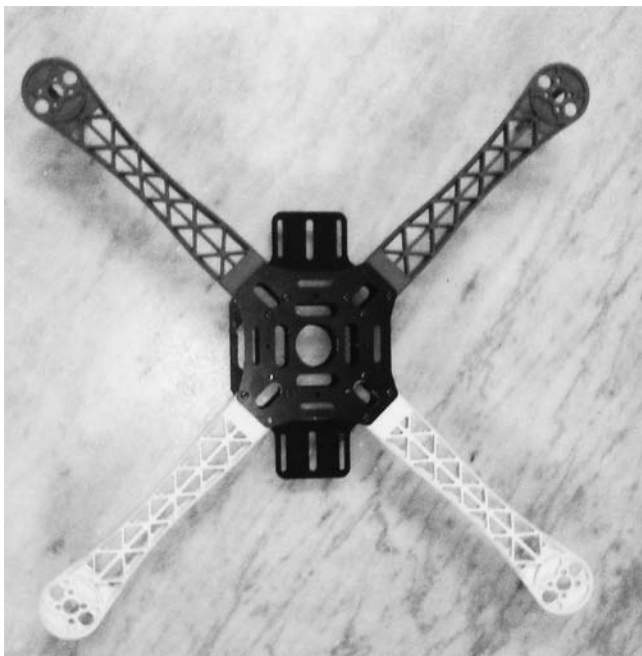


FIGURE 12.3
Quadrotor frame.

- Electronic speed controller: 20 A brushless electronic speed controller (ESC)
- Propeller: 10X4.5
- Radio controller: 2.4 GHz 6±9 channels

12.4 Flight Controller Unit

The most necessary component of a quadcopter unmanned aerial vehicle (UAV) system is a flight controller that is often known as an autopilot unit. It performs all necessary tasks regarding flight management. The main components of the flight controller are (1) a rate gyro that controls the roll, pitch, and yaw of the quadcopter along x , y , and z directions and provides stability to the quadcopter system when in space and (2) a magnetometer known as magnetic compass that is used to hold the orientation of the quadcopter. In addition to these, various external sensor modules are attached to obtain more precise flight.

As the focus of this chapter is to build a semi-autonomous quad, the main objective was to design a UAV that can fly under the control of a pilot with good attitude and stability. In this chapter, we will discuss two different flight controller units, namely, MultiWii Crius SE 2.5 and KK 5.5 controller.

12.4.1 MultiWii CRIUS SE2.5

MultiWii is an open-source community-driven flight controller hardware device and is shown in Figure 12.4. The hardware is physically based on the

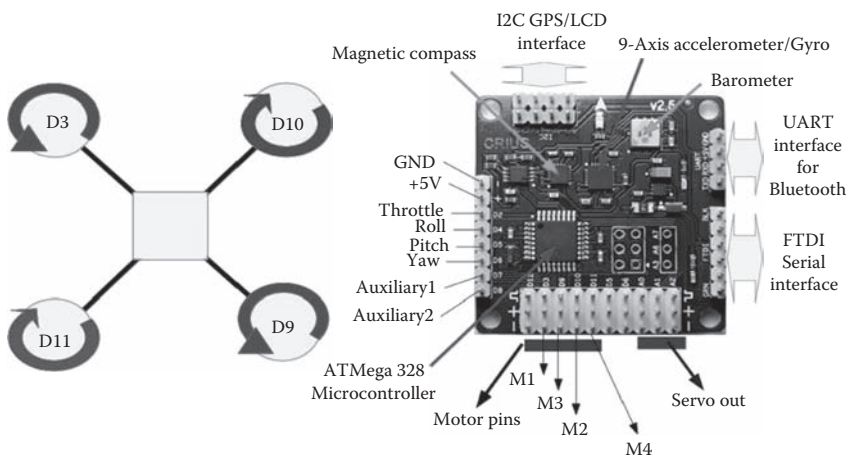


FIGURE 12.4
MultiWii 2.5 quadrotor configuration.

Arduino platform and several versions of MultiWii architecture are available in the market. The MultiWii Pro/AIO Pro Series flight controller consists of ATmega 2560-based microcontroller that is based on the Arduino MEGA hardware. Another lightweight version is MultiWii Crius SE and Lite that consist of ATmega 328P microcontroller that is based on Arduino Uno. The flight controller itself has onboard accelerometer/gyroscope, magnetic compass, and barometric pressure sensor. It supports the external GPS attachment with its inter-integrated circuit (I2C) communication header. In most cases, the GPS can be attached via an I2C navigation module. In turn, the I2C module also supports an interface LCD display to configure MultiWii onboard without connecting it to MultiWii graphical user interface (GUI). The external Bluetooth telemetry interface can also be attached with universal asynchronous receiver transmitter (UART) port to get real-time flight data. The Future Technology Devices International (FTDI) port present in the device is exclusively used to connect the MultiWii board with the PC/Lapto using USB interface. The D2+D12 pins are used to interface with motors and the radio receiver. Figure 12.4 shows the functionality of the different headers and their connection plan. In this project, we have made a quadcopter; therefore, four specific headers have been used to connect the ESC's PWM connector, namely, D3 for first (counterclockwise) motor, D10 for second (clockwise) motor, D9 for third (counterclockwise) motor, and D11 for fourth (clockwise) motor, respectively.

12.4.2 Flight Controller Comparison (Table 12.1)

In this section we emphasize several open source flight controller units and their features. Table 12.1 illustrates this in detail.

12.5 Assembling Parts

The assembling of the parts can be divided as follows:

1. *Frame Assembly:* Using a ready to fly (RTF) frame part or predefined frame, it is quite easy to assemble the booms using a standard screwdriver. Mostly, a scratch built frame consists of wood or lightweight metal. A frame made of wood is less durable, whereas frames made of lightweight metals like aluminum are more rigid and durable. The design methodology of the multirotor system suggests that the CG of the system should be exactly in the middle of the frame. This should be taken care of during designing, and additional changes should be made on the frame accordingly to maintain its stability. The power distribution unit should be added immediately after the frame is installed completely. Some frames have a built-in power distribution feature, but a standard power distribution board is recommended to avoid short circuit.

TABLE 12.1
Comparison of Several Flight Controller Units

FCU		MultiWii AIO pro	MultiWii 2.5 SE	MultiWii Lite	K.K 5.5
Processor		ATMega 2560±16 AU (8 bit AVR)	ATMega 328P (8 bit AVR)	ATMega 328P (8 bit AVR)	ATMega 168±20 AU (8 bit AVR)
Memory		256 kB (Flash)	32 kB (Flash)	32 kB (Flash)	16 kB (Flash)
		8 kB SRAM	2 kB SRAM	2 kB SRAM	1 kB SRAM
		4 kB EEPROM	1 kB EEPROM	1 kB EEPROM	512 B EEPROM
		16 MB on-board data flash			
Operating frequency		16 MHz	20 MHz	20 MHz	20 MHz
Throughput		16 MIPS at 16 MHz	20 MIPS at 20 MHz	20 MIPS at 20 MHz	20 MIPS at 20 MHz
Sensors	Gyroscope	MPU6050 6 axis	MPU6050 6-axis gyro	ITG3205 3 axis	Support for single axis (three different units at a time)
	Barometer	NA	BMP085	NA	NA
	GPS	Direct support for I2C GPS	GPS support using external I2C socket	NA	NA
	Magnetometer	HMC5883L 3 axis	HMC5883L 3 axis	NA	NA
	Altimeter	MS5611± 01BA01	NA	NA	NA
Inbuilt I2C support		Dedicated I2C-level conversion support	Dedicated I2C-level conversion support	Dedicated I2C-level conversion support	I2C-level conversion support not provided

2. *Assembling motors and ESC on frame:* After completely assembling the frame and power distribution module, the motors are mounted. Generally, brushless DC motors with proper dimension screws are preferred. It is important to set the motor wiring appropriately so that each spins in the correct direction. Multirotor UAV generally uses differential drives, where two diagonal motors are placed with the same polarity so that alternate motors spin in reverse direction. This can be easily done by alternating the connection of the ESC and the motor according to the requirement, which required a



FIGURE 12.5
Motor testing.

hit-and-trial methodology. While testing the motor direction, care should be taken to ensure that the propellers are not mounted over the motor shaft. The best approach to ensure the proper direction of the motor shaft is to affix tapes to the shaft and watch the direction of the movement of the tape. After installing the motor, Velcro bands or zip ties are used to tighten the ESC unit to the motor.

3. *Assembling autopilot:* The autopilot assembly on the frame is one of the challenging tasks. A vibration damper bed is used to install the autopilot, and then the autopilot is attached using screws, glue, Velcro, or zip tie. The installed autopilot should be immovable. During a flight, numerous vibrations may affect the orientation and the attitude of the multirotor in the sky as the gyroscope and accelerometer are highly sensitive to vibration, and therefore, gyroscope rate error gets increased. After successfully assembling the autopilot, other external sensor devices such as GPS, sonar, and 2.4 GHz receiver should be added. For a short time, some drone hobbyists prefer to protect their autopilot and sensor unit by attaching additional protectors such as foam block or plastic boxes. Care should be taken to ensure that these additional protectors do not affect the CG of the copter; therefore, an extremely lightweight component should be used as a protector.

12.6 Sensor and Speed Controller Calibration

Sensor calibration and speed controller are important criteria to be taken care of before the quadcopter is made to fly. Improper calibration of the sensor may lead to an unfriendly attitude of the copter, which may result in copter crash. In this section, we will discuss the gyro, accelerometer, and other sensor calibration. Here, the calibration of two different flight controllers has been discussed.

12.6.1 MultiWii Setup and Configuration

12.6.1.1 Configuring MultiWii Firmware

The MultiWii 2.5 firmware can be freely downloadable from <http://code.google.com/p/multiwii/>. Although the firmware is compatible with all units of MultiWii 2.5, it is reported that some of the boards are not compatible. Here, we exclusively attach the MultiWii firmware for MultiWii 2.5 board. The firmware can be uploaded using any version of the Arduino IDE. After unzipping the code, you can open MultiWii.ino file in Arduino IDE. From the tab, select config.h file and make necessary changes on it. While attaching the MultiWii 2.5 board to the computer via USB, make sure that you have selected a proper board type; in case of MultiWii 2.5, it is ATmega328p. The config.h file that is used here is basically a configuration file corresponding to the flight controller board. Based on several specific boards and the sensor specification, the parameter value of the file might change accordingly. The parameter here is in the form of macro in C++. In config.h, the most useful parameter settings for MultiWii 2.5 are discussed here.

12.6.1.1.1 Configurable Parameters

In the configurable parameter section, the following issues have to be changed:

- Choose the appropriate frame time type based on the frame you have designed. In our case, as we have chosen X-type frame, we have used `#define QUADX`.
- In the second option, we have to define min throttle range with `#define MINTHROTTLE 1150`, which signifies the minimum throttle PWM signal given to the ESC so that the ESC can move the motor of the copter.
- In the third option, we have to set maximum throttle PWM given to the ESC so that the motor of the copter will rotate at a maximum speed. Thus, we have to use definition `#define MAXTHROTTLE 1850`. For 20±25 AMP ESC, it is recommended to use 1850 PWM value. For higher range ESC, the throttle value can be raised up to 2000 PWM.

- Normally, the I2C speed setup is necessary when we use any I2C communication interface with MultiWii board. The default I2C speed value defined in MultiWii firmware is 400 kHz by taking `#define I2C_SPEED 400000L`. For some other types of board, the value might be 100 kHz. In our case, we have used the default.
- One of the major setups for the copter is to set up the minimum throttle command that ensures the minimum PWM signal given to arm the ESC. If the radio gives less than the mentioned value, the ESC of the copter will never arm. It is recommended that the min command should be set within the range 1000 ± 100 . If it exceeds the minimum throttle value, the motor will start spinning as the copter ARMED that might cause damage to the copter as well injury may occurs. The directive `#define MINCOMMAND 1000` will set that value to 1000 in our case.
- The next step is to choose the proper IMU board. Various IMU boards corresponding to the same type of firmware specification are available; in our case, we have selected `#define CRIUS_SE`.
- In MultiWii firmware, several options are available to attach the independent sensor module through I2C interfacing; in most cases, an I2C GPS is used with an I2C navigation board. Furthermore, some of the I2C magnetometer or barometric pressure sensors can also be interfaced with it via I2C communication. We have to pick the appropriate sensor module as per our requirement.
- Another important task is to configure the yaw direction of the copter. We can do this by mentioning the parameter `#define yaw_direction`. Values +1 and -1 will give clockwise and counterclockwise yaw effects, respectively. If the yaw is continuous while testing the copter, then make sure that proper yaw direction is chosen. In our case, it is chosen as -1.
- Arm/disarm stick assignment is one of the vital things to be noted while configuring the radio. By default, the arming/disarming has been done using throttle and yaw channel. This feature is controlled by the directive `#define ALLOW_ARM_DISARM_VIA_TX_YAW`. One can change the arm/disarm command from yaw to roll as well as by using the directive `#define ALLOW_ARM_DISARM_VIA_TX_ROLL`.

12.6.1.2 Sensor Calibration

In case of a MultiWii flight controller, we have to calibrate both the accelerometer/gyroscope and barometer. The ESC calibration can be done in a separate way. As MultiWii has its own configuration software, we can calibrate accelerometer/gyro directly from the application so that the calibration

process can easily be done. Download the MultiWii software from <https://code.google.com/p/mw-wingui/downloads/list>. Both Windows and Linux versions of the software are available. To calibrate the accelerometer, click on the MultiWii GUI Window ACC calibration button. Make sure that the copter is leveled while calibrating. In the next step, calibrate the magnetometer. While starting magnetometer calibration, one has 60 s to calibrate. The copter should be rotated 180° to x , y , and z directions to get the sample as shown in Figures 12.6 and 12.7. After calibration, press Ok.

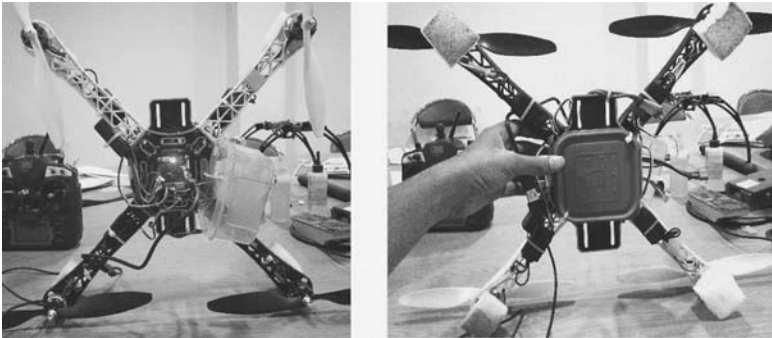


FIGURE 12.6
Calibration steps 1 and 2.

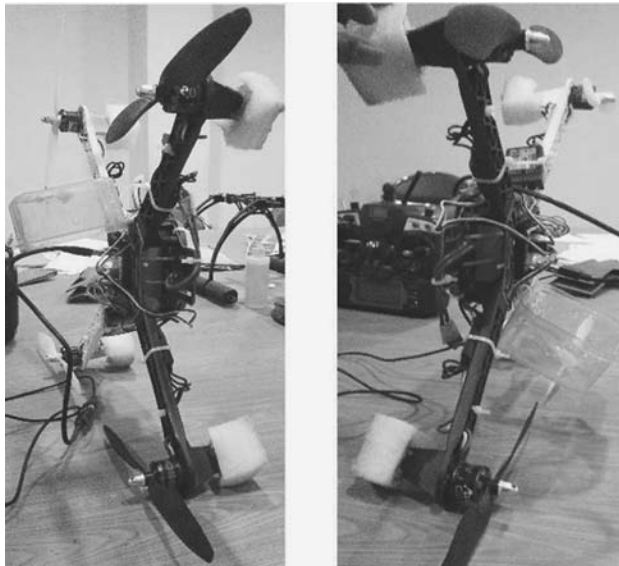


FIGURE 12.7
Calibration steps 3 and 4.

12.6.1.3 ESC Calibration

Calibration of ESC is one of the vital parts of the project because it sets up the maximum and the minimum PWM ranges to the ESC. For all ESC, the range should be the same; otherwise, the copter will show bad attitude and will not fly. While calibrating ESC, make sure that the propeller is detached from the motor to avoid any chance of injury. In the config.h file, there is a macro called `#ESC_CALIB_CANNOT_FLY`. Just uncomment this line and reupload the firmware to the board. As soon as the upload is completed, a musical sound is heard and the ESC will send PWM with a maximum rate; hence, the motor spins with a maximum RPM of 5000 ms.

12.6.2 Configure KK 5.5 Multicopter Board

The KK 5.5 board in Figure 12.8 is another open-source version of multicopter board. It is based on the AVR ATmega328p microcontroller. The KK board is widely used by multicopter hobbyists, and a very basic level of flight operation can be done using this board.

This board consists of nonintegrated gyroscope sensor only (x-gyro, y-gyro, z-gyro). It does not have any pressure sensor, magnetic compass, or onboard GPS unit. The purpose of this flight controller is to provide a very stable flight using only manual mode. Several steps are involved to set up this board. Mainly, it entirely depends on the setting up of the roll, pitch, and yaw gyro configuration. By adjusting three gain pots, the multicopter can be tuned for a stable flight.

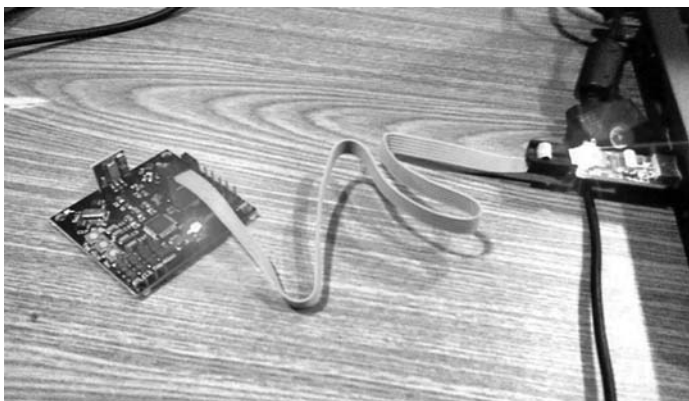


FIGURE 12.8
KK 5.5 multicopter board.

Here are some basic steps to configure the KK board:

1. *Stick centering*

Stick centering can be done in the following ways:

- a. Set pitch gain pot to zero.
- b. Set all trim switches of transmitter to the center.
- c. Power on controller.
- d. LED should flash three times to give ready signal.
- e. Check receiver power and wait for 5 s.
- f. LED should flash one time.
- g. Power off system and restore pitch gain pot.

2. *ESC calibration*

- a. Set yaw gain to zero.
- b. Put throttle stick to maximum.
- c. LED should flash three times.
- d. Wait for 3 s, the LED will then flash thrice.
- e. Wait for motor signal. Put the throttle to zero.
- f. Wait for confirm signal by motor.
- g. Power off and restore yaw pot to normal position.

3. *Gyro reversing*

Sometimes, one may need to reverse the gyro orientation and direction, and thus, the following steps are necessary:

- a. Set roll pot to zero.
- b. Power on.
- c. Reverse the gyro, move all Tx stick right and down, for normal move all Tx stick left and up.
- d. Power off and restore the roll gyro gain.

4. *Clear all settings*

To clear the settings, set all gain pot to zero and then power on the system and wait for 5 s.

Now, power off the controller and all settings will be reset.

12.7 Radio Setup and Calibration

The radio calibration is another step of developing the semi-autonomous UAV. In most of the cases, the radios have a standard frequency range for civilian and hobby drones. The IEEE802.11 standard 2.4 GHz is the standard

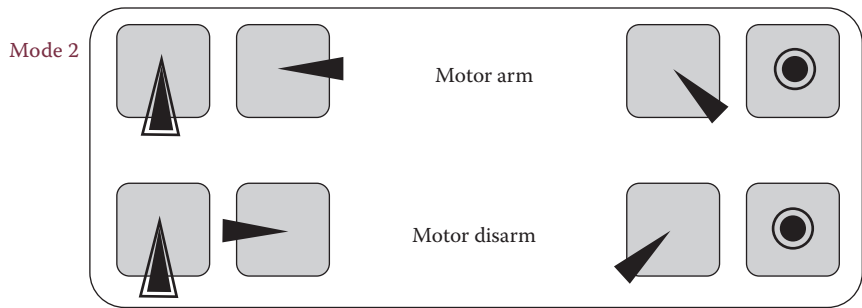


FIGURE 12.9
Arming stick configuration.



FIGURE 12.10
Radio control assignment.

configuration. In some countries, other frequency sets are also preferred. To configure the radio, we have to select the mode of operation first. In normal case, mode 2 is the standard mode in which throttle channel is assigned in channel 3, roll in channel 1, pitch in channel 2, and yaw in channel 4.

The arming and disarming position of mode 2 is shown in Figure 12.9. While, arming at initial test make sure the propeller is took away from the motor.

Figure 12.10 shows the stick functionality of the radio controller in mode 2.

12.8 Radio TX/RX Binding Technique

Normally, the TX and RX of the radio system are bound together by default. But in some special cases, we have to bind them explicitly, and hence, a bind plug is used. A bind plug is a plug that connects the signal and ground of the receiver with a jumper wire, as shown in Figure 12.11. To bind TX and

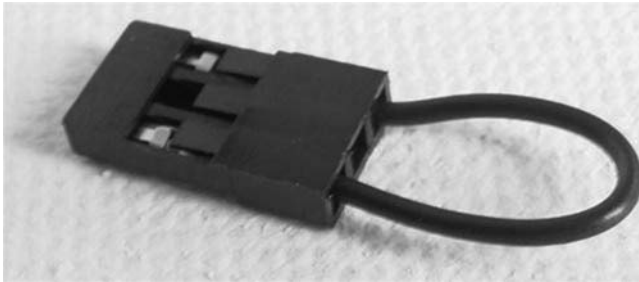


FIGURE 12.11
A bind plug.



FIGURE 12.12
A 2.4 GHz radio receiver.

RX, just place the bind plug in the bind pin of the receiver (Figure 12.12) and then power on the receiver. The receiver LED should blink. Now turn on Tx module by pressing the bind button of Tx. If the RX led is constant, then the binding can be concluded. Now, remove the bind plug from the receiver and repower the receiver and test the Tx/Rx connection by switching on the Tx module. If Rx led is on as you power on the Tx module, then the Tx and Rx module is meant to be connected.

12.9 Connection with GUI Interface

After successful connection with Tx/Rx, use the MultiWii GUI (Figure 12.13) to trim the value of each stick level. The following figure shows the

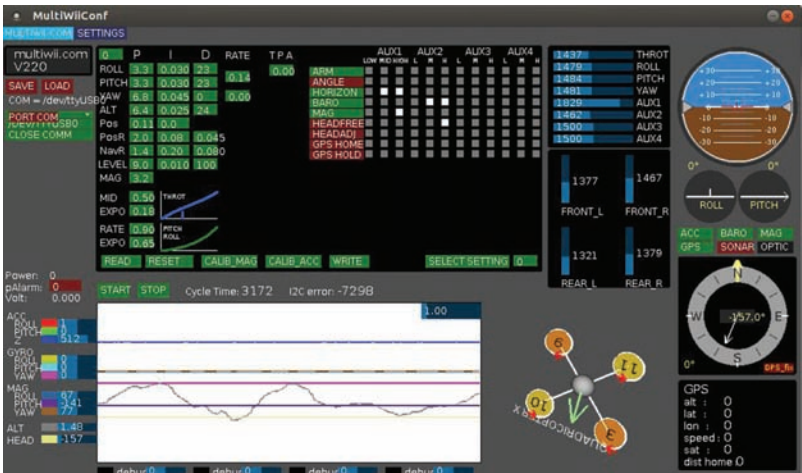


FIGURE 12.13
MultiWii GUI (Linux version).

MultiwiiConf Gui in Linux environment; one can use a Windows version of the same GUI (<http://code.google.com/p/mw-wingui/>). The vertical bar shows the throttle level applied to each motor of quadcopter. We can adjust and trim the throttle level and the pitch, roll, and yaw values by viewing the thrust value applied on each motor from Tx trim switch. The green indicators indicate different types of operation mode. When the ARM is green, it suggests that the UAV is in the armed state. In this case, baro, magnetic compass, and stable mode are enabled. We can assign that mode to any channel between aux1 and aux4. The PWM value corresponding to them will ensure the activity of the modes that have been selected. The white square box shows the mode selection corresponding to high, medium, and low PWM value.

The throttle expo can be set by the GUI to control the rate of throttle applied to the motors while working. On the right side of the GUI, the top-most component gives the level of flight, which has been computed by the gyro angle of the copter corresponding to the land surface. Below that roll and pitch indicator has been provided which shows the current roll and pitch angle. Below the roll±pitch indicator, the sensor and its activity are indicated. Below are the two sub-windows through which we can get the heading of UAV from the compass and the location from the GPS device. At the bottom side of the GUI, real-time graphs are found. This panel gives the current reading of the roll±pitch±yaw of the copter in real time. The 3D quad animation is the visualization of the attitude of the copter that changes at real time.

12.9.1 PID Tuning

Tuning of the PID values in the multicopter system is the most important thing to be performed. PID stands for proportional, integral, and derivative. These factors have a significant impact on the behavior of the multirotor system. The default PID for the MultiWii SE 2.5 series is shown in Figure 12.14. Two types of tuning method are applied.

12.9.1.1 Basic PID Tuning

The default PID setting of the designer is the most recommended setting. Furthermore, it can be modified as follows:

- Hold the copter securely and safely in the air.
- Throttle should be increased up to the point of hovering, where it starts to feel light gradually.
- Try to lean the multirotor down onto the pitch, roll, and yaw axes.
- You should feel a reactive force against your pressure on each axis.
- Change the value of P until it is hard to move against the reaction of the copter.
- Now try the multirotor to be rocking. Increase P until it starts to oscillate and then reduce slightly; repeat the same for yaw axis.

12.9.1.2 Advanced PID Tuning

PID is a closed-loop control system (Figure 12.15) that tries to get the actual result that is closer to the desired result as the input gets adjusted. Quadcopters or multicopters have an internal PID controller that does the stabilization task. Three different algorithms are generated for PID control. P $\propto$ proportional algorithm computes present error, I $\propto$ integral algorithm computes past error, and D $\propto$ derivative algorithm predicts future errors.

0	P	I	D	RATE	TAP
ROLL	3.3	0.030	23	0.14	0.00
PITCH	3.3	0.030	23		
BO YAW	6.8	0.045	0	0.00	

FIGURE 12.14
Default PID.

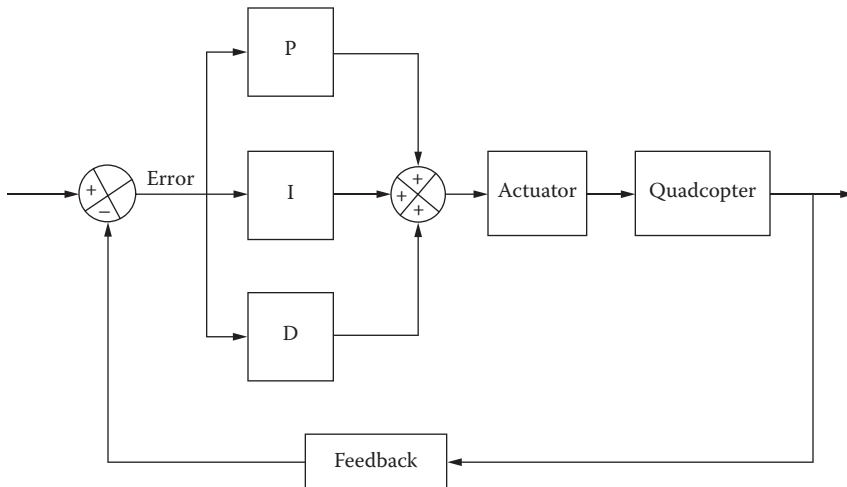


FIGURE 12.15
Closed-loop PID controller proportional.

12.9.1.2.1 Proportional

1. *Increasing value for P:* The multicopter will become solidly stable until P is too high. If it moves out of range, it starts to oscillate and lose control. A very strong resistive force can be observed during any attempts to move the multirotor.
2. *Decreasing value for P:* The copter will start to drift in control until P is too low. When it is too low, it becomes highly unstable. For any attempts to change orientation, it becomes very less resistive.

Note:

Aerobatic flight: Slightly higher P is necessary.

Gentle smooth flight: Slightly lower P value is suitable.

12.9.1.2.2 Integral

A variable amount of corrective force based upon the angle of error from the desired position is provided by I value. The larger the deviation and/or the longer the deviation, the larger is the corrective force. A higher I will increase the heading hold capability, and therefore, it is limited to prevent excessively high I.

1. *Increasing value for I:* Overall position-holding ability increases, drift reduces due to unbalanced frames, and so on.
2. *Decreasing value for I:* Will improve change reactions, reduce drifting, reduce ability to hold position, and cause very sluggish flight.

12.9.1.2.3 *Derivative*

Value D moderates the speed due to which the multirotor returns to its original position. A lower D means a very quick snapback of multirotors to their initial position.

1. *Increasing value of D*: Causes damping to changes. It reacts slower to changes and appears smoother with respect to the applied throttle and other sticks.
2. *Decreasing value of D*: Provides less dampening to changes. It reacts faster to changes. It seems like a small sparrow flying. Aerobic settings are recommended for a micro-sized quadcopter rather than for a giant-sized quad.

Note:

Aerobic flight: D should be lower.

Gentle smooth flight: D should be increased slightly.

12.9.1.3 **Standard Guideline for PID Tuning**

12.9.1.3.1 *Aerobic Flying*

The value of P should be increased until oscillation starts (reduce the value), then the value of I is changed until the wobble is unsatisfactory, and then it is reduced slightly.

Decrease the value of D until recovery from dramatic control changes results in unsatisfactory recovery oscillations, and then increase D slightly.

These steps are repeated.

12.9.1.3.2 *Stable Flying (Using RC)*

Value for P should be increased until oscillations start and then it is reduced slightly.

Value for I should be decreased until it feels too loose/unstable and then increased slightly.

Then the value of D is increased and the same procedure is applied.

12.9.1.4 **General Guidelines**

For stable flying with less wobble (in FPV mode):

- If you have fast wobbles, then lower P.
- If you have slow wobbles, then lower I.
- For smoother changes, increase D.

Note:

For acrobatic flying: Lower D to make sharper snappier movements.

12.10 Position, Navigation, Level, and Magnetometer Performance Tuning

The MultiWii GUI gives the ability to tune the position hold rate and precision rate for navigation. The default performance setting (Figure 12.14) of the MultiWii is smart enough to hold the position most efficiently. The navigation and position hold rates are mostly defined by the GPS position error; moderate p (from 1.2 to 2.5) value for navigation is accurate. GPS with 5 Hz update rate is more appropriate for waypoint navigation of MultiWii-based quad. Extremely low navigation p rate causes a zig-zag motion of multicopter during navigation, whereas a higher rate might cause a jerky performance.

The setting for the throttle expo can also be updated through the MultiWii GUI. The expo and the mid value really change the attitude of the UAV smoothly. For our case 50% throttle value is assigned for the medium throttle whereas expo is 18% that causes a gentle and balanced throttle output. The roll±pitch rate of 90% signifies the mapping of the roll±pitch stick 90 percent to the actual roll and pitch of the copter. Expo of 65% is good enough to properly control the roll±pitch of the quad. The mid and expo value depends upon the size and the architecture of the copter. The expo, throttle, mid, and roll±pitch rates for a standard 450 mm copter are shown in Figure 12.16.

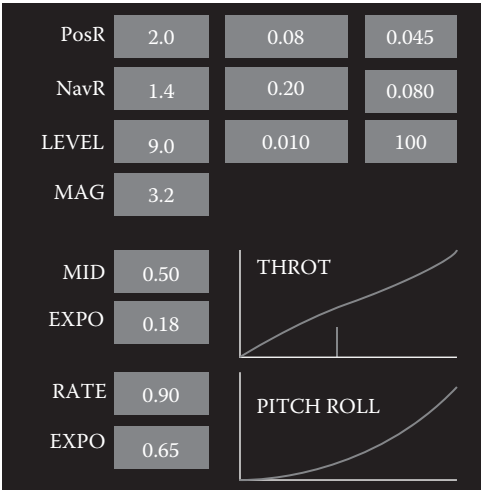


FIGURE 12.16 Default performance.

12.11 Additional Channel Assignments

MultiWii can perform auto horizon, altitude hold, and direction- and position-hold operations. Such operations can be performed by additional channels present in MultiWii and are shown in Figure 12.17. Four channels are dedicated to this, namely, aux1, aux2, aux3, and aux4. For each channel, high, mid, and low range can be utilized for several operations.

- ARM*: Provides arming of the copter motors.
- ANGLE*: Provides a stable flight as the PID had adjusted properly.
- Horizon*: It is basically an amalgamation of ACRO and ANGLE that combines the stabilization effect in case of gentle RC sticks. For busy RC operation we may obtain an aerobatic performance such as flip.
- BARO*: Only the barometer reading is used to maintain a certain altitude, until there is a command from the radio.
- MAG*: Considers a magnetometer/compass as a prerequisite of HEADFREE.
- GPS HOME*: This mode uses compass and GPS to return home from a particular location to the starting point (called launch site). Based on the flight mode (ANGLE/HORIZON/ACRO), this is stabilized. The GPS altitude is not very accurate and, therefore, is not used for holding the height.
- GPS HOLD*: Holds current location by using GPS and barometer (if available).
- HEADFREE*: It only holds the orientation (yaw) of the copter and always navigates in the same 2D direction for the same (roll±pitch) stick movement. This mode never impacts the flight mode (ANGLE/HORITON/ACRO).
- HEADADJ*: New yaw origin has been set for HEADFREE mode.

	AUX1			AUX2			AUX3			AUX4		
	LOW	MID	HIGH	L	M	H	L	M	H	L	M	H
ARM												
ANGLE												
HORIZON												
BARO												
MAG												
HEADFREE												
HEADADJ												
GPS HOME												
GPS HOLD												

FIGURE 12.17
Additional channel assignment.

12.12 Summary

Figure 12.18 shows the quadcopter UAV in flight. In this chapter, we have entirely discussed the design of a semi-autonomous quadrotor system. This system further can be updated to a fully autonomous system by adding a proper navigation and guidance unit (Figure 12.19). The utilization of an



FIGURE 12.18
On-the-fly quadcopter.



FIGURE 12.19
Complete quadrotor.

open-source autopilot has been discussed in detail here. Choosing an autopilot, which is very important to build an autonomous drone, has also been explained.

12.13 Concepts Covered in This Chapter

- Open-source autopilot
- Autopilot sensor calibration
- Speed controller calibration
- Configuring autopilot firmware
- 2.4 GHz radio setup
- Use of MultiWii GUI environments

13

Autonomous Hexacopter System

13.1 Introduction

A hexacopter multirotor is one of the popular multirotor architectures. It is primarily taken into account in various applications such as professional aerial photography, wildlife documentary shooting, etc. A hexacopter system is advantageous over a quadcopter for the following reasons: (1) it is more aerially stable than a quadcopter, especially in extreme wind conditions, (2) it produces more thrust than that is given by the same configuration quadcopter due to its two extra engines, and (3) it has a more robust architecture than the quadcopter. A hexacopter system can land safely even if one engine fails as long as the other five engines are active, while the quadcopter cannot do so. A slight yaw effect will be experienced by the hexacopter in such a case.

13.2 Structural Design of the Autonomous Hexacopter

The hexacopter has six arms, as shown in Figure 13.1, which can be designed in various ways. A normal hex structure is an extension of the + quadcopter or X quadcopter with an additional two arms at the horizontal and vertical positions, respectively. Extensive hexacopter can be designed by implementing an H-shaped hexacopter or hexagonal frame-based hexacopter. In all cases, the mandatory requirements are (1) equal length of the arms and (2) constant distance of the motors from the center.

The propeller orientation for the hexacopter system is similar to that of the quadcopter, that is, all odd numbers of the propellers are clockwise and all even numbers are counterclockwise.

13.3 Components

Components used in developing a hexacopter are similar to those used in developing a quadcopter.

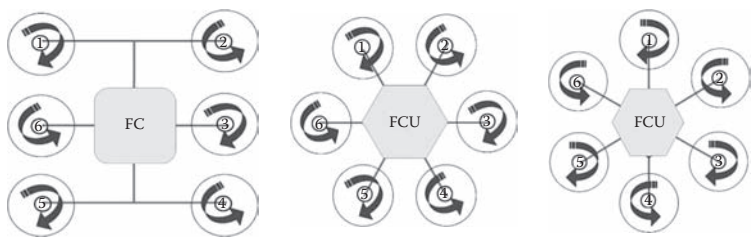


FIGURE 13.1
Different types of hexacopter and their motor spin directions.

13.3.1 Frames

In this example, a standard Hiller HJ550 glass fiber hexacopter frame is used and is shown in Figure 13.2. Similar frame types can be designed using wood or carbon fiber material as well. Care should be taken when attaching the booms with the center of the hexacopter. The distance between all motors should be the same to avoid abnormal oscillation.

13.3.2 Motors and ESC

Here, we have used six 850 kV 2830 brushless out-runner motors along with six 20 A brushless electronic speed controllers. In this project, carbon fiber propellers are preferred instead of normal plastic electric propellers because they are more durable and well balanced.



FIGURE 13.2
Hexacopter frame structure.

13.3.3 Radio Units

In this project, two more radio equipment are introduced along with the previous one and are as follows:

1. The first one is 5.8 GHz for the video transmitter+receiver unit that uses the IEEE 802.11a standards. This unit transmits video data within a range of 100 m. The maximum transmission rate of such system is up to 54 Mbps in ideal condition. The transmitter system has eight channels, while the receiver has four channels. Full specification of the TX and RX unit is as follows:
 - a. Transmitting unit weight: 34.5 g
 - b. Power input: 12 V $\pm$ 5%
 - c. Receiving current: 100 mA
 - d. Transmitting current: 100 mA
 - e. Transmitter channel number: 8 channels
 - f. Receiver channel number: 4 channels
 - g. Antenna gain: 3 db
 - h. Video signal resistance: 75 Ω
 - i. Antenna connection: SMA

The advantage of 5.8 GHz unit is that the Tx/Rx unit is highly robust due to very low interference rate. Signal disruption and noise in this frequency are not very high. Along with the video, the audio data transmission can be done. The disadvantage of such system is that the range of the transmission is reduced at a very high frequency. However, signal amplification can be done to increase the range of the transmission.

2. The second one is a 433 MHz radio Tx/Rx unit (Figures 13.3 and 13.4) that sends the flight information to the base station of the hexarotor UAV. In this example, we have used a typical 3DR telemetry unit. It is an easily configurable telemetry unit that is directly supported by the Mission Planner and the Ardupilot Mega (APM) Planner software and will be discussed later. The system is well configurable in Windows, Ubuntu Linux, and tablet PC. Mostly, the telemetry unit is plugged and played in PC or tablet after installing the Mission Planner ground station software or installing the FTDI interface driver on the computer; refer to http://www.ftdichip.com/Drivers/CDM/CDM20824_Setup.exe or <http://www.ftdichip.com/Drivers/VCP.htm>. The driver is installed as follows.
 - a. Insert the telemetry receiver to the PC. Now right-click ^aMy Computer^o and select ^aProperties.^o



FIGURE 13.3
Ground station telemetry.



FIGURE 13.4
UAV telemetry.

- b. Move to the device manager in the device list and ensure that you see an unknown device.
- c. Right-click the unknown device and then choose ^a Update Driver.^o
- d. As a new window appears, just mention the path of the driver. If a warning window appears, just click ^a Install this driver software anyway.^o

If the driver in .exe format file appears, then it can be directly installed by clicking, but while installation, the telemetry receiver should be inserted into the PC.

The specifications of the telemetry unit are as follows:

Two-way full-duplex communication through adaptive TDM

- UART interface
- Transparent serial link
- MAVLink protocol framing
- Frequency hopping spread spectrum (FHSS)
- Error correction corrects up to 25% of bit errors
- Configurable through Mission Planner and APM Planner
- Supply voltage: 3.7 ± 6 VDC (from USB or DF13 connector)
- Transmit current: 100 mA at 20 dBm
- Receive current: 25 mA

13.3.4 Autopilot Unit

The Ardupilot Mega 2.52 (Figure 13.5) is the autopilot unit. It is highly versatile for more precise autonomous flight. It has an onboard ATmega 2560 microcontroller along with barometric pressure sensor and gyroscope.

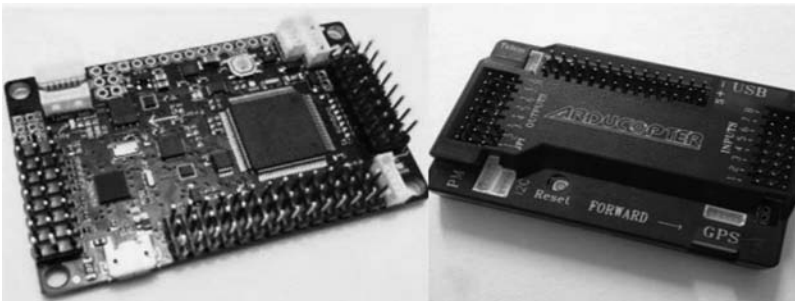


FIGURE 13.5
Ardupilot Mega 2.52.

In the previous versions of APM, an internal measurement unit (IMU) shield is used to attach all the sensors to the microcontroller, but in the new version 2.X, the sensors are onboard except for the magnetic compass and the GPS receiver, which are external systems that can be interfaced with the board through I2C interfacing. The external magnetic compass gives the autopilot more flight stability because of low interference with electromagnetic signals caused by other units of the autopilot.

The APM 2.52 is capable of interfacing sonar, airflow, and optical flow sensors for even more precise flight in extreme wind conditions. There is a provision of onboard power management by attaching an onboard power module for APM. Not only that an on-screen display module can even be interfaced through the APM to obtain the flight information in real time. Many open-source low-cost flight controller boards are available in the market such as MultiWii AIO Pro and Paris Sirius board v4r6, but APM 2.6 is preferred because it performs better and is easy to configure. The APM system commonly uses a special type of protocol known as MAVLink (Micro Aerial Vehicle Communication Link Protocol) to connect and transfer real-time state information from a copter or a plane. Detailed information on MAVLink is found in <http://qgroundcontrol.org/mavlink/start>.

13.4 Component Assembly

The major add-on components that are used to build a fully autonomous UAV are the autopilot board, radio control receiver, and the telemetry transmitter. The system assembly steps are as follows:

1. After attaching the motors with proper screws and corresponding ESC, the autopilot output pin should be attached with the ESCs PWM input plug (Figure 13.6) in a proper way. In ESC, the middle pin is +5 V, black wired pin is GND, and white wired pin is signal, as shown in Figure 13.6. The signal pin should be connected in the inner rails of pins.
2. It is important to connect the motors in such a way that they rotate in a proper direction. The motor spin direction of our hexacopter is shown in Figure 13.7. The assignment of channel from ESC to APM is given in Figure 13.7, that is, starting from the assignment of right-side corner motor to pin 5 and then pins 1, 4, 6, 2, and 3, respectively, in clockwise direction.
3. After the motors are connected, the ESC should be connected with a common power distribution cable so that the battery distributes power to all the motors. In power Distribution Connection we must use soldering to join all the ESCs with power distribution.



FIGURE 13.6
ESC cable. (Courtesy of 3D Robotics, Berkeley, CA.)

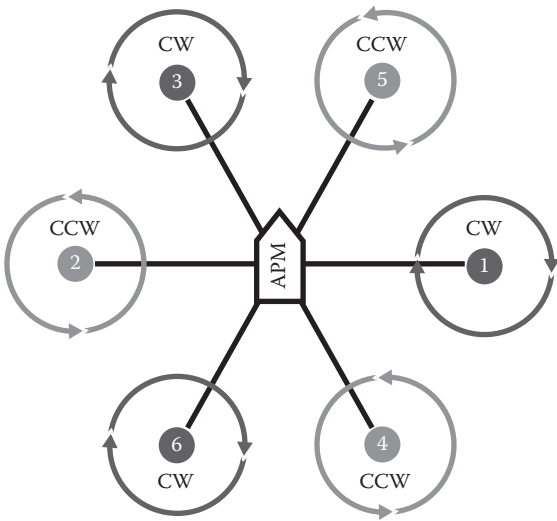


FIGURE 13.7
Motor spin direction and channel assignment.

4. Afterward, the autopilot is placed on the top of the hexacopter. Metal screws are used to attach the electronic components, but plastic screws are preferred to attach the sensor and controller components. Often, a foam-type material is used to mount the autopilot, but make sure that the material is soft enough to observe the vibration created by the frame. The vibration is considered as noise and stabilizes the UAV accordingly, such as other autopilot sensors. While attaching the autopilot, care must be taken because more vibration means more unstable behavior.
5. To connect the radio channels with autopilot, a simple jumper wire has been used. Typical channel assignments for the mode2 RC controller can be maintain to connect the 2.4 GHz radio receiver. The channel assignment is as follows:

Channel	Operation
Ch1	Roll
Ch2	Pitch
Ch3	Throttle
Ch4	Yaw
Ch5	Mode select (optional)

Generally, use the mode select channel to perform various modes of operation via the UAV, where for different PWM output values, the flight mode have changed. In flight mode Section 13.8, we will discuss its details.

6. Telemetry and external GPS with compass module can be connected via a communication port that is assigned to a specific device that is to be connected with APM. The GPS and telemetry modules are added using specific SMD connector that comes with the package. While attaching those modules, no change in configuration is required. Only calibration is a must for the compass installation discussed in Section 13.7.2.

The detailed component assembly is shown in Figure 13.8.

13.5 APM Ground Station Software Installation

The APM Mission Planner software (Figure 13.9) is available at <http://ardupilot.com/downloads/?category=4> and the APM Planner software at <http://ardupilot.com/downloads/?category=35>. For installing a Mission Planner ground station, use an updated version of DirectX. Next, just click

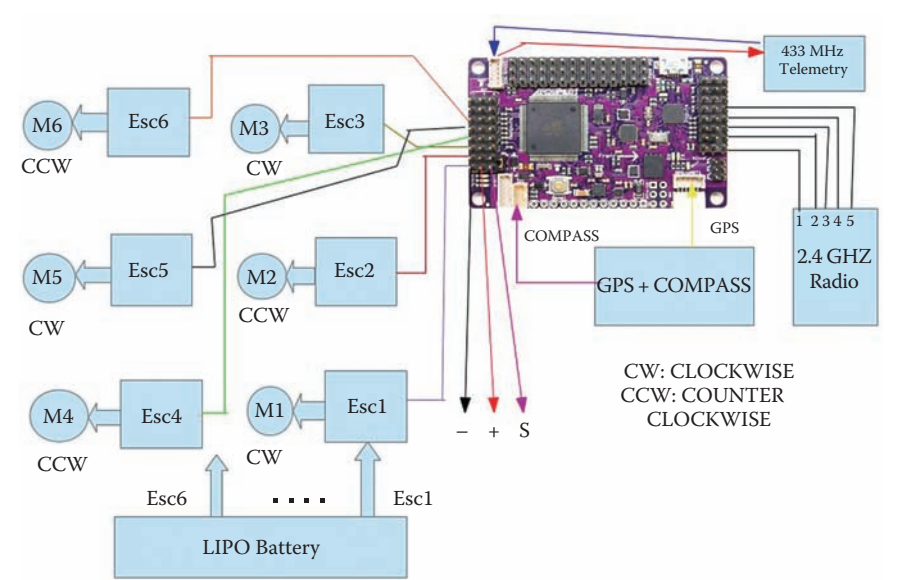


FIGURE 13.8
System architecture (assembly).



FIGURE 13.9
APM ground station on-flight data.

on the .msi file and the setup wizard will start. In the case of the APM planner, the ^a.exe^o file is available to be installed by direct clicking. Mission Planner installation window might ask for the driver installation. In such case click the option ^ainstall driver software anyway^o.

13.6 APM Firmware Loading

After installing the APM ground station software, connect the APM hardware with ground station. If the Arduino driver is already installed in the computer, then ground station will automatically detect the APM hardware as Arduino Mega2560 and setup the communication port to the COMXX. Now, set baud rate to 115200, start the wizard, and select the frame type shown in the wizard window, which is typically the hexacopter. As we choose the copter type, the wizard (Figure 13.10) automatically loads the firmware corresponding to the given frame.

13.7 Sensor and Radio Calibration

13.7.1 Accelerometer and Gyroscope Calibration

The sensor should be calibrated to make the copter work properly. Calibration of the accelerometer and gyroscope is important. The two sensors' unit is

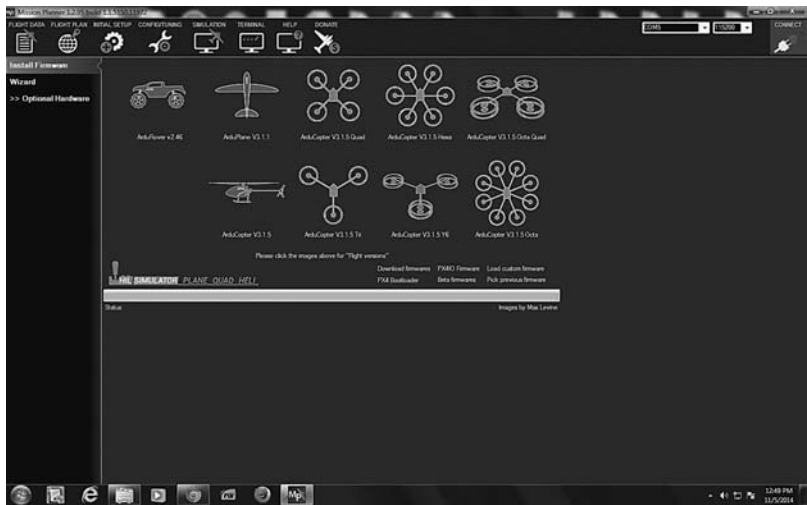


FIGURE 13.10
Firmware loading wizard.

**FIGURE 13.11**

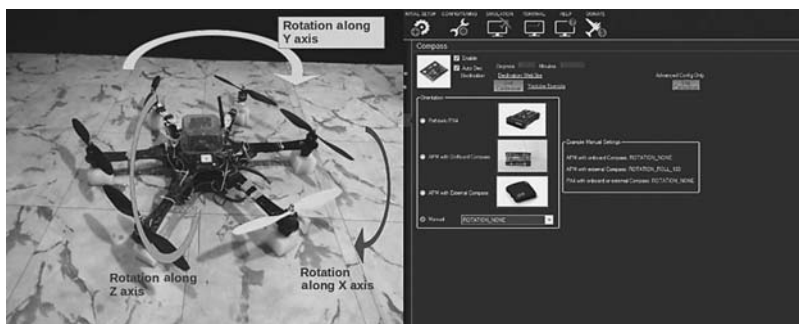
Accelerometer calibration step.

highly responsible to make the copter fly. The APM ground station has a complete wizard that helps to calibrate the copter efficiently. On the calibration window, click on the Accelerometer calibration. Now, follow the directions as in the APM software. Press the button and APM directs to move the copter in level, then press OK, then move the copter left, right, nose up, nose down, as shown in Figure 13.11, and finally back. When the entire movement operation is complete, the APM automatically shows that the calibration is successful.

When starting calibration, make sure that the copter is placed on a proper leveled surface, otherwise the calibration might go wrong. The calibration should not be started immediately after pressing the OK button; a delay of a few seconds is recommended.

13.7.2 Compass Calibration

After performing the gyro setup, calibration wizard asks for the calibration of the compass as there is a time period of 60 s to calibrate it. Note that as you start it immediately starts to take samples. Then, rotate the copter from 0° to 180° from left to right, front to rear and horizontally in its axis as well (Figure 13.12). Ensure that the APM wizard shows enough samples (above 300) after calibration. Before calibration, the orientation

**FIGURE 13.12**

Compass calibration window to select compass type.

(COMPASS_ORIENATATION) should be set in a proper direction. The orientation specifies the heading offset of the compass with respect to north. In our case, it is 0°, which means that there is no orientation.

13.7.3 Radio Calibration

After switching on the radio (Figure 13.13), ensure that the radio is in airplane mode. In our example, we chose a mode2 radio, where right vertical stick implies pitch, horizontal stick implies roll, left vertical stick implies throttle, and horizontal stick implies yaw. While starting calibration, make sure that all sticks are in middle position. As we start calibration, put all sticks and switches in minimum at once that shows the minimum allowable PWM range of the radio stick. Then, all of them should be set to maximum allowable PWM value. Now, press the end calibration button. The system will record minimum and maximum PWM values corresponding to each stick and switch.

13.7.4 ESC Calibration

To calibrate the ESC, first disconnect APM from the ground station computer. Power on the radio transmitter, then put the throttle channel to maximum output, and connect the battery to APM. A musical tone comes from ESC and the LEDs blink in cyclic order. Now, the system is in calibration mode. Disconnect the battery and again connect with the system; a musical tone followed by three beeps is heard, which indicates the number of cells in the battery (three beeps means three cells in our case). The LEDs blink similar to those in a police car. It indicates that the calibration has started; therefore, the throttle should be in its minimum position and the continuous beep tone is heard. Raise the throttle, and thus, the motor starts to spin. Before increasing the throttle, ensure that propellers are not mounted on the motor. Remove the propellers before ESC calibration for safety purposes.

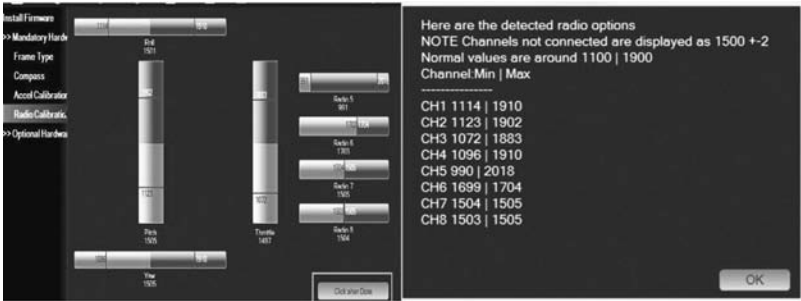


FIGURE 13.13
Radio calibration and results.

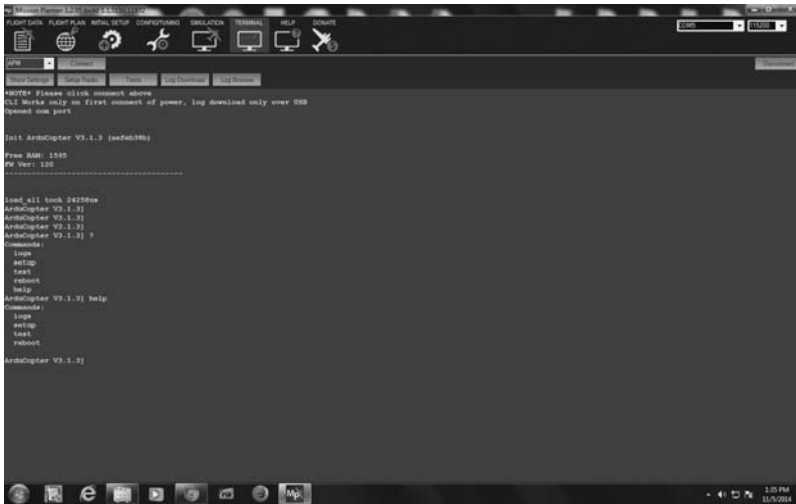


FIGURE 13.14
CLI interface.

13.7.5 Motor Test

After calibration, we can check whether the motors are spinning in a proper direction or not. We have to use an interface in the ground station known as CLI (command line interface; Figure 13.14). While entering into the command line, interface just reconnects the APM hardware and observes the command console that will appear. Just type Help on the console to get the necessary command.

To enter into test mode, give command] *test*, and as test mode appears, just put command motors in test mode *test] motors*; as you enter motor command, motors will spin one by one. Then, you can check whether all the motors are moving in proper direction or not. If it is not in proper direction, then just press enter to stop spinning and change the polarity of the motor cable.

13.8 Flight Parameter Settings

To get a successful flight, we need to set up proper flight parameters. The parameters that are highly important to fly the UAV in an efficient manner are as follows.

1. **ARMING_CHECK**: if this parameter is enabled, then it will check all the sensor health before arming the copter. If all the sensors including GPS value are good, then it will permit the copter to fly; otherwise, the copter will not. This check is basically a safety check.

2. *MAG_ENABLE*: if this parameter is enabled, the magnetic compass starts its function. It is highly recommended when autonavigation is performed.
3. *AHRS_GPS_USE*: this parameter performs the function of attitude heading reference system GPS that is mainly used for GPS-based waypoint navigation. All flight modes that depend upon the GPS are enabled by this parameter. Before using GPS, make sure that the GPS is locked and it shows 3D Fix.
4. *RTL_ALT*: this parameter sets the altitude of the copter while returning home by a specified meter by the user.
5. *RTL_ALT_FINAL*: this shows the final altitude achieved by the copter when it returns to the base. This should be 0 when the copter is about to land.
6. *RTL_LOIT_TIME*: this describes the time in seconds taken for the loitering of the copter before landing.
7. *LAND_SPEED*: this describes the speed in centimeter/seconds to land the copter at ground.
8. *FS_GCS_ENABLE*: this describes the ground station fail safe enable. If the copter and ground station link has failed, it immediately returns to home.
9. *FENCE_ENABLE*: this enables the boundary limit of the copter on the basis of the altitude and horizontal distance; supplied by user as meter.
10. *FENCE_TYPE*: this provides the type of fence in terms of altitude, circle, or both.
11. *FENCE_ACTION*: setting up this parameter either copter performs RTL or report.
12. *FENCE_ALT_MAX*: this sets the maximum altitude of fence from 10 to 1000 m.
13. *FENCE_RADIUS*: this sets the maximum radius of fence from 30 to 10,000 m.
14. *GPSGLITCH_ENABLE*: this enables the GPS error protection due to improper GPS reading.
15. *BAROGLTCH_ENABLE*: enabling this parameter altitude error performed by barometer gets protected.

13.9 Flight Modes

A multicopter system may run on several flight modes. The main purposes of the flight modes are to perform specific tasks on the fly. In APM, there are

several modes of operation to fly the copter in manual, semiautonomous, and autonomous modes.

1. *Stabilize mode*: In this flight mode, the pilot manually controls the flight and a stable flight is achieved. In this case, only accelerometer and gyroscope are used to control the flight of the copter.
2. *Altitude hold*: In this mode, a semiautonomous performance of the copter has been obtained based on gyro, accelerometer and barometer. In this case, the copter will hold a fixed altitude, but it will not lock the position. Pilot can control the position and the direction in this mode.
3. *Loiter*: Loiter is a fixed-altitude mode, where the copter holds the altitude and positions simultaneously. Here, the GPS lock is essential to achieve this mode. This is an autonomous mode, where the pilot controls only the heading of the copter.
4. *Circle*: This is an autonomous mode in which the copter performs a circular path, where the radius of the circle is given by the user.
5. *Auto*: Auto is a fully autonomous flight mode where the copter follows a predefined flight through some predefined waypoints. The edition and mission planning of waypoints are covered in the following section.

13.10 Mission Design

To perform an autonomous flight, mission design is the most important part in APM. The Ardupilot Mega board supports APM Mission Planner tool (Figure 13.15) as well as APM planner tool to design a specific mission. Most of the mission consists of n number of waypoints and a home location.

13.10.1 Using Ground Station

The ground station software consists of real-time flight data as well as flight planner tab. In the real-time flight data tab, we can monitor the status and the operation that are currently done by the UAV. It will also help to visualize the telemetry log data. A telemetry log (.tlog) is generated by the telemetry module during flight operation. Each time it is saved under the Mission Planner installation folder/tlog subfolder. One can open the telemetry log and view and analyze the performance of the flight after the completion of flight in off-line mode. The detailed component description of real-time data tab is shown in the following.



FIGURE 13.15
Mission planning in ground station.

The flight planner gives complete ability to design a mission in specific regions using a predefined map. In the planner window, by using drag and drop method, we can add the waypoints and set the flight altitude of the waypoints by right-clicking on the waypoints. We can configure some specific mode of operation with respect to a certain waypoint. The autopilot-based waypoint navigation can be performed under certain parameter settings of the APM. The user can even change the parameter to set the specific task. The parameter setting window is shown in the following.

At all points, we can read the previously loaded waypoints into the flight planner as well as write new waypoints as per our requirements. We can choose the map provider from the Mission Planner as well.

Often, the APM Planner software is used to configure a flight mission on non-PC devices such as the smartphone and tablet. In this environment, the mission planning technique is the same. Here, the properties of waypoints can be set up directly as the list of waypoints are available in the planner.

13.10.2 Waypoint Navigation Algorithm

The Ardupilot Mega uses a precise waypoint navigation algorithm that basically defines a procedure to navigate the UAV within a certain predefined path. The waypoint navigation algorithm uses the GPS location and the heading of the UAV. An APM-based UAV starts its mission with a stabilized mode flight. After that, the auto mode is enabled and the UAV performs

the waypoint navigation based on the location information and the bearing value taken by the compass. The autopilot mode gets executed in a sequence of steps as shown in the following algorithm.

Algorithm 13.1

```

compass_value ← Read 10 Hz compass
Throttle_value ← Take Throttle I/P
GPS_location ← read 10 Hz GPS
initial_location ← set_home(compass,GPS_location)
Final_loc ← read from wp list
if: Navigation FLAG=OK
  do: Navigate while(! Final_loc)
    loc2 ← read_target_location
    loc1 ← initial_location
    dlat ← loc2.lat - loc1.lat
    dlng ← (loc2.lng ± loc1.lng)*(cos(|loc2.lat|/107))
    ofx ← loc2.lng - loc1.lng
    ofy ← loc2.lat ± loc1.lat
    distance ← sqrt(sqr(dlat) + sqr(dlng))*0.01113195
    dfact ← distance/R
    bearing ← 9000+arctan(-dlng/dlat)*(18000/3.141)
    targetlat = arcsin(sin(loc1.lat)*cos(dfact) +
      cos((loc1.lat)*sin(dfact)*cos(bearing))
    targetlng = loc1.lng+arctan((sin(bearing).sin(dfact).cos(loc1.lat))/
      (cos(dfact)-
        sin(loc1.lat)*sin(updatedlat)))
    end while
  end if

```

This algorithm shows that the autopilot takes the compass and the GPS readings at a frequency rate of 10 Hz and the GPS location information gets stored. Then, the initial location is set with the given location information and the compass information. After that, the navigation flag is checked, and if it is OK, then the autopilot is ready to navigate. Navigation has been performed, but navigation location is not the final location. Subsequently, compute the distance of the next location by equilateral projection methodology. That will be performed by computing Δ latitude and Δ longitude then finally $\sqrt{(\Delta\text{latitude})^2 + (\Delta\text{longitude})^2 \times \text{scale factor}}$. The distance bearing is also computed by $9000 + \arctan(-\Delta\text{longitude}/\Delta\text{latitude}) \times (18,000/3.141)$, where 9000 and 18,000/3.141 is a constant; the first value signifies the turn toward 90° north and the second value is the scale factor. The distance and bearing are used to compute the target latitude and longitude of the next waypoint traveled by UAV.

13.10.3 GPS Glitch and Its Protection

In the case of waypoint navigation, the navigation of the copter is often hampered due to erroneous information of the GPS location. Such a phenomenon is known as GPS glitch and is shown in Figure 13.16. The primary cause of the GPS glitch is the unavailability of the GPS signal, GPS denial, and high dilution of precession value. At times, cloudy weather, tall buildings, trees, hills, and large obstacles may scatter GPS signal, which results in bad GPS data.

In the APM system, various techniques are present that protect from a GPS glitch. In the case of lower version, the board glitch protection logic is designed in such a way that during bad GPS position, it will alert on ground station. The glitch can be detected by comparing the position an the new position update given by GPS along with the velocity and the new position. It is said to be good if the following condition holds

1. The two points are within the GLITCH_RADIUS, that is, by default 5 m, but the user can update it as per requirement.
2. In the new position, of copter within the glitch radius is under acceleration of 10 m/s^2 .

The extended Kalman filter (EKF)±based inertial navigation can be selected by choosing `AHRS_EKF_USE = 1` parameter.

1. As the GPS position measurement has been used, they are compared with the internal IMU measurement.
2. If the difference exceeds the statistical confidence level set by `EKF_POS_GATE`, then the measurement is neglected.

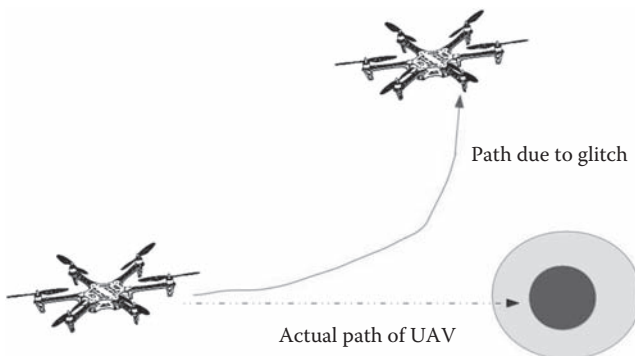


FIGURE 13.16
GPS glitch.

3. While the GPS location information has not been used, a circle has been defined within the uncertainty radius that has grown around a predicted location. The rate of the growing of the circle is controlled by `EKF_GLITCH_ACCL` parameter. The value of acceleration is nearly about 1.5 m/s^2 , and it might vary depending upon the aerial maneuver.
4. If the subsequent location falls under the circle, then the location is accepted and circle radius gets reset.
5. If the duration of glitch is very long, then the location value gets rejected subsequently. Hence, the radius of the circle increases until `GPS_GLITCH_RAD`. In such case, an offset value is set to GPS position so that it matches to the estimated vehicle position. The offset gets reduced to zero with a rate of 1 m/s , causes a drift on the copter with a rate of 1 m/s , and results in enough time to control the UAV by ground operator.

13.11 Adding FPV Unit

Figure 13.17 shows the first-person view (FPV) unit, which is a most obvious module in an autonomous UAV. When a UAV performs a mission, it may move to a high altitude or it goes out of sight. In such cases, the FPV plays a significant role, which views the flying activity and operation of the UAV. Normally, an FPV module is mounted on the UAV in a hard-mounted fashion, that is, like attaching other modules directly over the UAV using

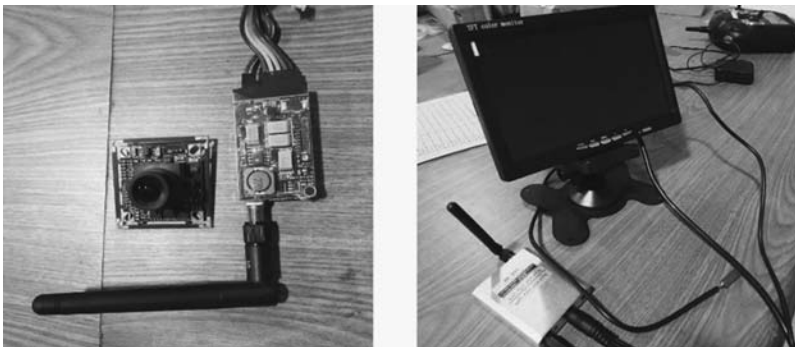


FIGURE 13.17
5.8 GHz video transmission and receiving unit.



FIGURE 13.18
FPV display from ground station.

screws or nuts. Another solution is to attach the camera module with a camera gimbal or camera stabilizer.

In most cases, the FPV is done by transmitting live video feed from the UAV to the ground station. In this project, we have used a 5.8 GHz video transmitter/receiver system that is superior enough to send video data in real time. As the frequency range of this module is very high, it causes less interference, and hence, the clarity is obviously of good quality.

To get better FPV video, the video transmitter is placed away from the other transmitters such as 2.4 GHz radio, telemetry, and the ESC because they might add some noise to video signals, and therefore, the signal might be disrupted. The following figure 13.18 shows the detail description of the FPV transmitter with camera and receiver installation. Figures 13.17 and 13.18 show the FPV view as well as its setup, respectively. In the video configuration window, make sure that the video device is showing. In video format selection, select proper video format to see the bird's-eye view of the FPV.

13.12 Final Hexacopter UAV

The final hexacopter UAV is displayed in Figure 13.19, which shows on-the-fly hexacopter, and Figure 13.20, which shows the hexacopter UAV.

13.12.1 Flight Path Visualization and Log Analysis

As autonomous flight is performed successfully, the APM telemetry system logs the entire flight information including the attitude and the behavior of the copter. The log is stored as the `.tlog` file in APM root folder. The activity of the



FIGURE 13.19
On-fly hexacopter.



FIGURE 13.20
Hexacopter.

UAV can even be visualized and analyzed with respect to the sensor behavior and UAV attitude. To avail this functionality, just load the `^a.tolg^o` file in the Mission Planner and export a `.kml` or create a graph log to analyze the flight. Then, download the data flash log that provides the complete visualization of the sensor behavior during flight.

13.13 Summary

This chapter has covered various methodologies to build a prototype autonomous hexacopter UAV. Subsystems of a hexacopter starting from its frame type to its autopilot have been explained in detail; FPV system has also been discussed. At this point, how the basic waypoint navigation algorithm works when the UAV moves into autonomous flight mode has become clear. In a nutshell, this chapter helps to create a fully autonomous hexacopter in a step-by-step fashion.

13.14 Concepts Covered in This Chapter

- How to select hexacopter frame
- How to add necessary components in hexacopter
- How to calibrate the sensors
- How to set up flight parameters
- How to set up flight mode
- How to set up ground station and mission
- How to avoid GPS glitches
- How to install an FPV in UAV

14

Conclusion

14.1 Tools Used

In this book, various tools used to set up mechanical and electrical devices are described, besides explaining how to install components in a robotic system. The tools that are used in the robotics projects are as follows:

- *Electric drill*: It is an essential tool for robotics project developers (Figure 14.1). Several drills are available in the market. Drill bits (Figure 14.2) are chosen based on the size of the holes that are to be made. Therefore, the drill bit size is most important in this context. Various drills such as a taper point drill, brad point drill, double ended drill, and three-wing drills are most often used. Normal bits are used for wood and stainless steel. High-carbon steel bits are recommended for drilling thin or thick metals. The 0.39 mm bits are used for soft wood, while the 0.79 mm bit is used for both hard and soft woods. The 1.19 mm bit is optimum for harder woods.
- *Hacksaw*: In robotics projects, panels are cut to design frames of robots, in which a standard hacksaw blade is used and is illustrated in Figure 14.3. Various types of hacksaws are needed to perform this task. Generally, manual or electric hacksaw blades consist of alloy steel, high-carbon steel, or high-speed steel. Standard saw blade is generally 8 teeth per inch (8 TPI). It is ideal to use 1.8 mm thick 14 TPI blades for wood and metal frames, whereas 1.4 mm 18 TPI blades are ideal for heavy metals. Medium and thin metals that are highly recommended to make robotic frames require 1.1 and 0.8 mm thick, 24 and 32 TPI, respectively.
- *Screwdriver set*: Choosing a proper screwdriver (Figure 14.4) may significantly reduce the development time of the project. Several types of screwdriver set are available in the market. A set with a larger variety of screwdrivers is always preferred than a set with a lesser variety and more cost.



FIGURE 14.1
Electric drill.

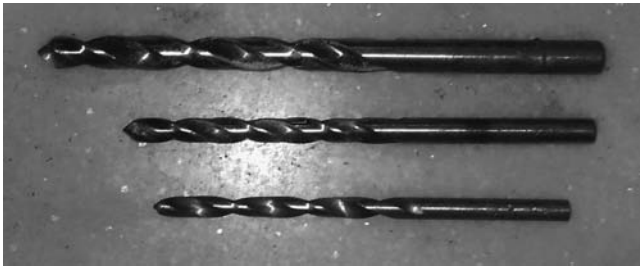


FIGURE 14.2
Drill bits.

- *Zip ties and tapes:* Zip ties are very useful components (Figure 14.5). A set of 100 zip ties can be bought from any local electrical shop. They are often used for electrical wiring and hence are called wiring ties. Zip ties are available in the market in various sizes. Generally, a nylon 200×3.0 mm or 200×3.2 mm tie is suitable to hold the components of the projects. Tape is often used in electrical wiring. It could be used to attach components in various robotic parts. The only drawback of tape is that it becomes loose very easily as the adhesives gradually fail.

14.2 Important Safety Notes

Design of an embedded system and robotic device is quite challenging and interesting too. While working with various robotic projects, a couple of safety measures should be maintained. Often, the robot body and frames are

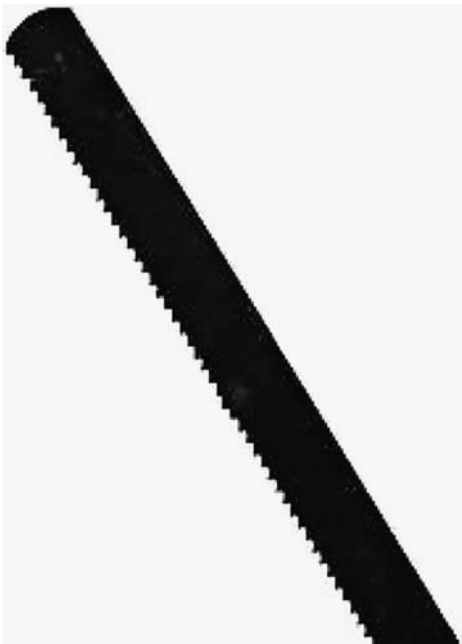


FIGURE 14.3
Hacksaw blade.

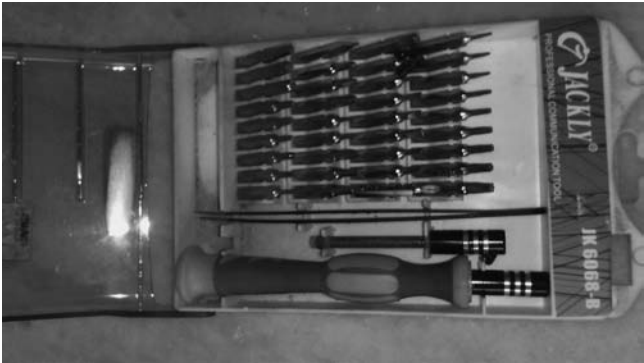


FIGURE 14.4
Screwdriver set.

made of hard plastic materials, woods, or metals. Therefore, these materials have to be cut in the desired shapes. A high-speed cutting tool is used to cut those materials. Electrical cutters are very high-speed instruments and one should be careful while handling them. A better way to use those devices is to wear hand gloves and safety goggles. The drilling of various parts is another important issue. The proper drill bit must be chosen for drilling.

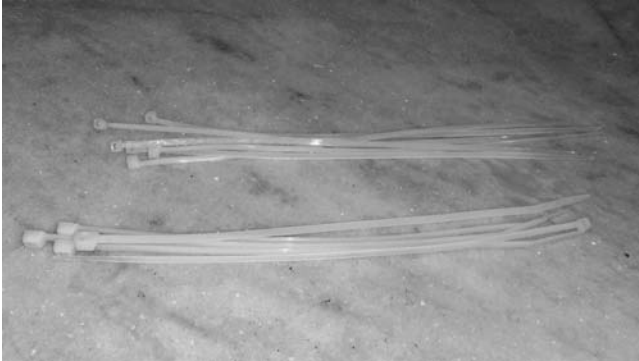


FIGURE 14.5
Zip tie.

Various drill bits are available, and the appropriate drill has to be chosen based on the holes that are to be made. If not, the process might be unsatisfactory. While drilling, it should be ensured that any electronic device or battery or expensive component is not present in the vicinity of the working environment. If present, the high-speed drill bit might slip and come in contact with expensive components nearby, thus causing serious damage to them. If it comes in contact with a battery, then it might explode.

For a multicopter project, one of the most important safety issues that has been already discussed is the removal of propellers while testing the motors and electronic speed controller (ESC) during calibration. During ESC calibration in MultiWii, the motor spins with its maximum allowable speed range of ESC, which is high enough to cause an injury. The Ardupilot Mega (APM) provides a safer way to avoid injury by controlling the maximum revolution per minute (RPM) through the applied throttle.

14.3 Frequently Asked Questions

What is GPIO?

GPIO is the abbreviation for *general purpose input/output*. These are the headers/ports that are dedicated to various external devices that can be interfaced with a single-board computer like Raspberry Pi. Devices such as the LCD screen and touch keypad are some of them.

What is a watchdog timer and what is its use?

This is a special kind of timer to detect software malfunction of any automated computer system. In most embedded systems, if the firmware/software hangs and causes a locked situation, the watchdog timer immediately sends a reset pulse to restart the system.

What is S-Video?

S-Video stands for super video, which is a technology to transmit video by dividing it into two separate forms of signal, one for color and the other for brightness. The signals when sent to a television result in a sharper image.

How does a soil moisture sensor work?

A soil moisture sensor is basically a sensor having two electrodes. When it is in the soil, current flows through the soil. The humidity of the soil directly depends upon the volume of water in the soil. The resistivity of the humid soil also changes as the volume of water changes in an inverse proportional rate.

What is the maximum output voltage for Arduino?

There are two different output voltages for Arduino: 3.3 and 5 V.

What is the purpose of the AREF in Arduino?

The AREF configures the default reference voltage supplied by the Arduino input pin. It is recommended that the AREF should be between 0 and 5 V.

What is the job of Tx and Rx in Arduino?

This is the UART communication port in the Arduino. The PWM pins 0 and 1 are dedicated to this application. These pins are used to connect a computer or other serial device directly to the Arduino.

What is the job of I2C in the Arduino?

I2C stands for inter-integrated circuit communication. In the Arduino, it is basically used to communicate with other peripheral devices such as GPS module.

What is Processing?

Processing is an open-source development environment and a programming language for 2D and 3D visualization and interfaces with hardware and several web-based software services.

What is Firmata?

Firmata is a library that contains Firmata protocol. The purpose of this protocol is to make communication with a microcontroller device to the software application for any other realtime device that may be a PC or laptop. Several utility methods and callback functions are available in the Firmata library to develop one's own version of the firmware without creating a new ownership protocol.

What are the different OS supports for Raspberry Pi?

The Raspberry Pi has several OS supports such as Pidora, which is a Pi version of Fedora. The Raspbian Wheezy is a Linux-based OS dedicated for Raspberry Pi. The Ubuntu MATE is a Ubuntu desktop version for Raspberry Pi. Open-elect OS as an Entertainment Center, and RISC OS, which is a non-Linux distribution. The OSMC and Raspbmc are the Media Center OS for Raspberry Pi.

What is Pidora?

Pidora is the lightweight version of Fedora exclusively designed for Raspberry Pi. It is primarily compatible with most of the ARM family system.

What is the Python support for Google Spreadsheet and Excel in Linux and Windows?

Google Data for Python Library, often called Gdata, is a support for Google Spreadsheet. The Openpyxl is the Python support for Excel\Libra Calc application for Linux and Windows.

What is PuTTY?

PuTTY is an open-source hyper terminal emulator program. It is an OS mostly used for Network File Transfer and Network Configure. It supports Telnet, SSH, and raw socket connection.

What is IEEE 802.15.4?

It is the standard for low-power, low-range wireless communication. ZigBee mesh networks follow this standard where high data throughput and low power consumption are desirable.

What are the differences between Bluetooth and ZigBee?

Bluetooth is a standard of IEEE802.15.1, while ZigBee is a standard of IEEE802.15.4. In Bluetooth, a maximum of eight cell nodes can be connected to each other, but in ZigBee, more than 65000 cell nodes can be connected. ZigBee is a self-healing network, where if one node gets damaged, it creates a revert loop, but Bluetooth has no provision to implement self-healing. A Bluetooth network is established at a point-to-point master+slave basis and a maximum of seven slaves can be allowed. ZigBee can implement mesh, star, or any other generic topology. The data transfer rate of ZigBee is 250 kbps but that of Bluetooth is 1 Mbps.

What to do if the magnetic compass in APM shows reverse direction (east as west and vice versa)?

Set the compass orientation in compass calibration window as ORIENTATION_NONE.

What action is taken if the motor is not ARMing in MultiWii 2.5?

In most versions of the MultiWii, the motor will arm after reaching a specified PWM value of throttle a yaw. For mode2, operation of a radio arming parameter should be #define ALLOW_ARM_DISARM_VIA_TX_YAW and should define min throttle as 1150 by writing #define MINTHROTTLE 1150 in config.h. If it does not work, change the min throttle values and uploads to board and test them with a hit-and-trial method until the arm indicator is not green.

What is AHRS?

Attitude heading reference system comprises sensors on three axes and provides attitude information for the aircraft, including roll, pitch, yaw, and heading. Most of the AHRS consists of MEMS sensors and processing unit. The main difference between AHRS and IMU is that the AHRS has an

inbuilt capability to process the attitude information, whereas IMU provides only raw sensor data.

What is the HDOP value of APM GPS?

The HDOP value can be seen in the GPS panel of Ardupilot Mission Planner software. It is called horizontal dilution of GPS satellite precision, which refers to the level of accuracy the device can compute its exact location. It changes based on several atmospheric parameters. Generally, a DOP value below 1.5 is highly optimized for GPS-based waypoint navigation. A value of >2 is not suitable for navigation.

How to increase the flight time of the multicopter?

The adjustment of the motor and propeller rating is challenging as it increases the flight time of the multicopter. Low RPM such as 850 kV and a high-torque motor generally needs a larger propeller. Larger propeller surface produces more lift with less voltage and therefore affects the flight time significantly in a positive way. The second thing is the weight; if the weight is high, then the lifting power is also high and hence requires high wattage. Avoid unnecessary load.

What is MEMS?

MEMS, expanded as microelectromechanical system, is a technology that can be defined as the miniaturization of mechanical and electromechanical elements. The size of the MEMS device varies up to certain microns. Recently, many sensors such as gyro, accelerometer, and compass have been available in the form of MEMS.

What to do if the multicopter moves in any absurd direction during waypoint navigation?

Typically, the multicopter moves away to an absurd position due to an improper magnetic compass reading and GPS glitch. The reading of the magnetic compass may be affected by electrical interference of the signal wires or due to interference with control and video transmission channels. This can be avoided by making enough separation between wires and wireless module and magnetic compass module. The GPS glitch can be avoided by setting up a small glitch radius. Finally, if things become worse, it is better to take manual control and manual landing.

What is Geo Fencing for Ardupilot?

In Ardupilot Mega, Geo Fencing is the maximum allowable radius for a drone to travel. If it reaches that maximum allowable radius, necessary actions such as alert, auto landing, and return to launch can be performed.

What to do if the multicopter takes a yaw in an arbitrary direction in MultiWii 2.5?

In general, the MultiWii has two yaw control options. The config.h `#define YAW_DIRECTION` parameter tells the direction of yaw. By default it is 1, if the copter gets yaw in any direction, and then just change it from 1 to -1 . Make sure the motors are placed properly on the boom of the copter.

14.4 Final Words

In this book, various cutting-edge technologies have been discussed and the ways to interact with these technologies have been explained. Initially, the concept of open-source hardware and software gives a very generic knowledge of those systems that is easily developed and deployable. The open-source hardware discussed in this book is a new concept that is widely used in the development of various cutting-edge products. Along with that, the concept of robotics and aerial robotics has been introduced. Aerial robotics is a new concept through which we can conceptualize and implement robots that are able to fly. In most cases, fixed-wing and multirotor drones (and sometimes a swarm of drones) are used to perform a collaborative task as aerial robots.

It Is the Time, Start Making Things

Bibliography

- Acosta, R. Open source hardware. PhD diss., Massachusetts Institute of Technology, Cambridge, MA, 2009.
- Afraimovich, E. L., O. S. Lesyuta, and I. I. Ushakov. Geomagnetic disturbances and operation of the GPS navigation system. *Geomagnetism and Aeronomy C/C of Geomagnetizm i Aeronomiia* 42(2) (2002): 208±215.
- Audronis, T. *Building Multicopter Video Drones*. Packt Publishing Ltd, 2014.
- Aye, T. S., P. T. Tun, Z. M. Naing, and Y. M. Myint. Development of unmanned aerial vehicle manual control system. *World Academy of Science, Engineering and Technology* 2 (2008): 371±375.
- Barabanov, M. A Linux-based real-time operating system. PhD diss., New Mexico Institute of Mining and Technology, Socorro, NM, 1997.
- Bayarsaikhan, B. Ultrasonic distance sensor. U.S. Patent D637,096, issued May 3, 2011.
- Beard, R. W., D. Kingston, M. Quigley, D. Snyder, R. Christiansen, W. Johnson, T. McLain, and M. Goodrich. Autonomous vehicle technologies for small fixed-wing UAVs. *Journal of Aerospace Computing, Information, and Communication* 2(1) (2005): 92±108.
- Billah, M. M., M. Ahmed, and S. Farhana. Walking hexapod robot in disaster recovery: developing algorithm for terrain negotiation and navigation. *Proceedings of World Academy of Science, Engineering and Technology* 42 (2008): 328±333.
- Billings, C. E. Toward a human-centered aircraft automation philosophy. *The International Journal of Aviation Psychology* 1(4) (1991): 261±270.
- Brock, J. D., R. F. Bruce, and M. E. Cameron. Changing the world with a Raspberry Pi. *Journal of Computing Sciences in Colleges* 29(2) (2013): 151±153.
- Chandrakasan, A. P., W. J. Bowhill, and F. Fox. *Design of High-Performance Microprocessor Circuits*. New York: Wiley-IEEE Press, 2000.
- Chao, H. Y., Y. C. Cao, and Y. Q. Chen. Autopilots for small unmanned aerial vehicles: A survey. *International Journal of Control, Automation and Systems* 8(1) (2010): 36±44.
- Chia-Chun, T. USB hub with wireless communication function. U.S. Patent Application 10/995,361, filed November 24, 2004.
- Ciurana, E. *Developing with Google App Engine*. Berkeley, CA: Apress, 2009.
- Cobb, S., D. Lawrence, J. Christie, T. Walter, Y. C. Chao, D. Powell, and B. Parkinson. Observed GPS signal continuity interruptions. In *Proceedings of Ion GPS*, Vol. 8, pp. 793±795. Institute of Navigation, 1995. ION-GPS-95, Palm Springs, California.
- Cooper, M. G., E. M. Elliott, and D. A. Hartzell. Autopilot flight director system. U.S. Patent 4,644,538, issued February 17, 1987.
- De Santis, A., B. Siciliano, A. De Luca, and A. Bicchi. An atlas of physical human±robot interaction. *Mechanism and Machine Theory* 43(3) (2008): 253±270.
- Farrell, J. *Aided Navigation: GPS with High Rate Sensors*. New York: McGraw-Hill, 2008.
- Fitzgerald, B. and T. Kenny. Open source software the trenches: Lessons from a large-scale OSS implementation. *ICIS 2003 Proceedings* (2003): 27.

- Geiger, W., J. Bartholomeyczik, U. Breng, W. Gutmann, M. Hafen, E. Handrich, M. Huber et al. MEMS IMU for AHRS applications. In *Position, Location and Navigation Symposium, 2008 IEEE/ION*, Monterey, CA, pp. 225±231. IEEE, 2008.
- Graf, B. and O. Barth. Entertainment robotics: Examples, key technologies and perspectives. *Safety* 6(7) (2002): 8.
- Groover, M. P., M. Weiss, and R. N. Nagel. *Industrial Robotics: Technology, Programming and Application*. New York: McGraw-Hill Higher Education, 1986.
- Haga, Y., K. Kazami, Y. Sawajiri, S. Suda, and M. Sato. Insect robot. U.S. Patent 6,681,150, issued January 20, 2004.
- Hoffmann, F., N. Goddemeier, and T. Bertram. Attitude estimation and control of a quadcopter. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Taipei, Taiwan, pp. 1072±1077. IEEE, 2010.
- Iizuka, K., H. Uzuhashi, M. Kano, T. Endo, and K. Mohri. Microcomputer control for sensorless brushless motor. *IEEE Transactions on Industry Applications* 3 (1985): 595±601.
- Iovine, J. *PIC Microcontroller Project Book*. New York: McGraw-Hill, 2004.
- Kim, Y.-J. I2C communication system and method enabling bi-directional communications. U.S. Patent Application 11/028,319, filed January 4, 2005.
- Leens, F. An introduction to I2C and SPI protocols. *IEEE Instrumentation & Measurement Magazine* 12(1) (2009): 8±13.
- Lewis, F. L., C. T. Abdallah, and D. M. Dawson. *Control of Robot Manipulators*, Vol. 236. New York: Macmillan, 1993.
- Long, X.-L., J.-P. Jiang, and K. Xiang. Towards multirobot communication. In *IEEE International Conference on Robotics and Biomimetics, 2004 (ROBIO 2004)*, Shenyang, China, pp. 307±312. IEEE, 2004.
- Luukkonen, T. Modelling and control of quadcopter. Independent Research Project in Applied Mathematics, Espoo, Finland, 2011.
- Madan, L., K. Anand, and B. Bhushan. Real-time operating system. *International Journal of Research in Science and Technology* 4, 39±50, 2014.
- Marsh, J. Automated plant watering system. U.S. Patent 8,584,397, issued November 19, 2013.
- Mitra, S. and B. Land. Autonomous quadcopter docking system. Ithaca, NY: Cornell University, 2013.
- Monk, S. *Programming the Raspberry Pi: Getting Started with Python*. McGraw-Hill, 2013a.
- Monk, S. *Raspberry Pi Cookbook*. O'Reilly Media, Inc., 2013b.
- Murray, C. and C. Soric. Home automation system. U.S. Patent 7,082,339, issued July 25, 2006.
- Nelson, T. G., J. R. Brooks, and W. Woo. Instrument communication through signal jacks. U.S. Patent 6,098,095, issued August 1, 2000.
- Nof, S. Y. *Handbook of Industrial Robotics*, Vol. 1. New York: John Wiley & Sons, 1999.
- Nonami, K., F. Kendoul, S. Suzuki, W. Wang, and D. Nakazawa. *Autonomous Flying Robots: Unmanned Aerial Vehicles and Micro Aerial Vehicles*. Springer Science & Business Media, 2010.
- Patel, H. P., M. B. Thakor, V. J. Joshi, J. R. Patel, and D. Patel. Design and development of multi-copter. *International Journal for Innovative Research in Science and Technology* 1(11) (2015): 166±169.
- Pawar, A., S. G. Joshi, and V. P. Sulakhe. Development of autopilot for VTOL application.

- Pearce, J. M. Building research equipment with free, open-source hardware. *Science* 337(6100) (2012): 1303±1304.
- Piyabongkarn, D., R. Rajamani, and M. Greminger. The development of a MEMS gyroscope for absolute angle measurement. *IEEE Transactions on Control Systems Technology* 13(2) (2005): 185±195.
- Predko, M. *Programming and Customizing the PC Microcontroller*. McGraw-Hill, 1997.
- Purohit, A. and P. Zhang. SensorFly: A controlled-mobile aerial sensor network. In *Proceedings of the Seventh ACM Conference on Embedded Networked Sensor Systems*, New York, pp. 327±328. ACM, 2009.
- Richardson, M. and S. Wallace. *Getting Started with Raspberry Pi*. O'Reilly Media, Inc., 2012.
- Singer, P. W. Military robotics and ethics: A world of killer apps. *Nature* 477(7365) (2011): 399±401.
- Spasov, P. *Microcontroller Technology: The 68HC11*. Upper Saddle River, NJ: Prentice-Hall, 1998.
- Sturza, M. A. GPS navigation using three satellites and a precise clock. *Navigation* 30(2) (1983): 146±156.
- Sze, S. M. (ed.). *Semiconductor Sensors*, Vol. 55. New York: Wiley, 1994.
- Szekacs, A., T. Szakáill, and Z. Hegyközi. Realising the SPI communication in a multiprocessor system. In *Fifth International Symposium on Intelligent Systems and Informatics, 2007 (SISY 2007)*, Subotica, Serbia, pp. 213±216. IEEE, 2007.
- Taylor, R. H. and D. Stoianovici. Medical robotics in computer-integrated surgery. *IEEE Transactions on Robotics and Automation* 19(5) (2003): 765±781.
- Townsend, C. and S. Arms. *Wireless Sensor Networks*. Williston, VT: MicroStrain Inc., 2005.
- Upton, E. and G. Halfacree. *Raspberry Pi User Guide*. John Wiley & Sons, 2014.
- Xu, Y. and T. Kanade. *Space Robotics: Dynamics and Control*. Springer Science & Business Media, 1993.
- Zhanfang Yin, Yang Fuguang, Li Yibin, and Liu Wenjiang. High efficiency UART communication based on DMA controller in ARM processor and its application. *Microcomputer Information* 2 (2008): 067.
- Zhang, J., W. Liu, and Y. Wu. Novel technique for vision-based UAV navigation. *IEEE Transactions on Aerospace and Electronic Systems* 47(4) (2011): 2731±2741.
- Zhong, W., Y. Bao, Y. Fu, and Y. Li. Some suggestions on the embedded system education. *Research and Exploration in Laboratory* 12 (2006): 029.

Web References

- <http://3drobotics.com/wp-content/uploads/2013/07/APM-2.6-web-version.pdf>.
- <http://arxiv.org/pdf/1503.02718.pdf>.
- <http://beagleboard.org/beagleboard-xm>.
- <http://blog.alvarolopez.net/en/2012/09/telemetria-y-data-logger-with-arduino-part-ii/>.
- <http://code.google.com/p/multiwii/source/browse/trunk/MultiWii/config.h?r=1187>.
- <http://conservationdrones.org/mission-planner/>.
- http://copter.ardupilot.com/wiki/ac2_simple_geofence/.
- <http://diydrones.com/forum/topics/gps-glitch-and-loss-of-control>.
- <http://diydrones.com/forum/topics/quadcopter-frame-location-of>.

<http://docs.oracle.com/javase/7/docs/api/java/util/StringTokenizer.html>.
<http://image.helipal.com/helipal-hobbywing-skywalker-quattro-20a.pdf>.
<http://jarviestudios.com/blog/2013/03/200-beginner-tips-for-quadcopters-and-the-dji-phantom/>.
<http://opensource.com/resources/what-open-source>.
<http://opensource.org/licenses>.
<http://pandaboard.org/>.
<http://playground.arduino.cc/ComponentLib/Servo>.
<http://quadcopter-robotics.blogspot.in/2015/02/quanum-nova-cx-20-with-apm-fc-pilot-tips.html>.
<http://quadquestions.com/blog/2015/02/14/fpv-video-transmitter-selection-guide/>.
<http://uavcoach.com/how-to-fly-a-quadcopter-guide/>.
<http://www.airspacemag.com/flight-today/the-man-who-invented-the-predator-3970502/?no-ist>.
<http://www.economist.com/node/7001829>.
<http://www.hobbyking.com/hobbyking/store/uploads/811103388X7478X20.pdf>.
<http://www.mutiwii.com/software>.
<http://www.robotshop.com/media/files/pdf/user-manual-kk-control-package.pdf>.
http://www.spektrumrc.com/ProdInfo/Files/SPMR5510-Manual_EN.pdf.
<https://docs.google.com/file/d/0B0zolo4QEjsHM0tSV3VpTUhBWU0/edit?pli=1>.
<https://github.com/digistump/DigisparkArduinoIntegration/blob/master/libraries/DigisparkVirtualWire/VirtualWire.h>.
<https://github.com/google/gdata-python-client>.
<https://github.com/multiwii/multiwii-firmware>.
<https://personal.xively.com/dev/docs/api/>.
<https://www.arduino.cc/>.
<https://www.robots.com/articles/viewing/robotics-safety>.

Embedded Systems and Robotics with Open Source Tools

provides easy-to-understand and easy-to-implement guidance for rapid prototype development. Designed for readers unfamiliar with advanced computing technologies, this highly accessible book:

- Describes several cutting-edge open-source software and hardware technologies
- Examines a number of embedded computer systems and their practical applications
- Includes detailed projects for applying rapid prototype development skills in real time

Embedded Systems and Robotics with Open Source Tools

effectively demonstrates that, with the help of high-performance microprocessors, microcontrollers, and highly optimized algorithms, one can develop smarter embedded devices.



CRC Press
Taylor & Francis Group
an **informa** business
www.crcpress.com

6000 Broken Sound Parkway, NW
Suite 300, Boca Raton, FL 33487
711 Third Avenue
New York, NY 10017
2 Park Square, Milton Park
Abingdon, Oxon OX14 4RN, UK

K26364

ISBN: 978-1-4987-3438-7



www.crcpress.com

